

Processes

Definition

A **process** is a running program. Modern operating systems use processes to manage many activities happening apparently at the same time.

1. Why Processes Are Needed

Main Idea

Modern computers often do several things at once. To manage these activities properly, the operating system needs a way to model and control **concurrency**. Processes provide that model.

Concurrency

Concurrency means multiple activities are in progress during the same period of time, even if one CPU is actually executing only one instruction at any instant.

2. Example: Web Server

Web Server Example

A web server receives requests from many users asking for web pages.

What Happens

When a request arrives:

- the server checks whether the page is in cache,
- if it is, the page is sent back quickly,
- if it is not, a disk request is started.

Problem

Disk operations are very slow from the CPU's point of view. While the system is waiting for one disk request to finish, many more requests may arrive.

Need for Processes

So the server must handle multiple outstanding activities at the same time. Processes, and especially threads, help manage this concurrency.

3. Example: User PC

PC Example

A personal computer also performs many activities at once, often without the user noticing them.

Hidden Processes

After booting, the system may automatically start processes for:

- checking incoming email,
- updating antivirus definitions,
- printing files,
- backing up photos,
- running web browsing tasks.

Need for Management

All of these activities must be managed properly. Therefore, a **multiprogramming system** supporting multiple processes is very useful.

4. Multiprogramming

Definition

Multiprogramming is a system in which multiple processes are kept in memory and the CPU switches among them rapidly.

CPU Switching

In a multiprogramming system:

- the CPU runs one process for a short time,
- then switches to another process,
- then switches again, and so on.

Time Slice

Each process may run for only tens or hundreds of milliseconds before the CPU moves to another process.

5. Illusion of Parallelism

Important Idea

Strictly speaking, on a single-CPU system only **one process** runs at any exact instant. However, because the CPU switches so fast, it appears that many processes are running at the same time.

Pseudoparallelism

This apparent simultaneous execution is called **pseudoparallelism**.

Meaning

Pseudoparallelism gives the illusion of parallel work even though only one process is actually using the CPU at one instant.

6. True Parallelism

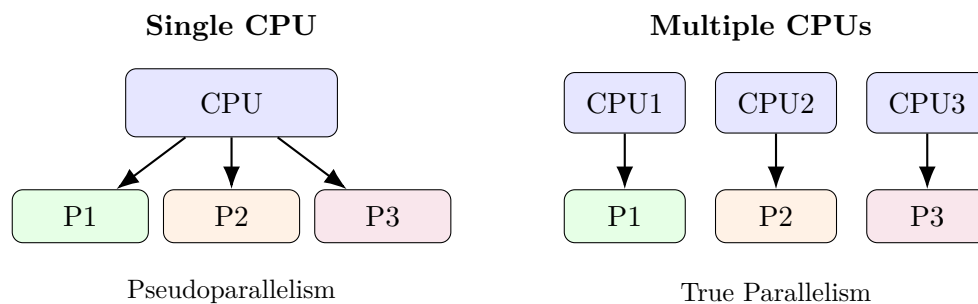
Real Hardware Parallelism

True parallelism occurs in multiprocessor systems, where two or more CPUs share the same physical memory and can actually run processes simultaneously.

Difference

- **Pseudoparallelism:** one CPU, rapid switching
- **True parallelism:** multiple CPUs, real simultaneous execution

6.1 Simple Comparison Diagram



7. Difficulty of Managing Parallel Activities

Human Difficulty

Keeping track of many parallel or seemingly parallel activities is difficult for people.

Why a Model is Needed

Because parallel activity is hard to understand directly, operating-system designers developed a conceptual model called **sequential processes**.

8. Sequential Process Model

Conceptual Model

A **sequential process** is a model in which each process is viewed as if it were executing its own sequence of instructions step by step.

Purpose

This model makes concurrency easier to understand, design, and manage.

9. Importance of Processes

Why Processes Matter

Processes are important because they help the operating system:

- organize multiple activities,
- share CPU time among tasks,
- handle concurrency,
- provide the illusion of several things happening together.

10. Summary Points

Quick Review

- A process is a running program.
- Modern computers handle many activities at once.
- Multiprogramming helps manage multiple processes.
- On one CPU, fast switching creates pseudoparallelism.
- On multiple CPUs, true parallelism is possible.
- The sequential-process model makes concurrency easier to handle.

The Process Model

Definition

In the **process model**, all runnable software in the computer is organized as a number of **sequential processes**. A process is an instance of a program in execution, together with the current values of its program counter, registers, and variables.

1. Basic Idea

Main Concept

Conceptually, each process has its own **virtual CPU**. In reality, the real CPU switches rapidly from one process to another, but it is easier to understand the system by imagining many processes running in parallel.

Multiprogramming

This rapid switching among processes is called **multiprogramming**. It creates the illusion that multiple processes are progressing at the same time.

2. Processes and Virtual CPU

Virtual CPU

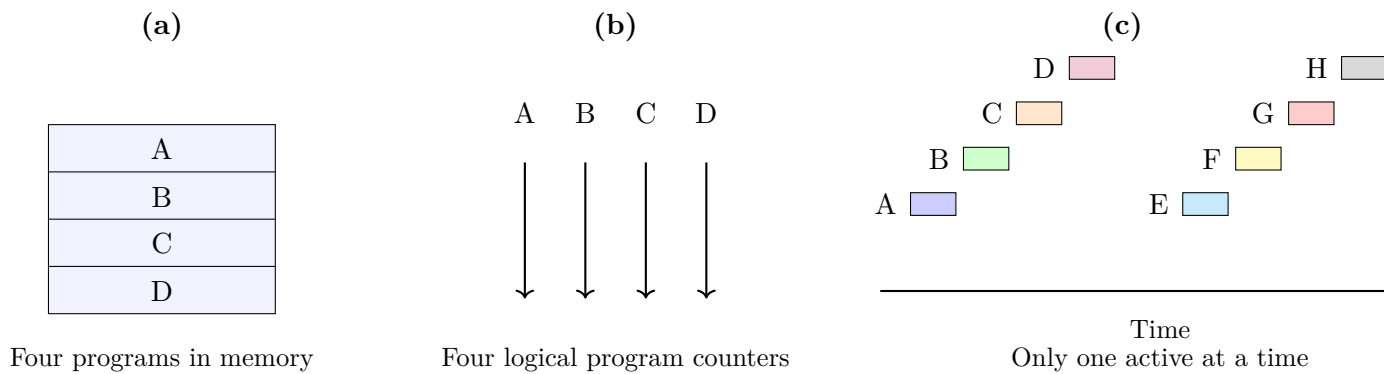
Each process appears to have its own CPU and its own flow of control. This is only a conceptual model, but it helps explain how the operating system manages many activities together.

Reality

Actually:

- there is only one real CPU in the simple model,
- only one real program counter exists,
- when one process runs, its logical program counter is loaded into the real CPU,
- when it stops, that value is saved again.

2.1 Conceptual Process Model



3. Single CPU Assumption

Simplifying Assumption

To understand the basic idea, we often assume that the system has only **one CPU**. In reality, many modern systems have multicore processors, but the single-CPU model is easier to study first.

Important Note

Even if a computer has two or more cores, each individual core can still execute only one process at a time.

4. Timing and Process Execution

Nonuniform Execution

Because the CPU switches among many processes, a process does not run at a perfectly steady rate. Its progress may vary from one execution to another.

Programming Rule

Processes should not be written with fixed assumptions about timing. A process may be interrupted and resumed later, so timing based only on CPU loops is unreliable.

Audio-Video Example

Suppose an audio process wants to wait a little before starting sound to match a video. If it simply runs a loop 10,000 times as a delay, the CPU may switch to another process during that loop. Then the sound may start too late and become out of sync with the video.

Real-Time Requirement

If a process must perform an action within a specific number of milliseconds, special real-time measures are needed to guarantee correct timing.

5. Process vs Program

Key Difference

The difference between a **process** and a **program** is very important:

- a **program** is passive, stored on disk,
- a **process** is active, currently executing.

Short Meaning

A program is just code. A process is the actual activity of running that code.

6. Cake Analogy

Cake Example

Consider a person baking a cake:

- the recipe is the **program**,
- the baker is the **processor**,
- the ingredients are the **input data**,
- the baking activity is the **process**.

Interruption Example

If the baker stops making the cake to give first aid to someone, the current state of the cake-making activity must be remembered. Later, the baker returns and continues from the same point. This is similar to the CPU switching from one process to another and then back again.

7. What a Process Has

Components of a Process

A process has:

- a program,
- input data,
- output,
- a current state,
- the current program counter,
- register values,
- variable values.

8. Scheduling

Scheduling

Since one processor may be shared among several processes, the operating system uses a **scheduling algorithm** to decide when to stop one process and run another.

Purpose

Scheduling helps the system share the CPU fairly and efficiently among many processes.

9. One Program, Multiple Processes

Important Fact

If the same program runs twice, it counts as **two different processes**.

Example

A user may:

- open the same word processor twice,
- print two files at the same time,
- or run the same command in multiple windows.

Even if the program code is the same, these are separate processes.

Code Sharing

The operating system may keep only one copy of the program code in memory and let both processes share it, but conceptually they are still two different processes.

10. Summary Points

Quick Review

- A process is an executing instance of a program.
- Each process is viewed as if it has its own virtual CPU.
- CPU switching among processes is called multiprogramming.
- Timing of process execution is not uniform.
- A process is active, but a program is passive.
- Running the same program twice creates two separate processes.

Process Creation

Definition

Process creation is the mechanism by which the operating system starts a new process to perform system work, run a user program, or execute a scheduled job. In practice, a new process is usually created because an **existing process** requests it through a system call.

1. Why Process Creation Is Needed

Main Idea

General-purpose operating systems cannot keep every possible process loaded from the beginning. They must create and terminate processes dynamically as work appears.

Simple vs General-Purpose Systems

- In very simple or single-purpose systems, all required processes may already be present when the system boots.
- In modern general-purpose systems, new processes must be created whenever users, services, or jobs need them.

2. Four Main Events That Create Processes

Process-Creation Triggers

Four principal events cause processes to be created:

1. **System initialization**
2. **A running process makes a process-creation system call**
3. **A user requests a new program**
4. **A batch system starts the next queued job**

3. Creation at System Boot

Boot-Time Processes

When the operating system starts, it usually creates many processes automatically.

Types of Boot-Time Processes

- **Foreground processes:** interact directly with users.
- **Background processes:** perform system services without direct user interaction.

Daemons

Background service processes that wait for events and wake up when needed are called **daemons**. They may handle email, web requests, printing, news delivery, and similar tasks.

Examples

- In UNIX-like systems, running processes can be listed with `ps`.
- In Windows, running processes can be viewed using **Task Manager**.

4. Creation by a Running Process

Purpose

A running process may create one or more additional processes to divide work into smaller, related tasks.

Why This Helps

- It separates responsibilities cleanly.
- It lets multiple activities progress in parallel.
- On multiprocessor systems, different processes may run on different CPUs and improve performance.

Example

One process may fetch a large amount of data from a network and place it into a shared buffer, while another process removes the data from that buffer and processes it. This cleanly separates input work from computation.

5. Creation by User Request

Interactive Systems

Users often create a new process by typing a command in a shell or by clicking an application icon.

UNIX vs Windows Behavior

- In command-based UNIX systems, the new process often takes over the window or terminal in which it was started.
- In Windows, a newly created process does not necessarily begin with a window, but it may create one or more windows afterward.

Practical Result

Users can keep multiple windows open at the same time, each associated with a different process, and switch between them as needed.

6. Creation in Batch Systems

Batch Jobs

In large batch systems, users submit jobs to a queue. When the operating system decides that enough resources are available, it creates a new process and runs the next job.

Where This Appears

This model is especially common in large mainframe and scheduled-processing environments.

7. How Process Creation Happens Internally

Key Mechanism

In every case, a new process is created because an **existing process** executes a **process-creation system call**.

Who May Make the Call?

The caller may be:

- a running user process,
- a system process started by keyboard or mouse activity,
- or a batch manager process.

What the System Call Tells the OS

The call instructs the operating system:

- to create a new process, and
- to determine which program will run inside that new process.

8. UNIX Process Creation

UNIX Model

In UNIX and most POSIX-style systems, process creation usually happens in two steps:

- `fork()` creates the child process.
- `execve()` loads a new program into that child.

8.1 What `fork()` Does

Initial State After `fork()`

`fork()` creates a child that is initially an almost exact clone of the parent. Right after the call:

- parent and child both exist,
- both start with the same memory image,
- both inherit the same environment strings,
- both inherit the same open files.

Why Use Two Steps?

The two-step model lets the child adjust its file descriptors after `fork()` but before `execve()`. This is essential for input, output, and error redirection.

Shell Example

When a user types a command such as `sort`, the shell typically:

- calls `fork()` to create a child,
- then the child calls `execve()` to run `sort`.

9. Windows Process Creation

Windows Model

Windows uses a single Win32 function, `CreateProcess()`, to both create the process and load the target program.

What `CreateProcess()` Can Specify

The call can include:

- the program to execute,
- command-line parameters,
- security attributes,
- whether open files are inherited,
- priority information,
- window settings,
- and a structure used to return information about the new process.

Key Difference

Windows combines process creation and program loading into one API call, while UNIX usually separates them into `fork()` and `execve()`.

10. Address Space and Resource Sharing

Important Rule

After a process is created, parent and child have **distinct address spaces**. A change made in one process's memory is not directly visible in the other.

UNIX Memory Behavior

In UNIX, the child's initial address space begins as a copy of the parent's. However, the two are still separate processes with separate writable memory.

Copy-on-Write

Many UNIX systems use **copy-on-write**. This means memory is not copied immediately. Instead, a memory region is copied only when one of the processes tries to modify it.

What May Still Be Shared

Although writable memory is not shared by default, some resources may still be shared, such as:

- open files,
- read-only program text,
- and selected operating-system-managed resources.

Windows Memory Behavior

In Windows, parent and child begin with different address spaces from the start.

11. UNIX vs Windows Summary

System	Creation Method	Main Idea
UNIX POSIX	/ <code>fork() + execve()</code>	First create a child process, then usually replace the child's memory image with a new program.
Windows	<code>CreateProcess()</code>	One call both creates the process and loads the requested program.

12. Quick Review

Exam-Focused Points

- Process creation allows the OS to start new execution units as work appears.
- The four main triggers are boot, system call, user request, and batch-job initiation.
- **Daemons** are background processes that wait for events and provide services.
- UNIX usually uses `fork()` followed by `execve()`.
- Windows usually uses `CreateProcess()`.
- Parent and child are separate processes with separate writable address spaces.
- **Copy-on-write** improves efficiency by delaying memory copying until modification is necessary.

Process Termination

Definition

Process termination is the point at which a running process stops execution and leaves the system. A process may terminate **voluntarily** after finishing its work or **involuntarily** because of an error or an external kill request.

1. Four Main Reasons for Termination

Termination Cases

A process usually terminates for one of four reasons:

1. **Normal exit** (voluntary)
2. **Error exit** (voluntary)
3. **Fatal error** (involuntary)
4. **Killed by another process** (involuntary)

Reason	Type	Meaning
Normal exit	Voluntary	The process finishes its intended job and asks the OS to terminate it.
Error exit	Voluntary	The process detects a problem itself and exits cleanly.
Fatal error	Involuntary	The OS stops the process because of an illegal or invalid action.
Killed by another process	Involuntary	Another authorized process explicitly requests its termination.

2. Normal Exit

Main Idea

Most processes terminate normally because they have completed the work they were created to do.

Examples

- A compiler terminates after compiling the source program.
- A word processor or browser terminates when the user chooses the close or exit option.

System Calls

To terminate normally, a process typically uses a system call:

- UNIX: `exit`
- Windows: `ExitProcess`

Clean Shutdown

Interactive programs often remove temporary files, release resources, and then terminate cleanly when the user asks them to close.

3. Error Exit

Meaning

An **error exit** happens when the process itself detects that it cannot continue correctly and chooses to terminate.

Compiler Example

Suppose a user enters:

```
cc foo.c
```

If `foo.c` does not exist, the compiler reports the problem and exits.

Interactive Programs

Screen-oriented interactive applications often handle bad input differently. Instead of terminating, they usually display a dialog box and ask the user to try again.

4. Fatal Error

Involuntary Termination

A **fatal error** occurs when the process performs an illegal or invalid action, often because of a bug. In this case, termination is usually forced by the operating system.

Common Causes

- executing an illegal instruction,
- referencing nonexistent memory,
- dividing by zero.

UNIX Behavior

In some systems such as UNIX, a process may request to handle certain errors itself. Then, instead of immediate termination, the operating system sends a **signal** to interrupt the process and let it respond.

5. Killed by Another Process

External Termination

A process may also terminate because another process explicitly asks the operating system to kill it.

Relevant Calls

- UNIX: `kill`
- Windows: `TerminateProcess`

Authorization Rule

The process requesting termination must have the necessary permission to kill the target process.

6. What Happens to Child Processes?

Possible Policy

Some operating systems immediately terminate all child processes when their parent process ends.

Important Distinction

This is **not** the default behavior in UNIX or Windows. In those systems, terminating one process does not automatically kill every process it created.

7. Key Contrast

Voluntary vs Involuntary

- **Voluntary termination:** the process decides to exit on its own.
- **Involuntary termination:** the operating system or another process forces termination.

8. Quick Review

Exam-Focused Points

- A process does not run forever; eventually it terminates.
- The four main reasons are normal exit, error exit, fatal error, and being killed by another process.
- `exit` in UNIX and `ExitProcess` in Windows are used for normal termination.
- Fatal errors usually come from illegal instructions, invalid memory access, or divide-by-zero errors.
- In UNIX, some errors can be handled through signals instead of immediate termination.
- `kill` in UNIX and `TerminateProcess` in Windows are used to terminate another process.
- UNIX and Windows do not automatically kill all child processes when a parent terminates.

Process Hierarchies

Definition

Process hierarchy describes the parent-child relationships formed when a process creates another process. A child process can create more processes, producing a tree-like structure of descendants.

1. Basic Idea

Main Concept

In some operating systems, a process remains associated with the process it created. This creates a **parent-child relationship**, and repeated process creation forms a **hierarchy**.

Key Property

Each process has only **one parent**, but it may have:

- no children,
- one child,
- or many children.

Tree Structure

Because each process has a single parent, process hierarchies are organized like a **tree**, not a network. One process sits above its descendants, and children may create further descendants of their own.

2. UNIX Process Hierarchies

UNIX View

In UNIX, a process and all of its children and later descendants form a structured hierarchy. Related processes are often treated together as part of a **process group**.

Process Group

A **process group** in UNIX is a set of related processes that can receive signals together. This is especially important for command-line sessions.

Keyboard Signals

When a user sends a signal from the keyboard, the signal is usually delivered to all processes in the process group currently attached to that terminal or window.

How a Process Responds

After receiving a signal, each individual process may:

- catch the signal and handle it,
- ignore the signal,
- or take the default action, which is often termination.

3. UNIX System Startup Example

How UNIX Builds the Process Tree

After the computer boots, UNIX starts a special process called **init**. This process helps build the system's overall process hierarchy.

Startup Sequence

The basic sequence is:

1. **init** starts running from the boot image.
2. It reads configuration information, such as how many terminals exist.
3. It creates one process for each terminal.
4. Each of those processes waits for a user to log in.
5. After a successful login, the login process starts a shell.
6. The shell may create more processes when the user runs commands.

Result

Because each process is created by an earlier one, all processes in the UNIX system can be viewed as part of one large tree, with **init** at the root.

4. Windows and Process Hierarchies

Important Difference

Windows does **not** have a true process hierarchy in the UNIX sense. All processes are treated as fundamentally equal.

Parent Handle

When a Windows process creates another process, the parent receives a special token called a **handle**. This handle can be used to control the child process.

Why This Is Not a Real Hierarchy

The parent can pass that handle to another process. Once that happens, the original parent-child control relationship is no longer strict, so the hierarchy is not preserved in the same way as in UNIX.

UNIX vs Windows

In UNIX, a process cannot simply disinherit its children. In Windows, control can effectively be transferred through handles, so the parent-child relationship is weaker.

5. UNIX vs Windows Summary

System	Hierarchy Model	Key Idea
UNIX	True process tree	Parent-child relationships remain meaningful, and all system processes can be viewed as descendants in one tree.
Windows	No strict hierarchy	Processes are equal; control over a child depends on handles, which can be passed to other processes.

6. Why Process Hierarchies Matter

Practical Importance

Process hierarchies help the operating system:

- organize related processes,
- manage groups of processes together,
- deliver signals to a whole process group,
- and describe system startup in a clear tree structure.

7. Quick Review

Exam-Focused Points

- A process hierarchy forms when parent and child processes remain associated.
- A process has only one parent, but it may have many children.
- In UNIX, processes and their descendants form a meaningful tree structure.
- UNIX process groups allow keyboard signals to be delivered to related processes together.
- Each UNIX process may catch, ignore, or take the default action for a signal.
- During UNIX startup, **init** acts as the root of the system's process tree.
- Windows does not maintain a strict process hierarchy like UNIX.
- In Windows, handles can be passed to other processes, weakening the original parent-child relationship.

Process States

Definition

Process states describe the condition a process is in at a given moment, especially whether it is currently executing, waiting for the CPU, or unable to continue until some external event occurs.

1. Why Process States Matter

Main Idea

Although each process has its own program counter and internal state, processes often interact with one another. Because of this interaction, a process may sometimes have to wait instead of continuing immediately.

Pipeline Example

Consider the shell command:

```
cat chapter1 chapter2 chapter3 | grep tree
```

Here:

- **cat** produces output by concatenating files,
- **grep** consumes that output and searches for lines containing **tree**.

If **grep** is ready to run but no input has arrived yet, it cannot continue and must wait.

2. The Three Basic States

Three-State Model

A process can be in one of three basic states:

1. **Running**: the process is currently using the CPU.
2. **Ready**: the process can run, but the CPU is temporarily assigned to another process.
3. **Blocked**: the process cannot run until some external event occurs.

State	Meaning	Can it run immediately?
Running	The process is actively executing.	Yes
Ready	The process is able to run, but is waiting for CPU time.	Yes, when scheduled
Blocked	The process is waiting for input or another external event.	No

3. Ready vs Blocked

Core Distinction

Ready and **blocked** are not the same:

- A **ready** process is willing and able to run, but has no CPU right now.
- A **blocked** process cannot proceed even if the CPU is free.

Why This Difference Matters

The first case is a CPU allocation issue. The second case is a logical waiting issue. For example, a process cannot process keyboard input until the user has actually typed something.

4. The Four State Transitions

Transition Overview

There are four main transitions between the three states.

No.	Transition	Cause
1	Running → Blocked	The process cannot continue and must wait for input or another event.
2	Running → Ready	The scheduler removes the process from the CPU to let another process run.
3	Ready → Running	The scheduler chooses that process to use the CPU.
4	Blocked → Ready	The awaited external event happens, so the process becomes runnable again.

4.1 Transition 1: Running to Blocked

Why It Happens

This transition occurs when the operating system discovers that the process cannot continue right now.

Examples

- In some systems, the process may call a system call such as **pause** to block itself.
- In UNIX, if a process reads from a pipe or a special file like a terminal and no input is available, it is blocked automatically.

4.2 Transitions 2 and 3: Scheduler-Controlled Changes

Scheduler Action

Transitions 2 and 3 are caused by the **process scheduler**, a part of the operating system. These transitions happen without the process controlling them directly.

Running to Ready

Transition 2 happens when the scheduler decides that the running process has used the CPU long enough and should give another process a turn.

Ready to Running

Transition 3 happens when the scheduler selects a ready process and gives it the CPU again.

Scheduling Goal

Scheduling tries to balance two competing goals:

- **efficiency** for the whole system,
- **fairness** for individual processes.

4.3 Transition 4: Blocked to Ready

External Event

Transition 4 happens when the event a process was waiting for finally occurs, such as the arrival of input or completion of an I/O operation.

What Happens Next

Once the event occurs, the process becomes **ready**. If the CPU is free, it may run immediately. Otherwise, it waits in the ready state until the scheduler selects it.

5. Why the Process Model Is Useful

Simpler Way to Think

The process model makes the operating system easier to understand. Instead of focusing on low-level interrupts, we can think in terms of user processes, disk processes, terminal processes, and other logical activities.

Disk Interrupt Example

Suppose a process is waiting for a disk read to finish. While waiting, it is blocked. When the disk interrupt occurs, the waiting process is unblocked and becomes eligible to run again.

Conceptual Benefit

This view turns many hardware events into process-state changes, which is much easier to reason about than tracking every interrupt directly.

6. Process-Structured Operating System View

Layered Picture

In a process-structured operating system, the lowest layer handles:

- interrupts,
- scheduling,
- and the mechanics of starting and stopping processes.

Above that layer sit many ordinary sequential processes.

Role of the Scheduler

In this model, the scheduler is the base layer, and the rest of the operating system is organized more cleanly in process form.

Reality Check

Real operating systems are usually not as neatly structured as this simplified model, but the model is still very useful for understanding how systems behave.

7. Quick Review

Exam-Focused Points

- A process can be in one of three states: running, ready, or blocked.
- Running means using the CPU right now.
- Ready means able to run but waiting for CPU time.
- Blocked means unable to continue until an external event happens.
- Ready and blocked are different: ready is waiting for the CPU, blocked is waiting for an event.
- The scheduler causes the running-to-ready and ready-to-running transitions.
- External events such as input arrival cause blocked processes to become ready again.
- The process model helps explain system behavior in a cleaner way than thinking only in terms of interrupts.

Implementation of Processes

Definition

To implement the process model, the operating system keeps a **process table**, usually with one entry for each process. Each entry stores everything needed to stop a process, keep track of it, and later restart it correctly.

1. The Process Table

Main Idea

The **process table** is an array of data structures maintained by the operating system. Each entry describes one process.

Another Name

Many authors call a process-table entry a **process control block (PCB)**.

Why It Is Needed

Whenever a process changes from **running** to **ready** or **blocked**, the operating system must save enough information so the process can later resume as if it had never been interrupted.

2. What the Process Table Stores

Typical Information

A process-table entry usually contains:

- CPU-related state such as the program counter and registers,
- stack information,
- memory-management information,
- open-file information,
- scheduling data,
- accounting data,
- process identity and relationship information.

Process management	Memory management	File management
Registers, program counter, program status word, stack pointer, process state, priority, scheduling parameters, process ID, parent process, process group, signals, start time, CPU time used, children's CPU time, next alarm time	Pointer to text segment information, pointer to data segment information, pointer to stack segment information	Root directory, working directory, file descriptors, user ID, group ID

System Dependent

The exact fields in a process table vary from one operating system to another. The important idea is not the exact list, but that the OS must store all information needed to suspend and resume a process correctly.

3. How One CPU Appears to Run Many Processes

Core Illusion

Even on a single CPU, the operating system creates the illusion of multiple sequential processes by rapidly saving the state of one process and restoring the state of another.

What Must Be Preserved

For this illusion to work, the OS must save the interrupted process state precisely and later restore it precisely.

4. Interrupt Vectors

Interrupt Vector

For each I/O class, the system keeps an **interrupt vector**, usually at a fixed location in memory. It stores the address of the interrupt service procedure that should run when that interrupt occurs.

Example Situation

Suppose **user process 3** is running when a disk interrupt occurs. The interrupt hardware immediately saves a small amount of execution state and transfers control to the interrupt service routine.

5. What the Hardware Does First

Initial Hardware Action

When an interrupt happens, the hardware typically:

- pushes the current program counter onto the current stack,
- pushes the program status word,
- sometimes pushes one or more registers,
- then jumps to the address stored in the interrupt vector.

Key Limit

That is usually all the hardware does. After that, the rest of the work is handled by software, especially the interrupt service procedure.

6. What the Software Does Next

Saving the Current Process

All interrupts begin by saving the processor registers, often into the process-table entry of the currently running process.

Temporary Stack

After that, the information pushed by the hardware is removed from the old stack, and the stack pointer is changed to a temporary stack used by the interrupt handler.

Why Assembly Is Needed

Low-level actions such as saving registers and changing the stack pointer cannot be expressed directly in high-level languages like C, so a short **assembly-language routine** performs them first.

Same for All Interrupts

This low-level assembly routine is often shared by all interrupts, because saving registers is basically the same operation no matter what caused the interrupt.

7. Role of the C Interrupt Handler

Higher-Level Processing

Once the low-level assembly work is finished, it calls a **C procedure** that handles the specific interrupt type.

What This Handler May Do

The C interrupt handler may:

- process the interrupt event,
- update operating-system data structures,
- mark some blocked process as ready,
- request that the scheduler choose the next process to run.

8. Scheduler and Process Restart

Scheduler Step

After the interrupt has been handled, the **scheduler** is called to decide which process should run next.

Restoring the Next Process

Control then returns to low-level code, which:

- loads the registers of the selected process,
- restores its memory map,
- and starts that process running.

Essential Result

If the same process is resumed later, it continues from exactly the same state it had before the interrupt.

9. Interrupt Handling Flow

Typical Sequence

The overall sequence is:

1. A process is running.
2. An interrupt occurs.
3. Hardware saves minimal state and jumps through the interrupt vector.
4. Assembly code saves registers and switches to a temporary stack.
5. A C interrupt handler processes the event.
6. The scheduler decides which process should run next.
7. Low-level code restores that process's registers and memory map.
8. Execution resumes.

10. Why This Works

Key Insight

A process may be interrupted thousands of times while it runs, but after each interrupt it can resume as though nothing happened, because its full execution context was preserved.

Big Picture

This save-and-restore mechanism is what makes process switching, scheduling, and the whole process model work in practice.

11. Quick Review

Exam-Focused Points

- The operating system implements processes using a **process table**.
- Each process-table entry stores everything needed to suspend and resume a process.
- A process-table entry is often called a **process control block (PCB)**.
- The exact table fields are system dependent, but usually cover process, memory, and file management.
- Interrupt vectors store the addresses of interrupt service procedures.
- Hardware saves a small amount of state first, then software does the rest.
- Low-level assembly code saves registers and switches stacks.
- A C interrupt handler processes the interrupt and may wake blocked processes.
- The scheduler selects the next process, and its state is restored.
- A resumed process continues from the same state it had before the interrupt.

Modeling Multiprogramming

Definition

Modeling multiprogramming means estimating how much CPU utilization improves when several processes are kept in memory at the same time.

1. Basic Idea

Main Concept

Multiprogramming improves CPU utilization because while one process is waiting for I/O, another process may be able to use the CPU.

Optimistic Intuition

If each process computes only a small fraction of the time and spends most of its time waiting for I/O, then keeping several processes in memory can keep the CPU busy much more often.

Why the Naive View Fails

The simple idea that “five processes each using the CPU 20% of the time will keep the CPU busy 100% of the time” is too optimistic. It assumes that all processes will never wait for I/O at the same time.

2. Probabilistic Model

Assumption

Suppose a process spends a fraction p of its time waiting for I/O. If there are n processes in memory, then the probability that **all** of them are waiting for I/O at the same time is:

$$p^n$$

CPU Utilization Formula

If all n processes are waiting, the CPU is idle. So the CPU utilization is:

$$\text{CPU utilization} = 1 - p^n$$

Meaning of Symbols

- p : fraction of time one process waits for I/O
- n : number of processes in memory
- p^n : probability that all processes are waiting
- $1 - p^n$: probability that at least one process can use the CPU

3. Effect of Degree of Multiprogramming

Degree of Multiprogramming

The number of processes kept in memory at once is called the **degree of multiprogramming**.

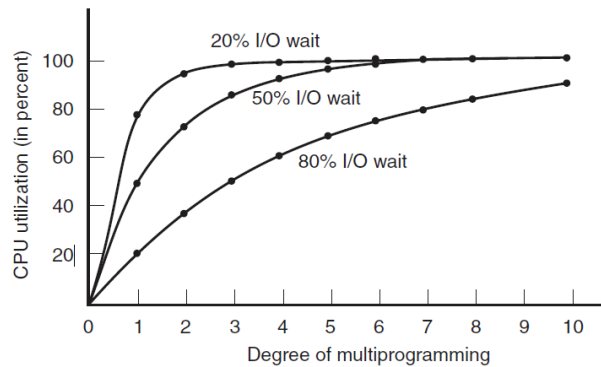


Figure 1: Caption text describing the image

Key Trend

As n increases, p^n becomes smaller, so CPU utilization increases.

Situation	Result
Small n	Higher chance that all processes are waiting, so CPU may be idle more often
Large n	Lower chance that all processes are waiting, so CPU utilization improves
Very high I/O wait (p near 1)	Many processes are needed to keep CPU waste low

4. Why High I/O Wait Is Common

Real Systems

Processes often spend a large fraction of time in I/O wait. This is common not only for disk-heavy server programs, but also for interactive processes waiting for a user to type or click something.

Example

If a process is waiting for user input at a terminal, that time counts as I/O wait. So values such as $p = 0.8$ are not unusual.

5. Worked Example: CPU Utilization

Example

Suppose each process waits for I/O 80% of the time, so:

$$p = 0.8$$

Then:

$$\text{CPU utilization} = 1 - 0.8^n$$

n	Idle Probability p^n	CPU Utilization $1 - p^n$
3	$0.8^3 = 0.512$	$48.8\% \approx 49\%$
7	$0.8^7 \approx 0.210$	79.0%
11	$0.8^{11} \approx 0.086$	91.4%

Interpretation

This shows that adding more processes improves utilization, but the improvement becomes smaller after a while.

6. Memory Expansion Example

Given System

Suppose a computer has:

- 8 GB total memory,
- 2 GB used by the operating system,
- each user program requiring 2 GB.

That leaves room for:

$$\frac{8 - 2}{2} = 3$$

user processes in memory.

Initial Utilization

With $p = 0.8$ and $n = 3$:

$$\text{CPU utilization} = 1 - 0.8^3 \approx 49\%$$

After Adding 8 GB

If another 8 GB of memory is added, total memory becomes 16 GB. Then:

$$\frac{16 - 2}{2} = 7$$

user processes can be kept in memory.

Now:

$$\text{CPU utilization} = 1 - 0.8^7 \approx 79\%$$

Performance Gain

CPU utilization rises from about 49% to 79%, so throughput improves by about 30%.

Adding Another 8 GB

If memory increases again from 16 GB to 24 GB, then:

$$\frac{24 - 2}{2} = 11$$

processes can fit in memory.

Utilization becomes:

$$\text{CPU utilization} = 1 - 0.8^{11} \approx 91\%$$

Diminishing Returns

The second memory upgrade raises utilization only from about 79% to 91%, an improvement of about 12%. This shows **diminishing returns**: the first upgrade helps more than the second.

7. Limits of the Model

Approximation

This probabilistic model is only an approximation. It assumes that all processes are independent.

Why It Is Not Exact

With a single CPU, several processes cannot all run at once. If one process becomes ready while another is already using the CPU, it must wait. So the processes are not completely independent.

More Accurate Approach

A more precise model can be built using **queueing theory**. Even so, the simple model still captures the key idea: multiprogramming reduces CPU idle time.

8. Main Conclusion

Takeaway

Multiprogramming improves performance by letting one process use the CPU while others wait for I/O. The formula $1 - p^n$ gives a simple way to estimate CPU utilization and reason about tradeoffs such as whether adding more memory is worth it.

9. Quick Review

Exam-Focused Points

- Multiprogramming improves CPU utilization by reducing idle time.
- If a process waits for I/O a fraction p of the time, then the probability all n processes are waiting is p^n .
- CPU utilization is approximated by $1 - p^n$.
- n is the degree of multiprogramming.
- High I/O wait values such as 80% are common in interactive and I/O-heavy systems.
- Increasing the number of processes in memory improves utilization.
- The improvement shows diminishing returns as n becomes large.
- The model is approximate because processes are not fully independent on a single CPU.

Thread Usage

Definition

A **thread** is a thread of control that runs inside a process. Unlike separate processes, multiple threads share the same address space and data, but each thread has its own execution flow.

1. Basic Idea

Traditional Process Model

In a traditional operating system, each process has:

- its own address space,
- and one thread of control.

What Threads Add

Threads allow multiple threads of control to exist inside the **same** address space. They run in quasi-parallel and behave almost like separate processes, except that memory is shared.

2. Why Threads Are Useful

Main Reasons

Threads are useful for four main reasons:

1. They simplify programs in which multiple activities happen at the same time.
2. They are lighter weight and faster to create and destroy than processes.
3. They improve performance when computation and I/O can overlap.
4. They allow real parallelism on multiprocessor systems.

Reason	Meaning
Simpler design	Different activities can be written as separate sequential threads instead of one complicated interrupt-driven program.
Lower overhead	Thread creation and destruction are often much faster than process creation and destruction.
Better overlap	One thread can compute while another waits for I/O.
Parallel hardware use	On multi-CPU systems, different threads can run truly in parallel.

Why Processes Alone Are Not Enough

Multiple processes cannot replace threads in some applications because separate processes do **not** share the same address space by default. Threads do.

3. Example 1: Word Processor

Two-Thread Design

Suppose a word processor has:

- one thread for user interaction,
- one background thread for reformatting the document.

Why This Helps

If the user edits page 1 of a very large document, the formatting thread can reformat the book in the background while the interactive thread still responds to scrolling, typing, and mouse actions.

Three-Thread Design

A third thread can periodically save the document to disk for backup, without freezing the interface or interfering with reformatting.

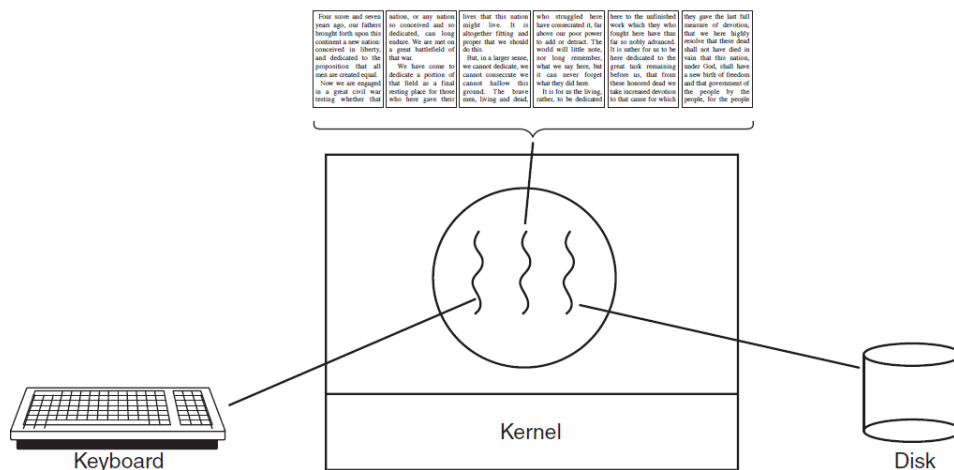


Figure 1: A word processor with three threads.

Why Three Processes Would Fail

Using three separate processes would be inappropriate because all three activities need access to the same document in memory. Threads work because they share that memory.

4. Example 2: Spreadsheet

Spreadsheet Use Case

When a user changes one cell, many other cells may need to be recomputed. A background thread can perform recalculation while the interactive thread still accepts new user actions.

Backup Thread Again

As with a word processor, a third thread can perform periodic disk backups independently.

5. Example 3: Multithreaded Web Server

Server Structure

A common Web server design uses:

- one **dispatcher thread** to receive requests,
- multiple **worker threads** to process them,
- and a shared **Web page cache**.

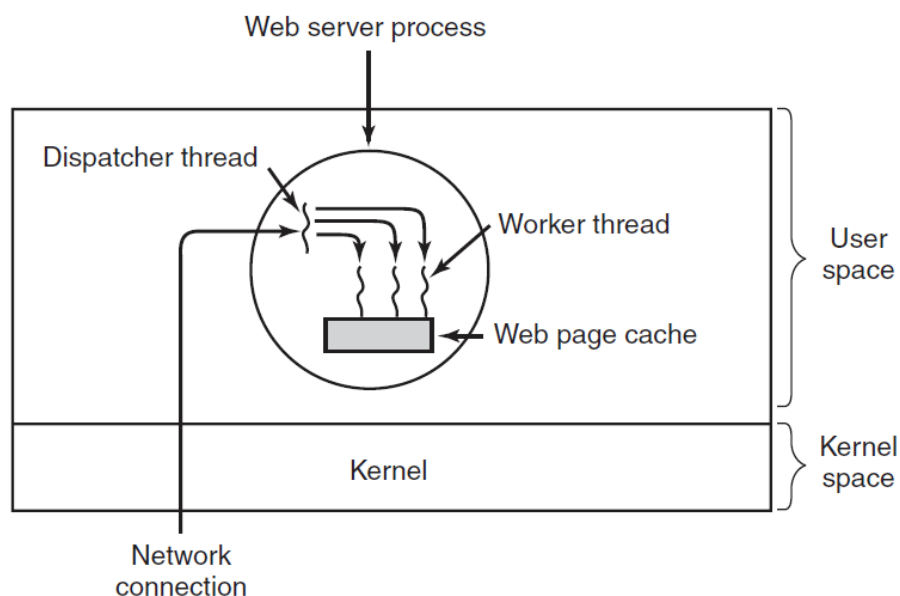


Figure 2: A multithreaded Web server.

How It Works

The dispatcher accepts a request, chooses an idle worker thread, passes the request to it, and wakes it up. The worker checks the shared cache. If the page is missing, it reads from disk and blocks while the disk I/O is in progress.

Performance Benefit

While one worker is blocked on disk I/O, another thread can run. This keeps the server responsive and increases the number of requests handled per second.

5.1 Dispatcher and Worker Outline

Dispatcher Thread Pseudocode

```
while (TRUE) {  
    get_next_request(&buf);  
    handoff_work(&buf);  
}
```

Worker Thread Pseudocode

```
while (TRUE) {  
    wait_for_work(&buf);  
    look_for_page_in_cache(&buf, &page);  
    if (page_not_in_cache(&page)) {  
        read_page_from_disk(&buf, &page);  
    }  
    return_page(&page);  
}
```

Programming Advantage

Each thread can still be written as a simple sequential program, even though the whole server behaves concurrently.

6. Without Threads

Single-Threaded Server

If the Web server has only one thread, then while it waits for disk I/O it cannot process any other request. This wastes CPU time and reduces throughput.

Finite-State Machine Approach

Without threads, one alternative is to use nonblocking I/O and explicitly save the state of each request in a table. This design is called a **finite-state machine**.

Why It Is Harder

In a finite-state machine, the programmer must explicitly save and restore computation state whenever the server switches from one request to another. This is much harder than writing blocking sequential code.

Model	Characteristics
Threads	Parallelism with blocking system calls
Single-threaded process	No parallelism, but still uses blocking system calls
Finite-state machine	Parallelism with nonblocking system calls and interrupts

7. Example 4: Large Data Processing

Three-Thread Pipeline

An application that processes very large amounts of data can be organized with:

- an input thread,
- a processing thread,
- an output thread.

How the Pipeline Works

The input thread fills an input buffer. The processing thread reads from that buffer, processes the data, and writes results to an output buffer. The output thread writes those results to disk.

Key Requirement

This works properly only if a blocking system call blocks the **calling thread**, not the entire process.

8. Main Conclusion

Takeaway

Threads make it possible to keep the simple sequential programming style of blocking calls while still achieving concurrency and better performance. Their biggest advantage is shared memory plus lighter-weight control flows.

9. Quick Review

Exam-Focused Points

- A thread is a thread of control inside a process.
- Multiple threads share the same address space and data.
- Threads are useful when many activities happen at once.
- Threads are lighter weight than processes and are usually faster to create and destroy.
- Threads improve performance when CPU work and I/O can overlap.
- A multithreaded Web server can use a dispatcher thread and several worker threads.
- Threads keep sequential programming simple while still allowing concurrency.
- A finite-state machine can also achieve concurrency, but it is harder to program.
- Threads are ideal when different activities need access to the same shared data.

The Classical Thread Model

Definition

The **classical thread model** separates two ideas that are combined in a traditional process: **resource grouping** and **execution**. A process groups resources, while a thread is the unit of execution scheduled on the CPU.

1. Two Concepts Inside a Process

Resource Grouping

One role of a process is to collect related resources together. A process may contain:

- an address space,
- program text and data,
- open files,
- child processes,
- pending alarms,
- signal handlers,
- accounting information.

Execution

The second role is execution. A **thread** is the entity that actually runs on the CPU. It has:

- a program counter,
- registers,
- and a stack.

Core Separation

Processes are used to group resources together. Threads are the execution units scheduled by the CPU.

2. What Threads Add

Main Idea

Threads allow multiple executions to take place within the same process environment. These threads run largely independently, but they share the same address space and process resources.

Lightweight Processes

Because threads behave in some ways like processes, they are often called **lightweight processes**. The term **multithreading** means allowing multiple threads inside one process.

Hardware Support

Some CPUs provide direct hardware support for multithreading, allowing thread switches to happen extremely quickly.

3. Processes vs Threads

Feature	Traditional Processes	Threads in One Process
Address space	Each process has its own	All threads share one address space
Resources	Separated per process	Shared inside the process
Execution flows	One per process in the simple model	Multiple inside the same process
Cooperation	Harder because memory is separate	Easier because memory is shared

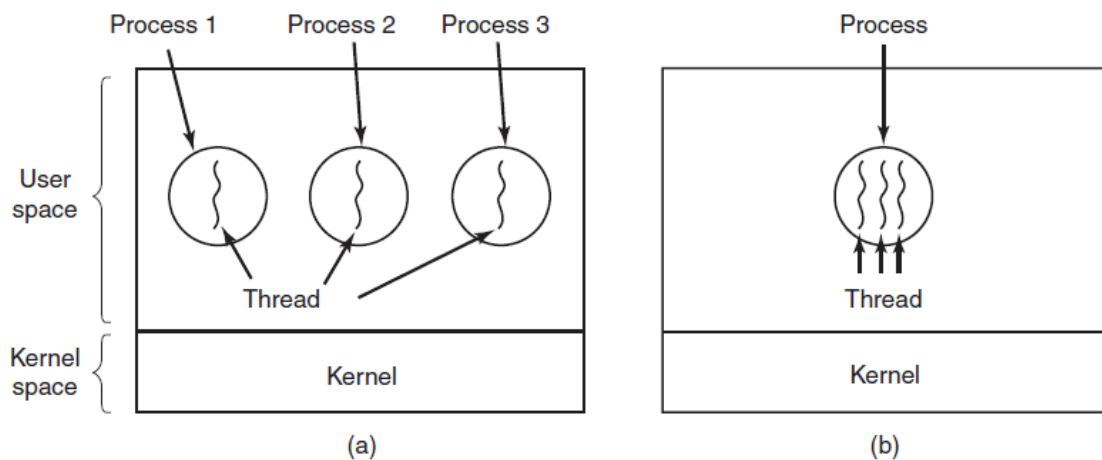


Figure 1: Three processes each with one thread, and one process with three threads.

Important Difference

Threads are not as independent as processes. Since all threads share the same address space, one thread can read or modify data used by another thread.

4. Shared and Private Items

Shared vs Private

Some items belong to the whole process, while others belong to each individual thread.

Per-process items	Per-thread items
Address space, global variables, open files, child processes, pending alarms, signals and signal handlers, accounting information	Program counter, registers, stack, state

Why This Matters

If one thread opens a file, the other threads in the same process can use that file too. This happens because the process, not the thread, is the unit of resource management.

Why No Protection Exists

There is normally no memory protection between threads of the same process because they are expected to cooperate, not compete like unrelated processes.

5. How Threads Run on a Single CPU

Quasi-Parallelism

On a single-CPU system, multiple threads do not literally run at the same instant. Instead, the CPU switches rapidly among them, creating the illusion of parallel execution.

Example

If one process has three compute-bound threads on a single CPU, they appear to run in parallel, but each one effectively gets about one-third of the CPU time.

6. Thread States

States

Like processes, threads can be in several states:

- **Running**
- **Ready**
- **Blocked**
- **Terminated**

Meaning

- A **running** thread currently has the CPU.
- A **ready** thread can run when scheduled.
- A **blocked** thread is waiting for some event.
- A **terminated** thread has finished and is no longer schedulable.

Same Transition Idea

The transitions among thread states are conceptually the same as the transitions among process states.

7. Why Each Thread Needs Its Own Stack

Private Execution History

Each thread has its own stack because each thread may be executing a different sequence of procedure calls.

Stack Frames

Each stack contains one frame for every procedure that has been called but not yet returned from. A frame stores:

- local variables,
- return addresses,
- and related execution data.

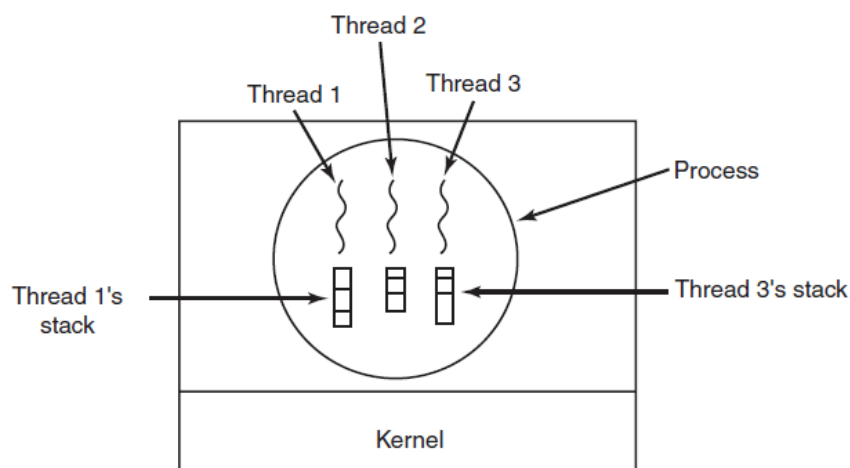


Figure 2: Each thread has its own stack.

Example

If thread A is inside procedures $X \rightarrow Y \rightarrow Z$, while thread B is inside $P \rightarrow Q$, then the two threads have different execution histories and therefore need separate stacks.

8. Basic Thread Operations

Starting Point

A multithreaded process usually begins with one thread. That thread can create additional threads.

Common Thread Calls

Typical thread library operations include:

- `thread_create`
- `thread_exit`
- `thread_join`
- `thread_yield`

Typical Usage

```
thread_create(worker_function);
thread_create(logger_function);

thread_yield();

thread_join(worker_id);
thread_exit();
```

What They Mean

- `thread_create`: starts a new thread in the same address space.
- `thread_exit`: ends the current thread.
- `thread_join`: waits for a specific thread to finish.
- `thread_yield`: voluntarily gives up the CPU.

Polite Scheduling

Voluntary yielding is especially important in thread systems where threads are expected to cooperate and give other threads a chance to run.

9. Complications Introduced by Threads

9.1 Problem 1: Fork

Fork Ambiguity

If a multithreaded process calls `fork`, an important question arises: should the child process receive all of the parent's threads or only the calling thread?

Why This Is Difficult

If only one thread is copied, the program may break because other threads may be essential. If all threads are copied, blocked operations such as keyboard reads or network reads become hard to interpret correctly in both parent and child.

9.2 Problem 2: Shared Data Structures

Race Risks

Because threads share data, one thread can interfere with another in dangerous ways.

Examples

- One thread may close a file while another is still reading from it.
- Two threads may both notice low memory and both start allocating memory, causing duplication or corruption.

Design Requirement

Multithreaded programs can work correctly, but they require careful design because shared memory and shared resources create synchronization problems.

10. Main Conclusion

Takeaway

The classical thread model separates **resource ownership** from **execution**. A process owns resources, while threads execute inside that process. This gives efficient cooperation through shared memory, but it also introduces correctness and synchronization problems.

11. Quick Review

Exam-Focused Points

- A traditional process combines resource grouping and execution.
- The classical thread model separates these two ideas.
- A process owns resources; a thread is the unit scheduled on the CPU.
- Threads in the same process share address space and process resources.
- Each thread has its own program counter, registers, stack, and state.
- Threads can be running, ready, blocked, or terminated.
- Each thread needs its own stack because each has its own execution history.
- Common thread operations include `thread_create`, `thread_exit`, `thread_join`, and `thread_yield`.
- Multithreading introduces complications such as `fork` ambiguity and races on shared data.

POSIX Threads

Definition

POSIX Threads, usually called **Pthreads**, is the IEEE standardized thread library for writing portable multithreaded programs. It is defined by IEEE standard 1003.1c and is supported by most UNIX systems.

1. Why Pthreads Matter

Main Idea

Different operating systems may implement threads differently, so a standard API is needed to write portable thread programs. Pthreads provides that standard interface.

Scope

The full Pthreads standard defines many function calls. In practice, a smaller core set is enough to understand the basic thread model and how Pthreads programs are written.

2. Core Pthreads Calls

Main Calls

Some of the most important Pthreads calls are listed below.

Thread Call	Description
<code>pthread_create</code>	Create a new thread
<code>pthread_exit</code>	Terminate the calling thread
<code>pthread_join</code>	Wait for a specific thread to exit
<code>pthread_yield</code>	Release the CPU to let another thread run
<code>pthread_attr_init</code>	Create and initialize a thread attribute structure
<code>pthread_attr_destroy</code>	Remove a thread attribute structure

3. Properties of a Pthread

What Each Thread Has

Every Pthread has its own identity and execution context.

Typical Properties

A Pthread has:

- a thread identifier,
- a set of registers,
- a program counter,
- an attribute structure.

Thread Attributes

The attribute structure stores information such as:

- stack size,
- scheduling parameters,
- and other thread configuration details.

4. `pthread_create`

Purpose

`pthread_create` creates a new thread.

Key Idea

The newly created thread gets its own thread identifier, much like a new process gets a PID with `fork`. The thread identifier is later used in operations such as `pthread_join`.

Same Address Space

Unlike `fork`, the new thread does not get a new address space. It automatically runs inside the same process and shares the same memory with the creating thread.

5. `pthread_exit`

Purpose

When a thread finishes its assigned work, it can terminate itself by calling `pthread_exit`.

Effect

This stops the calling thread and releases its stack.

6. pthread_join

Purpose

Sometimes one thread must wait for another thread to finish. It does this using `pthread_join`.

Behavior

The calling thread blocks until the specific target thread terminates.

7. pthread_yield

Purpose

`pthread_yield` allows a thread to voluntarily give up the CPU so that another thread can run.

Why This Exists

Processes are usually viewed as competing for CPU time, but threads in the same process are usually cooperating. Because of that, sometimes the programmer wants one thread to deliberately let another thread run.

8. Attribute Functions

Initialization

`pthread_attr_init` creates a thread attribute structure and initializes it with default values.

Customization

After initialization, fields inside the attribute structure can be changed, for example to adjust priority or stack-related settings.

Cleanup

`pthread_attr_destroy` removes the attribute structure and frees the memory used for it. It does **not** destroy threads that are using those attributes.

9. Typical Thread Lifecycle

Stage	Pthreads Action
Prepare attributes	<code>pthread_attr_init</code>
Create thread	<code>pthread_create</code>
Run thread function	thread executes assigned work
Wait if needed	<code>pthread_join</code>
Terminate thread	<code>pthread_exit</code>
Destroy attributes	<code>pthread_attr_destroy</code>

10. Example Program

Main Idea

The textbook example creates several threads in a loop. Each thread prints a greeting message and then exits. The main program creates all threads and then terminates.

Pthreads Example Code

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>

#define NUMBER_OF_THREADS 10

void *print_hello_world(void *tid)
{
    printf("Hello World. Greetings from thread %ld\n", (long) tid);
    pthread_exit(NULL);
}

int main(void)
{
    pthread_t threads[NUMBER_OF_THREADS];
    int status;
    long i;

    for (i = 0; i < NUMBER_OF_THREADS; i++) {
        printf("Main here. Creating thread %ld\n", i);
        status = pthread_create(&threads[i], NULL, print_hello_world, (void *) i);
        ;
        if (status != 0) {
            printf("Oops. pthread_create returned error code %d\n", status);
            exit(-1);
        }
    }

    pthread_exit(NULL);
}
```

What the Program Does

- The main thread creates 10 worker threads.
- Each new thread prints its identifier.
- After printing, each thread exits.
- The main thread also exits after creating the threads.

Nondeterministic Output

The order in which the messages appear is **not fixed**. Different runs may produce different interleavings because thread scheduling is nondeterministic.

11. Why the Example Matters

Programming Style

The example shows the standard Pthreads style:

- define a thread function,
- create threads with `pthread_create`,
- optionally wait for them using `pthread_join`,
- let each thread end using `pthread_exit`.

12. Main Conclusion

Takeaway

Pthreads gives a portable standard interface for multithreaded programming on UNIX-like systems. Its basic operations cover thread creation, termination, waiting, yielding, and attribute management.

13. Quick Review

Exam-Focused Points

- Pthreads is the IEEE POSIX thread standard.
- It is designed to make threaded programs portable across UNIX systems.
- Every thread has an identifier, registers, a program counter, and attributes.
- `pthread_create` creates a new thread.
- `pthread_exit` terminates the calling thread.
- `pthread_join` waits for a specific thread to finish.
- `pthread_yield` voluntarily gives the CPU to another thread.
- `pthread_attr_init` and `pthread_attr_destroy` manage attribute structures.
- Output order in multithreaded programs is often nondeterministic.

Implementing Threads in User Space

Definition

User-level thread implementation means that the entire threads package is managed in user space by a library. The kernel does not know that the threads exist and treats the whole process as a normal single-threaded process.

1. Where Threads Can Be Implemented

Main Choices

Threads can be implemented in:

- user space,
- the kernel,
- or a hybrid of both.

This Section

This section focuses only on **user-space threads**, where a run-time library performs all thread management.

2. Basic Idea of User-Level Threads

Kernel View

When threads are implemented entirely in user space, the kernel knows nothing about them. From the kernel's point of view, it is simply managing ordinary processes.

How It Is Built

The threads package is implemented by a **library** plus a **run-time system**. This run-time system provides operations such as:

- `pthread_create`,
- `pthread_exit`,
- `pthread_join`,
- `pthread_yield`,
- and usually several others.

Major Immediate Benefit

User-level threads can be used even on operating systems that do not support threads in the kernel.

3. Run-Time System and Thread Table

Private Thread Table

Each process must maintain its own private **thread table**. This table is managed by the run-time system, not by the kernel.

What the Thread Table Stores

The thread table is similar to a process table, but only for per-thread state such as:

- program counter,
- stack pointer,
- registers,
- thread state,
- and related execution information.

How It Is Used

When a thread becomes **ready** or **blocked**, the information needed to restart it later is saved in the thread table, just as the kernel saves process information in a process table.

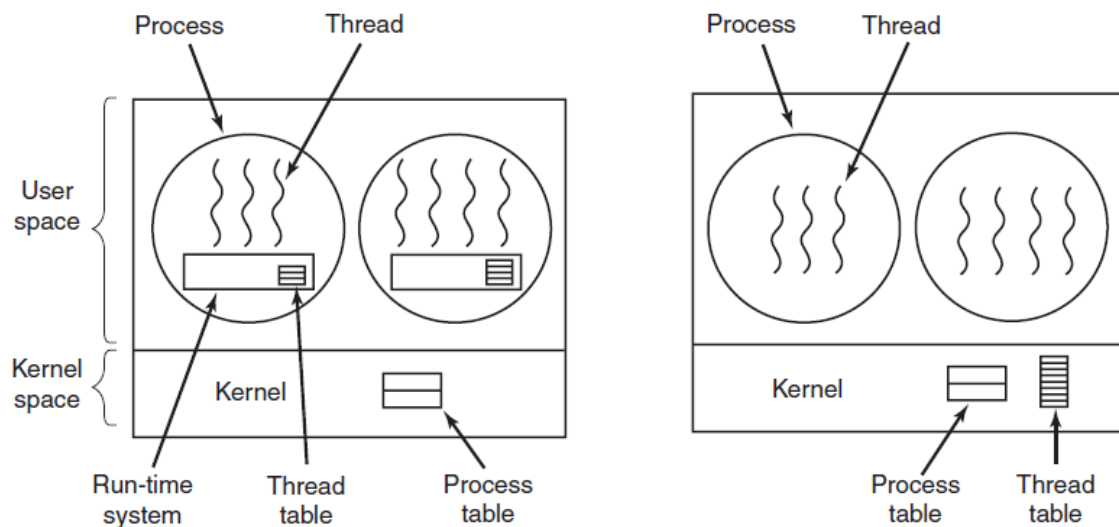


Figure 1: A user-level threads package and a kernel-managed threads package.

4. How Thread Switching Works

Local Blocking

If a thread performs an operation that blocks only with respect to its own process, such as waiting for another thread in the same process, it calls a run-time system routine.

Switch Sequence

The run-time system then:

1. checks whether the current thread must block,
2. saves its registers in the thread table,
3. selects another ready thread,
4. restores that thread's saved registers,
5. switches the stack pointer and program counter.

Why It Is Fast

This thread switch happens entirely in user space. No trap to the kernel is needed, no process-level context switch is needed, and there is usually no cache flush. As a result, user-level thread switching is often at least an order of magnitude faster than switching through the kernel.

5. Advantages of User-Level Threads

Advantage	Explanation
Works on existing OSes	Can be implemented even if the OS kernel has no native thread support
Very fast switching	Switching threads is just a library-level operation inside the process
Fast scheduling	Scheduler and save/restore logic are local procedures, not kernel calls
Custom scheduling	Each process can use its own scheduling policy
Better scalability	No per-thread kernel table and kernel stack cost for huge thread counts

Custom Scheduling

Different applications may need different scheduling behavior. With user-level threads, each process can choose its own thread scheduling algorithm.

Garbage Collector Example

In some applications, such as those using a garbage-collector thread, it is useful that the programmer can control thread scheduling without the kernel interrupting at inconvenient moments.

6. Problem 1: Blocking System Calls

Most Serious Practical Problem

If a user-level thread makes a blocking system call, the kernel blocks the entire process because the kernel does not know that other runnable threads exist inside it.

Example

Suppose one thread reads from the keyboard before any input is available. If the thread actually enters the blocking system call, the whole process stops, including all other threads.

Why This Is Bad

One main reason for using threads is to allow one blocked activity not to stop the rest of the program. User-level threads fail badly here when blocking system calls are involved.

7. Possible Workarounds

7.1 Nonblocking System Calls

Theoretical Fix

One solution is to change system calls so that they become nonblocking. For example, a `read` could return immediately with zero bytes if no input is ready.

Why This Is Unattractive

This approach requires changing the operating system and also changes the meaning of familiar calls like `read`, which would force many user programs to change as well.

7.2 Wrapper or Jacket

Wrapper Idea

If it is possible to check in advance whether a system call would block, the thread library can place a **wrapper** (also called a **jacket**) around that call.

UNIX Example with `select`

On many UNIX systems, the library can call `select` first to see whether a `read` is safe. If the read would block, the library does not make the real system call and instead runs another thread.

Weakness

This method works, but it is inefficient, awkward, and requires rewriting parts of the system call library.

8. Problem 2: Page Faults

Kernel Does Not Know About Threads

If one user-level thread causes a page fault, the kernel blocks the entire process while bringing the missing page in from disk.

Why This Hurts

Even if other threads in that process are ready to run, they cannot run because the kernel only sees one process and blocks it as a whole.

9. Problem 3: No Automatic Preemption Inside the Process

Running Forever

In a pure user-level thread system, if one thread keeps running and never voluntarily enters the run-time system, no other thread in that process will run.

Why This Happens

There are no per-thread clock interrupts inside the process to force round-robin scheduling. The scheduler only runs when a thread explicitly gives it a chance.

Possible but Messy Fix

The run-time system could request periodic clock signals to regain control, but this is crude, messy, and can introduce significant overhead or interfere with the application's own use of timers.

10. Most Devastating Criticism

Blocking-Heavy Applications

Programmers often want threads most in applications where blocking happens frequently, such as multithreaded Web servers. But these are exactly the cases where pure user-level threads perform badly because they make many system calls.

Why Kernel Help Becomes Natural

Once a thread traps into the kernel for a system call, it is usually not much more work for the kernel to switch to another thread if the old one blocks. That makes kernel-supported threads much more practical for blocking-intensive workloads.

CPU-Bound Programs

For programs that are almost entirely CPU bound and rarely block, threads often provide little advantage anyway. In such cases, the motivation for using threads is much weaker.

11. User-Level Switching Sketch

Conceptual Flow

```
thread_yield()
{
    save_current_thread_state();
    next = choose_ready_thread();
    load_thread_state(next);
    run(next);
}
```

Meaning

This shows the main attraction of user-level threads: saving state, scheduling, and restoring another thread can all happen without entering the kernel.

12. Main Conclusion

Takeaway

User-level threads are fast, flexible, and easy to add to systems without kernel thread support. However, they suffer from major problems with blocking system calls, page faults, and lack of automatic preemption, which makes them poorly suited for many real blocking-intensive applications.

13. Quick Review

Exam-Focused Points

- User-level threads are implemented entirely in user space by a library and run-time system.
- The kernel does not know the threads exist.
- Each process keeps its own private thread table.
- Thread switching in user space is very fast because it avoids kernel traps and full context switches.
- User-level threads allow custom scheduling per process.
- A blocking system call made by one thread can block the entire process.
- Page faults caused by one thread also block the whole process.
- Without voluntary yielding or timer tricks, one thread can monopolize the CPU inside the process.
- Wrappers or jackets can reduce blocking-call problems, but they are awkward and inefficient.
- User-level threads are least suitable for applications that block often, such as Web servers.

Implementing Threads in the Kernel

Definition

Kernel-level thread implementation means the kernel knows about threads and manages them directly. Thread creation, destruction, blocking, and scheduling are handled by the kernel rather than by a user-space run-time library.

1. Basic Idea

Kernel-Managed Threads

In a kernel-thread system, there is no separate run-time thread system inside each process for core thread management. The kernel itself maintains the thread data structures and performs thread operations.

No Per-Process Thread Table

Unlike user-level threads, there is no private thread table inside every process. Instead, the kernel keeps a **system-wide thread table**.

2. Kernel Thread Table

Central Thread Table

The kernel thread table stores information about every thread in the system.

Typical Stored Information

The kernel thread table contains per-thread data such as:

- registers,
- thread state,
- program counter,
- and other execution-related information.

Relation to the Process Table

The kernel still keeps the ordinary process table as well. The process table tracks processes, while the kernel thread table tracks individual threads.

3. How Thread Operations Work

Kernel Calls

When a thread wants to create or destroy another thread, it must make a **kernel call**. The kernel then updates the kernel thread table accordingly.

Cost Implication

Because thread operations require entering the kernel, they are more expensive than user-level thread operations that stay entirely in user space.

4. Blocking Behavior

Major Advantage

If a thread blocks in the kernel, the kernel can choose another runnable thread to execute.

What the Kernel Can Do

When one thread blocks, the kernel may schedule:

- another thread from the **same process**, or
- a thread from a **different process**.

Contrast with User-Level Threads

With user-level threads, the run-time system can only keep switching among threads inside its own process until the kernel takes the CPU away. With kernel threads, the kernel directly decides what runs next.

5. Advantages of Kernel Threads

Advantage	Explanation
Blocking system calls work naturally	If one thread blocks, others may still run
Page faults are handled better	A page fault in one thread does not necessarily stop the whole process
True kernel scheduling	The kernel can pick the best runnable thread system-wide
No wrapper tricks needed	No need for jackets or wrappers around blocking calls

No Need for New Nonblocking Calls

Kernel threads do not require the operating system to invent special nonblocking versions of common system calls just to keep other threads running.

6. Page Fault Advantage

Better Than User-Level Threads

If one thread causes a page fault, the kernel can check whether another thread in the same process is runnable and execute it while waiting for the missing page to be loaded from disk.

Why This Matters

This avoids one of the major weaknesses of user-level threads, where a page fault caused by one thread blocks the entire process.

7. Thread Recycling

Recycling Idea

Creating and destroying kernel threads is relatively expensive. To reduce this overhead, some systems **recycle** threads.

How Recycling Works

When a thread is “destroyed,” the system may simply mark it as not runnable and keep its kernel data structures. Later, instead of building a new thread from scratch, the kernel reactivates that old thread structure.

Why More Useful Here

Thread recycling is possible with user-level threads too, but it is much more useful for kernel threads because kernel thread creation and destruction cost more.

8. Main Disadvantage

Higher Overhead

The main drawback of kernel threads is that thread operations such as creation, destruction, and blocking require system calls, and system calls are relatively expensive.

Practical Result

If thread operations happen frequently, the extra kernel overhead can become significant.

9. Problems Kernel Threads Do Not Solve

9.1 Problem 1: `fork`

Fork Ambiguity

If a multithreaded process calls `fork`, it is not always obvious what the child process should contain.

Two Possible Answers

- The child could receive only one thread.
- The child could receive copies of all the parent’s threads.

Why the Best Choice Varies

If the child will immediately call `exec`, then copying only one thread may be best. If the child continues running the same program, copying all threads may be more appropriate.

9.2 Problem 2: Signals

Signal Delivery

In the classical model, signals are sent to processes, not to individual threads. That raises a question: when a signal arrives, which thread should handle it?

Possible Approach

Threads might register interest in particular signals, so that a signal is sent to the thread that requested it.

New Ambiguity

What happens if two or more threads register for the same signal? The design becomes complicated again.

10. Kernel Threads vs User-Level Threads

Feature	User-Level Threads	Kernel Threads
Who manages them?	User-space library	Kernel
Thread table location	Inside each process	In the kernel
Switch speed	Very fast	Slower, due to kernel involvement
Blocking system calls	Can block whole process	Usually block only the calling thread
Page fault effect	Can block whole process	Kernel can run another thread
Overhead	Low	Higher

11. Hybrid Implementations

Main Idea

A **hybrid implementation** tries to combine the advantages of user-level threads and kernel-level threads.

How It Works

One common hybrid design uses **kernel-level threads** as the units known to the kernel, and then **multiplexes user-level threads** onto some or all of those kernel threads.

Flexibility

This model gives the programmer control over:

- how many kernel threads to create,
- how many user-level threads to place on each kernel thread.

Because of this, it offers a very flexible design.

Kernel View

In a hybrid system, the kernel is aware only of the **kernel-level threads**. It schedules those threads, not the user-level threads placed on top of them.

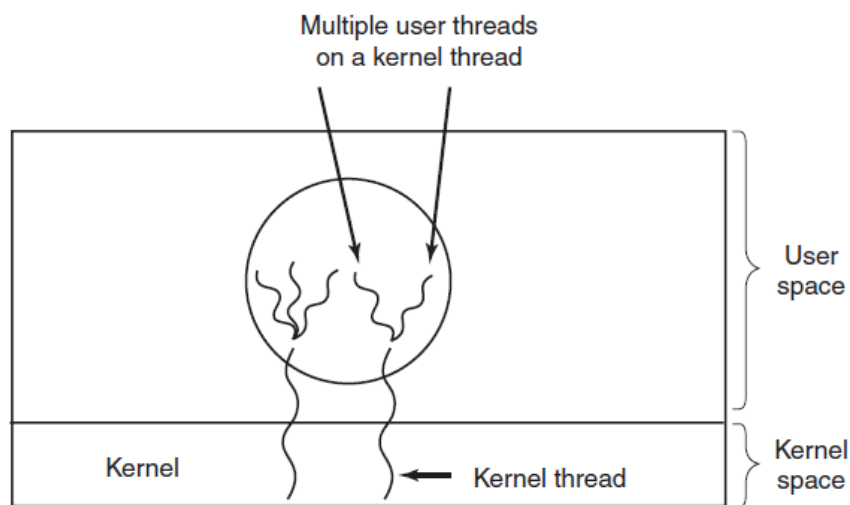


Figure 1: Multiplexing user-level threads onto kernel-level threads.

User-Level Scheduling on Top

The user-level threads running on top of a kernel thread are created, destroyed, and scheduled much like user-level threads in a process running on an operating system with no direct multithreading support.

Key Picture

You can think of the model as:

- the kernel scheduling a smaller number of kernel threads,
- while each kernel thread acts like a carrier for one or more user threads that take turns using it.

Feature	User-Level Threads	Kernel Threads	Hybrid Model
Who the kernel sees	Process only	Every thread	Kernel threads only
Who manages user execution	User library	Kernel	Both kernel and user library
Blocking behavior	Weak for blocking calls	Stronger	Depends on mapping and design
Flexibility	High inside one process	Lower-level OS control	Highest, but more complex

Main Tradeoff

Hybrid implementations provide great flexibility, but the design becomes more complex because thread management is split between user space and the kernel.

12. Main Conclusion

Takeaway

Kernel threads solve the biggest practical problems of user-level threads, especially blocking system calls and page faults. Their cost is higher overhead, and they still leave difficult design questions such as **fork** behavior and signal handling unresolved. Hybrid implementations try to combine the strengths of both approaches by multiplexing user-level threads onto kernel-level threads.

13. Quick Review

Exam-Focused Points

- In kernel threads, the kernel knows about and manages all threads.
- The kernel keeps a system-wide thread table in addition to the process table.
- Thread creation and destruction are done through kernel calls.
- If one thread blocks, the kernel can run another thread from the same or a different process.
- Kernel threads handle blocking system calls and page faults better than user-level threads.
- Some systems recycle kernel threads to reduce creation and destruction overhead.
- The main disadvantage of kernel threads is higher overhead due to system calls.
- Kernel threads do not completely solve issues such as **fork** semantics and signal delivery.
- Hybrid implementations multiplex user-level threads onto kernel-level threads.
- In a hybrid model, the kernel schedules kernel threads, while user-level threads are scheduled on top of them.

Scheduler Activations

Definition

Scheduler activations are a technique designed to combine the good performance and flexibility of user-level threads with the useful blocking behavior of kernel threads.

1. Why Scheduler Activations Were Proposed

Main Motivation

Kernel threads solve important problems such as blocking system calls and page faults, but they are slower. User-level threads are faster, but they block badly when the kernel becomes involved. Scheduler activations try to get the advantages of both.

Main Goal

The goal is to make user-space threads behave as if they had the functionality of kernel threads, while still keeping much of the performance and flexibility of user-level thread packages.

2. What the Model Tries to Achieve

Desired Properties

With scheduler activations:

- user threads should not need special nonblocking system calls,
- user threads should not have to test in advance whether a system call is safe,
- if one thread blocks on a system call or page fault, another ready thread in the same process should still be able to run.

Efficiency Principle

The system tries to avoid unnecessary transitions between user space and kernel space. If a thread blocks only because of user-level synchronization, there is no reason to involve the kernel.

3. Virtual Processors

Core Mechanism

The kernel assigns each process some number of **virtual processors**. The user-space run-time system then decides which user threads run on those virtual processors.

Initial Allocation

A process usually starts with one virtual processor, but:

- it can request more virtual processors,
- it can return processors it no longer needs,
- and the kernel can take processors back if another process needs them more.

Multiprocessor Case

On a multiprocessor system, these virtual processors may correspond directly to real CPUs.

4. Upcalls

Upcall

When the kernel learns that a thread has blocked or become runnable again, it notifies the process's run-time system by activating it at a known entry point. This notification mechanism is called an **upcall**.

What the Kernel Passes

When a blocking event occurs, the kernel informs the run-time system by passing:

- the identity of the affected thread,
- a description of the event that occurred.

UNIX Analogy

This is somewhat like a UNIX signal, except it is specifically used to notify the thread run-time system about thread-related kernel events.

5. What Happens When a Thread Blocks

Blocking Case

Suppose a thread blocks because of a blocking system call or a page fault.

Kernel Action

The kernel does not simply leave the whole process stuck. Instead, it performs an upcall to the process's run-time system.

Run-Time System Action

After receiving the upcall, the run-time system can:

- mark the current thread as blocked,
- choose another ready thread,
- load that thread's state,
- and resume execution.

6. What Happens When the Blocked Thread Becomes Ready Again

Unblocking Case

Later, the kernel may discover that the blocked thread can run again. For example:

- input becomes available,
- a pipe receives data,
- or a missing page is loaded from disk.

Second Upcall

The kernel then makes another upcall to notify the run-time system that the thread is ready again.

Run-Time Choice

The run-time system can then:

- restart that thread immediately, or
- place it on the ready list and schedule it later.

7. Interrupt Handling

Ordinary Interrupt Case

If a hardware interrupt happens while a user thread is running, the CPU switches into kernel mode.

Case 1: Interrupt Not Relevant

If the interrupt is unrelated to the current process, such as another process's I/O completion, then after the interrupt handler finishes, the interrupted thread is resumed as before.

Case 2: Interrupt Relevant

If the interrupt is relevant to the current process, such as the arrival of a page needed by one of its threads, then the kernel does not simply resume the interrupted thread. Instead, it starts the run-time system on that virtual CPU and lets it decide what thread should run next.

Possible Scheduling Choices

After the upcall, the run-time system may choose:

- the interrupted thread,
- the newly ready thread,
- or some other ready thread.

8. Why Scheduler Activations Are Efficient

Key Efficiency Idea

Scheduler activations reduce overhead by involving the kernel only when necessary. Purely user-level synchronization stays in user space, while kernel-relevant blocking events are still reported correctly.

Result

This preserves much of the speed and flexibility of user-level thread scheduling while avoiding the worst blocking problems of pure user-level threads.

9. Main Criticism

Layering Problem

Scheduler activations rely heavily on **upcalls**, and upcalls break the normal layering rule of system design.

Why This Is a Problem

In a normal layered system, layer n provides services to layer $n + 1$, but does not call procedures in layer $n + 1$. Upcalls violate this rule because the kernel calls into the user-level run-time system.

Meaning

This makes the design conceptually powerful, but architecturally controversial.

10. Comparison with Other Models

Feature	User-Level Threads	Kernel Threads	Scheduler Activations
Speed	High	Lower	Tries to stay high
Blocking system call handling	Poor	Good	Good
Need for wrappers	Often yes	No	No
Scheduling flexibility	High	Lower	High
Kernel involvement	Minimal	Heavy	Only when needed

11. Main Conclusion

Takeaway

Scheduler activations try to achieve the best of both worlds: the speed and flexibility of user-level threads plus the correct blocking behavior of kernel threads. Their key idea is that the kernel informs the user-level run-time system through upcalls whenever thread-relevant events occur.

12. Quick Review

Exam-Focused Points

- Scheduler activations were designed to combine user-level and kernel-thread advantages.
- The kernel gives each process some number of virtual processors.
- The user-level run-time system schedules user threads onto those virtual processors.
- When a thread blocks or becomes ready again, the kernel informs the run-time system using an upcall.
- Upcalls let the run-time system choose another thread without losing flexibility.
- Scheduler activations avoid the need for wrappers and special nonblocking system calls.
- They are efficient because unnecessary user-kernel transitions are avoided.
- Their main criticism is that upcalls violate the normal layered-system structure.

Pop-Up Threads

Definition

A **pop-up thread** is a newly created thread that appears when an incoming message arrives and immediately handles that message.

1. Why Pop-Up Threads Are Used

Distributed Systems Context

Threads are often useful in distributed systems, especially when a machine receives incoming messages or service requests from the network.

Traditional Approach

Normally, a process or thread waits in a blocked **receive** call. When a message arrives, it wakes up, accepts the message, unpacks it, examines it, and processes it.

2. The Pop-Up Thread Idea

Alternative Approach

Instead of waking an already existing thread, the system can create a **new thread** when the message arrives. That new thread is responsible for handling the message.

What Happens

The incoming message itself causes the thread to be created. The new pop-up thread is then given that message and starts processing it immediately.

3. Main Advantage

Fresh Start

Because a pop-up thread is brand new, it has no previous execution history to restore:

- no old register values,
- no old stack frames,
- no suspended state from earlier work.

Why This Helps

Since every pop-up thread starts from the same clean initial state, the system can create it quickly.

Latency Benefit

The main performance benefit is very low **latency** between message arrival and the start of processing.

4. Before and After Message Arrival

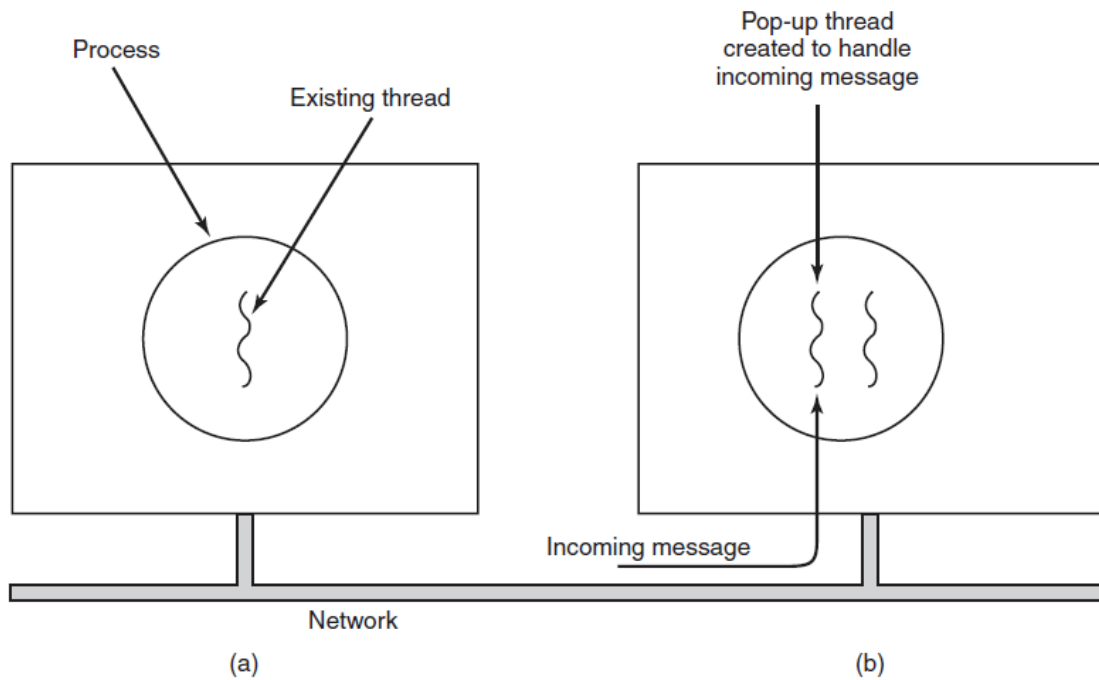


Figure 1: Creation of a new thread when a message arrives.

Situation	What Happens
Before message arrival	Existing thread is running or the process is waiting for work
After message arrival	A new pop-up thread is created specifically to handle the incoming message

5. Where Should the Pop-Up Thread Run?

Design Question

When pop-up threads are used, one important design question is: in which process or context should the new thread run?

Kernel-Space Option

If the system supports threads running in kernel context, the pop-up thread may be created there.

Why Kernel Space Can Be Attractive

Running the pop-up thread in kernel space is often:

- simpler,
- faster,
- and better for accessing kernel tables and I/O devices directly.

6. Advantages of Kernel-Space Pop-Up Threads

Advantage	Explanation
Fast start	No need to transition into user space first
Simple access	Can directly access kernel data structures
Useful for interrupt handling	Can interact directly with I/O devices and interrupt-related data

7. Risk of Kernel-Space Pop-Up Threads

Danger

A buggy pop-up thread in kernel space can do much more damage than a buggy thread in user space.

Example Problem

If a kernel pop-up thread runs too long and cannot be preempted, incoming data may be permanently lost.

Tradeoff

Kernel-space pop-up threads are faster and easier for low-level message handling, but they are riskier because kernel code has far more power and less protection.

8. Traditional vs Pop-Up Handling

Aspect	Traditional Thread	Waiting	Pop-Up Thread
How work begins	Existing thread wakes up		New thread is created on message arrival
Previous state to restore	Yes		No
Startup latency	Higher		Lower
Fresh execution context	No		Yes

9. When Pop-Up Threads Make Sense

Best Use Case

Pop-up threads are most useful when the system needs to react to incoming messages very quickly and message handling must begin with minimal delay.

10. Main Conclusion

Takeaway

Pop-up threads are designed to handle incoming messages with very low latency by creating a fresh thread on demand. Their biggest advantage is fast startup from a clean state, but kernel-space pop-up threads also introduce greater risk if they behave incorrectly.

11. Quick Review

Exam-Focused Points

- Pop-up threads are created when incoming messages arrive.
- They are often useful in distributed systems.
- Their main advantage is low latency between message arrival and processing start.
- A pop-up thread starts with no old execution history to restore.
- Pop-up threads may run in kernel space if the system supports it.
- Kernel-space pop-up threads are often faster and simpler for low-level handling.
- A buggy kernel pop-up thread can be dangerous and may cause data loss.

Making Single-Threaded Code Multithreaded

Definition

Making single-threaded code multithreaded means redesigning an existing program so that multiple threads can run safely and correctly. This is much harder than simply adding thread creation calls.

1. Why Conversion Is Hard

Main Idea

Many old programs were written for a single thread of control. When multiple threads are introduced, shared data, libraries, signals, and stacks can behave incorrectly unless the program is redesigned carefully.

Key Message

Multithreading is not a small patch on top of single-threaded code. It usually requires major redesign of data handling, libraries, and system-call behavior.

2. Main Problem Areas

Problem Area	Why It Breaks in Multithreaded Code
Global variables	Threads may overwrite each other's shared state
Nonreentrant library routines	Two threads may enter the same routine at once and corrupt internal data
Signals	It becomes unclear which thread should receive or handle them
Stack management	Multiple threads require multiple stacks, which may not be managed automatically

3. Problem 1: Global Variables

Thread-Private Global State

Local variables and parameters usually do not cause problems, but **global variables** do. In particular, some variables are logically global to one thread but should not be shared across all threads.

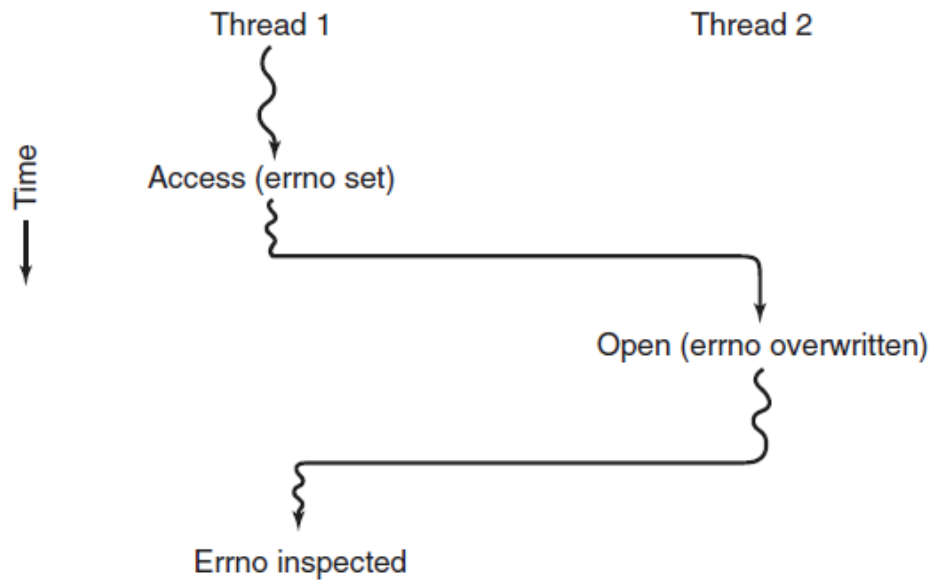


Figure 1: Conflicts between threads over the use of a global variable.

3.1 The `errno` Example

How the Error Happens

In UNIX, when a system call fails, the error code is stored in the global variable `errno`. In a multithreaded program, one thread may set `errno`, then get preempted before reading it. Another thread may then perform a failing system call and overwrite the same variable.

Result

When the first thread resumes, it reads the wrong value of `errno` and behaves incorrectly.

Core Lesson

A variable that worked fine as one process-wide global in single-threaded code may need to become **thread-private** in multithreaded code.

4. Solutions for Global Variables

Possible Solutions

Several approaches can reduce conflicts over globals.

Option 1: Ban Globals

One solution is to forbid global variables entirely. This is theoretically clean, but unrealistic for large existing software systems.

Option 2: Per-Thread Globals

A better approach is to give each thread its own private copy of variables such as `errno`. This creates a new scoping level:

- variables local to one procedure,
- variables visible to all procedures in one thread,
- variables visible everywhere in the program.

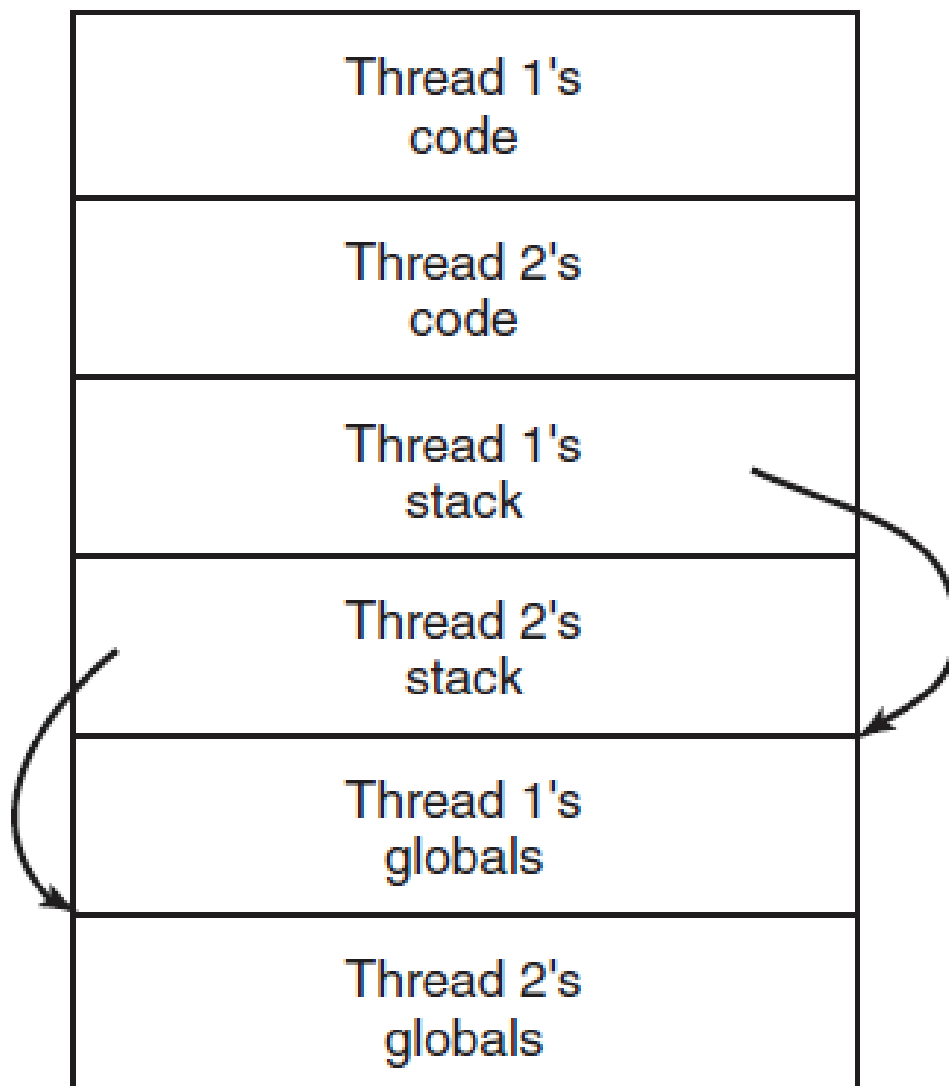


Figure 2: Threads can have private global variables.

4.1 How Per-Thread Globals Can Be Accessed

Method 1: Pass a Globals Structure

One approach is to allocate a block of memory holding the thread's global data and pass it as an extra parameter to all procedures in the thread.

Method 2: Thread-Global Access Routines

Another approach is to introduce special library calls for thread-wide global variables.

Example Interface

```
create_global("bufptr");  
set_global("bufptr", &buf);  
bufptr = read_global("bufptr");
```

Effect

Each thread that creates a variable like `bufptr` gets its own private storage location, even if another thread uses the same name.

5. Problem 2: Nonreentrant Library Procedures

Nonreentrant Routine

A **nonreentrant** routine is a procedure that was not designed to be entered again before a previous call has finished.

Network Message Example

Suppose a library routine builds an outgoing message in one fixed internal buffer. If one thread fills that buffer and is then interrupted, a second thread may overwrite the same buffer before the first thread finishes.

Memory Allocation Example

Procedures like `malloc` maintain internal data structures such as free-memory lists. If a thread switch happens while those structures are temporarily inconsistent, another thread may use invalid pointers and crash the program.

Why This Is Serious

Fixing these issues may require rewriting major parts of the library to make them reentrant.

6. Possible Fixes for Nonreentrant Libraries

Best Fix: Rewrite the Library

The cleanest solution is to redesign library routines so that they are reentrant and safe for concurrent use.

Wrapper or Jacket Approach

Another solution is to put a **jacket** around each library routine. The jacket marks the library as busy and blocks other threads from entering library code until the current call finishes.

Tradeoff

Jackets can make the code safe, but they greatly reduce parallelism because only one thread can use the protected library at a time.

7. Problem 3: Signals

Why Signals Are Difficult

Signals are already complicated in single-threaded programs. In multithreaded programs, they become even harder because not all signals naturally belong to the whole process in the same way.

Thread-Specific Signals

Some signals logically belong to one thread. For example, if a thread calls **alarm**, it makes sense for the resulting signal to return to that same thread.

User-Level Thread Problem

If threads are implemented entirely in user space, the kernel does not know about them and therefore cannot easily deliver a signal to the correct thread.

Process-Wide Signals

Other signals, such as keyboard interrupt, are not obviously thread specific. This creates difficult questions:

- Should one designated thread handle them?
- Should all threads receive them?
- Should a new pop-up thread handle them?

Conflicting Goals

One thread may want a signal such as **CTRL-C** to terminate the whole process, while another thread may want to catch it and continue running. These wishes are incompatible.

8. Problem 4: Stack Management

Single-Thread Case

In many systems, if a process stack overflows, the kernel automatically provides more stack space.

Multithreaded Case

When a process has multiple threads, each thread needs its own stack.

Why This Becomes Difficult

If the kernel does not know about every thread stack, it may not recognize that a memory fault is actually a request to grow one thread's stack. As a result, automatic stack growth may fail.

9. Big Picture

Main Conclusion of the Section

These problems are not impossible to solve, but they show that converting single-threaded code into multithreaded code requires more than simply adding thread creation. It often requires:

- redefining system-call behavior,
- rewriting libraries,
- redesigning data handling,
- and preserving backward compatibility for programs that still use only one thread.

10. Quick Review

Exam-Focused Points

- Converting single-threaded code to multithreaded code is much harder than it first appears.
- Process-wide globals such as `errno` can cause thread conflicts.
- One solution is to give each thread its own private global variables.
- Many library routines are not reentrant and can break when used by multiple threads at once.
- Jackets or wrappers can protect nonreentrant libraries, but reduce parallelism.
- Signals become harder because it is unclear which thread should receive or handle them.
- Multiple threads require multiple stacks, which complicates stack growth and fault handling.
- A real multithreaded conversion usually requires substantial redesign.

Race Conditions

Definition

A **race condition** occurs when two or more processes or threads access shared data at the same time, and the final result depends on the exact timing or order of execution.

1. Why Race Conditions Happen

Main Idea

Processes often cooperate by sharing storage. The shared storage may be in main memory, in a kernel data structure, or in a shared file. The location does not change the core problem: multiple processes can read and write the same data.

Key Message

Sharing data is not automatically wrong. The danger appears when several processes access the same shared value without coordination.

When the operating system switches the CPU from one process to another, the first process may stop in the middle of a multi-step operation. If another process changes the same shared data before the first process resumes, the first process may continue using stale information.

Practical Mental Model

The sequence “read a shared value, use it, then write it back” is not automatically one indivisible action. Another process may run between those steps.

2. Print Spooler Example

Scenario

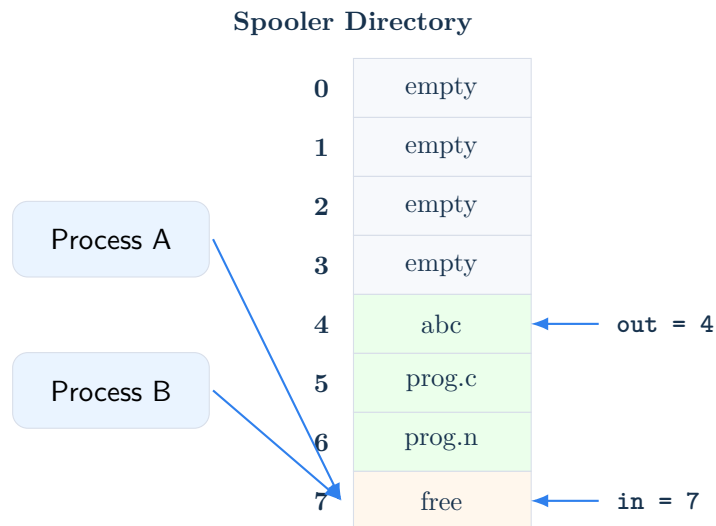
When a process wants to print a file, it places the file name in a shared spooler directory. A printer daemon later checks that directory, prints queued files, and removes their names.

The spooler directory contains numbered slots. Two shared variables control the queue:

- **out**: points to the next file that should be printed.
- **in**: points to the next free slot where a new file name can be placed.

Initial State

Slots 0–3 are empty because their files have already been printed. Slots 4–6 are full. Slot 7 is the next free slot, so **out** = 4 and **in** = 7.



3. How the Race Occurs

The Timing Problem

Processes A and B both decide to print a file at nearly the same time. Both need to read `in`, use the slot number, write a file name, and then update `in`.

1. Process A reads `in` and stores 7 in its local variable `next_free_slot`.
2. A clock interrupt occurs before A writes its file name.
3. The CPU switches to process B.
4. Process B reads `in`. It also sees 7.
5. Process B writes its file name into slot 7 and updates `in` to 8.
6. Process A resumes and still believes slot 7 is free.
7. Process A writes its file name into slot 7, overwriting B's file name.
8. Process A updates `in` to 8.

Result

The spooler directory still looks internally consistent because `in = 8`. However, process B's print request has silently disappeared.

4. Timeline View

Step	Process A	Process B
1	Reads <code>in</code> = 7.	–
2	Gets interrupted before writing.	–
3	–	Reads <code>in</code> = 7.
4	–	Writes file name into slot 7.
5	–	Sets <code>in</code> = 8.
6	Resumes with old local value 7.	–
7	Writes file name into slot 7, replacing B's entry.	–
8	Sets <code>in</code> = 8.	B's request is lost.

5. Critical Section

Critical Section

A **critical section** is a part of a program that accesses shared data and must not be executed by more than one process or thread at the same time.

For the print spooler, the dangerous sequence is:

```
next_free_slot = in
spooler_directory[next_free_slot] = file_name
in = next_free_slot + 1
```

Core Lesson

The problem is not any single line by itself. The problem is that the whole read-modify-write sequence must behave like one protected operation.

6. Why Race Conditions Are Hard to Debug

Timing Dependence

Race conditions are difficult because the bug depends on scheduling. The same program may work correctly in many test runs, then fail when the CPU switches processes at an unlucky moment.

- The failure may be rare.
- Re-running the same program may make the bug disappear.
- Adding print statements or logs may change timing and hide the bug.
- More CPU cores increase the chance of true parallel overlap.

Why the Spooler Bug Is Subtle

The printer daemon may not detect anything wrong. The queue still has a valid-looking slot 7 and a valid-looking `in` value. The damage is logical: one user's request was overwritten.

7. How Operating Systems Prevent Race Conditions

Synchronization

Synchronization means using rules or mechanisms that control how processes or threads enter critical sections and access shared data.

Mechanism	Purpose
Locks / mutexes	Allow only one process or thread to enter a critical section at a time.
Semaphores	Control access to shared resources using counters.
Atomic operations	Perform small read-modify-write operations indivisibly.
Monitors	Combine shared data with safe procedures for accessing it.
Message passing	Avoid shared writable data by communicating through messages.

8. Quick Review

Key Points

- A race condition occurs when the result depends on the order and timing of concurrent access.
- Shared storage may be memory, a kernel data structure, or a file.
- In the spooler example, processes A and B both believe slot 7 is free.
- Process A overwrites process B's file name, so B's print request is lost.
- The queue can still look internally consistent, making the bug hard to notice.
- The dangerous code region is called a critical section.
- Race conditions are prevented using synchronization mechanisms such as locks, semaphores, atomic operations, monitors, or message passing.

Critical Regions

Definition

A **critical region**, also called a **critical section**, is the part of a program where a process accesses shared memory, shared files, or other shared resources that can lead to race conditions.

1. Why Critical Regions Matter

Main Idea

Race conditions are avoided by making sure that two processes do not read or write the same shared data at the same time.

Key Message

The central requirement is **mutual exclusion**: if one process is using a shared variable, file, or resource, other processes must be excluded from using that same shared item until the first process is finished.

In the print spooler example, the race happened because process B started using the shared variable `in` before process A had finished using it. A correct solution must prevent that kind of overlap.

2. Internal Work vs. Critical Work

Normal Execution

Most of the time, a process performs private computations that do not affect other processes. These instructions do not need special protection because they do not touch shared data.

Critical Execution

When a process accesses shared memory, shared files, shared variables, or shared devices, it enters a dangerous part of the program. This part must be controlled carefully to avoid races.

Simple View

A process can be thought of as alternating between two kinds of work:

- **Noncritical work**: private computation with no shared-data risk.
- **Critical region**: shared-data access where mutual exclusion is required.

3. The Goal

Mutual Exclusion Goal

If one process is inside its critical region, no other process should be allowed to enter its own critical region for the same shared resource.

If this rule is enforced, then the shared data is accessed by only one process at a time. That prevents the timing problem where one process sees stale or half-updated data from another process.

Important Distinction

Preventing two processes from being in their critical regions at the same time avoids race conditions, but a complete solution must also be fair and efficient.

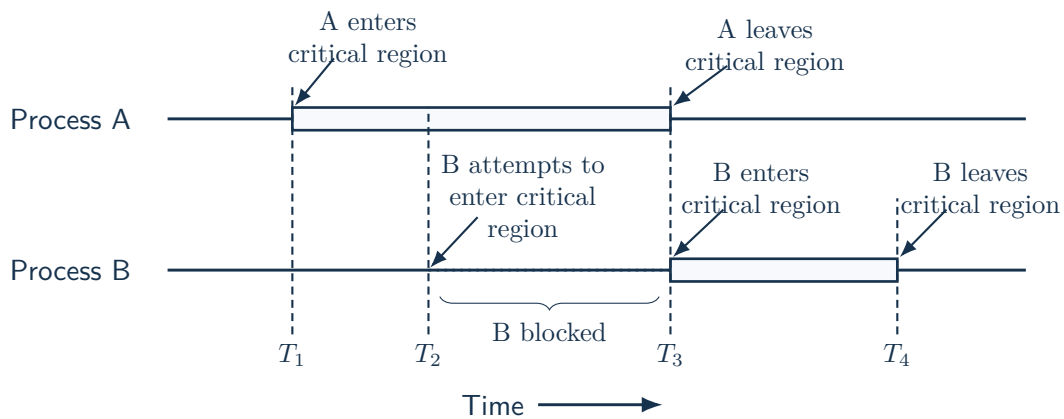
4. Four Requirements for a Good Solution

No.	Requirement	Meaning
1	No two processes may be simultaneously inside their critical regions.	This is the mutual exclusion rule. Only one process may access the protected shared data at a time.
2	No assumptions may be made about speeds or the number of CPUs.	The solution must work whether processes run slowly, quickly, on one CPU, or on many CPUs.
3	No process running outside its critical region may block any process.	A process that is not using the shared data should not prevent others from entering their critical regions.
4	No process should have to wait forever to enter its critical region.	The solution must avoid starvation. Every waiting process should eventually get a turn.

5. Timing Diagram

What the Diagram Shows

Process A enters its critical region first. Process B tries to enter while A is still inside, so B is blocked. When A leaves, B enters immediately. Later, B leaves and both processes are outside their critical regions again.



6. What Happens at Each Time

Time	Event
T_1	Process A enters its critical region and starts using the shared resource.
T_2	Process B attempts to enter its critical region, but it is not allowed because A is already inside.
T_2-T_3	Process B is blocked. It must wait until the shared resource becomes available.
T_3	Process A leaves its critical region, so process B can enter immediately.
T_4	Process B leaves its critical region. Now no process is inside a critical region.

7. Why the Four Requirements Matter

Requirement 1: Mutual Exclusion

This is the basic safety rule. Without it, two processes can modify shared data at the same time and create a race condition.

Requirement 2: No Timing Assumptions

A correct operating-system solution cannot depend on one process being faster than another. It also cannot depend on having exactly one CPU.

Requirement 3: No Outside Blocking

If a process is doing private work outside its critical region, it should not stop another process from entering a critical region. Otherwise, the system wastes parallelism.

Requirement 4: No Starvation

A process that wants to enter its critical region should eventually be allowed to enter. It should not wait forever while other processes repeatedly get access.

8. Common Terms

Term	Meaning
Race condition	A bug where the result depends on the timing of concurrent access to shared data.
Critical region	The part of the program that accesses shared resources and must be protected.
Mutual exclusion	A rule that allows only one process at a time to use a shared resource.
Blocked process	A process that cannot continue because it is waiting for a resource or condition.
Starvation	A situation where a process waits forever and never gets access to the resource it needs.

9. Quick Review

Key Points

- A critical region is code that accesses shared data or shared resources.
- Mutual exclusion prevents more than one process from entering a protected critical region at once.
- Race conditions can be avoided if no two processes are in their critical regions at the same time.
- A full solution must also be fair and efficient.
- Correctness must not depend on process speed or the number of CPUs.
- A process outside its critical region should not block another process.
- No process should wait forever to enter its critical region.

Mutual Exclusion with Busy Waiting

Part 1: interrupts, lock variables, and strict alternation

Definition

Mutual exclusion means making sure that while one process is inside a critical region using shared data, no other process can enter a conflicting critical region.

1. Purpose of Busy-Waiting Solutions

Main Idea

The goal is to prevent race conditions while processes update shared memory, shared files, or shared data structures.

Key Requirement

Only one process should be allowed inside the protected critical region at a time. Every proposal in this section is an attempt to enforce that rule.

Busy Waiting

Busy waiting means repeatedly checking a condition in a loop until it becomes true. A lock that uses busy waiting is called a **spin lock**.

Cost

Busy waiting wastes CPU time because the waiting process keeps running but does no useful work. It is only reasonable when the expected wait is very short.

2. Approach 1: Disabling Interrupts

How It Works

On a single-processor system, a process can disable interrupts just after entering its critical region and re-enable them just before leaving.

With interrupts disabled, clock interrupts cannot occur. Since a process switch normally happens because of clock or other interrupts, the current process keeps the CPU and can update shared memory without being interrupted.

Case	Effect
Single CPU	Disabling interrupts can prevent another process from running on that CPU.
User process	Dangerous, because a user program might disable interrupts and never enable them again.
Kernel code	Sometimes useful for a few instructions while updating kernel lists or variables.
Multicore system	Not enough, because other CPUs continue running and can still access shared memory.

Why It Is Not a General Solution

Giving user processes the power to turn off interrupts is unsafe. A broken or malicious program could stop the system from responding.

Kernel Use

Disabling interrupts can still be useful inside the operating system kernel for very short critical updates, such as modifying the ready-process list.

Multicore Limitation

On a multiprocessor or multicore machine, disabling interrupts on one CPU does not stop another CPU from accessing the same shared memory. More sophisticated synchronization is needed.

3. Approach 2: Lock Variables

Lock Variable

A **lock variable** is a shared variable used to indicate whether some process is already inside a critical region.

The simple idea is:

- `lock = 0`: no process is inside the critical region.
- `lock = 1`: some process is inside the critical region.

```
while (lock == 1){  
    // busy wait  
    /*loop tasks*/  
}  
lock = 1;  
  
critical_region();  
  
lock = 0;  
noncritical_region();
```

Why This Looks Reasonable

A process first waits until `lock` is 0. Then it sets `lock` to 1 and enters the critical region. When it leaves, it sets `lock` back to 0.

3.1 The Fatal Race

Step	Process A	Process B
1	Reads <code>lock = 0</code> .	–
2	Gets interrupted before setting <code>lock = 1</code> .	–
3	–	Reads <code>lock = 0</code> .
4	–	Sets <code>lock = 1</code> and enters the critical region.
5	Resumes and sets <code>lock = 1</code> .	Already inside the critical region.
6	Enters the critical region too.	Race condition occurs.

Fatal Flaw

Testing the lock and setting the lock are two separate operations. A process switch can happen between them, allowing two processes to enter the critical region.

Why Checking Twice Does Not Fix It

If the process checks the lock again just before storing into it, the race can still happen immediately after the second check. The operation must be atomic, not merely checked more often.

4. Approach 3: Strict Alternation

Strict Alternation

Strict alternation uses a shared variable `turn` to force two processes to take turns entering their critical regions.

Initially, `turn = 0`, so process 0 is allowed to enter first.

Process 0

```
while (TRUE) {
    while (turn != 0){
        /*loop tasks*/
    }
    critical_region();
    turn = 1;
    noncritical_region();
}
```

Process 1

```
while (TRUE) {
    while (turn != 1){
        /*loop tasks*/
    }
    critical_region();
    turn = 0;
    noncritical_region();
}
```

Notice the Semicolon

The empty statement after each inner `while` loop is intentional. The process repeatedly tests `turn` until it is allowed to continue.

5. Why Strict Alternation Fails

What It Gets Right

Strict alternation prevents two processes from being inside their critical regions at the same time, so it avoids the race condition.

What It Gets Wrong

It forces the processes to alternate even when one process is not interested in entering its critical region.

Suppose process 0 finishes its critical region and sets `turn = 1`. If process 1 is still doing noncritical work, process 0 cannot enter again. It must wait for process 1, even though process 1 is not using the shared data.

Problem	Explanation
Unnecessary waiting	A fast process may be blocked by a slow process doing unrelated noncritical work.
Wasted CPU time	The blocked process spins in a loop checking <code>turn</code> .
Violates condition 3	A process outside its critical region can block another process from entering.
Poor practical behavior	Neither process can enter twice in a row, even if the other process does not need the critical region.

Print Spooler Meaning

If critical regions are associated with writing the spooler directory, strict alternation would mean process 0 could not queue two print jobs in a row unless process 1 also took a turn.

6. Comparison of the First Three Attempts

Method	Main Benefit	Main Failure
Disabling interrupts	Simple on one CPU.	Unsafe for user processes and ineffective across multiple CPUs.
Lock variable	Simple software idea.	Testing and setting the lock are not atomic.
Strict alternation	Prevents simultaneous critical-region entry.	Blocks a process even when the other process is outside its critical region.

7. Quick Review

Key Points

- Busy waiting repeatedly tests a condition while consuming CPU time.
- A spin lock is a lock that uses busy waiting.
- Disabling interrupts can work briefly inside the kernel on a single CPU, but is not a general user-level solution.
- A simple lock variable fails because checking and setting the lock are separate operations.
- Strict alternation prevents races but forces processes to take turns unnecessarily.
- Strict alternation violates the requirement that a process outside its critical region must not block another process.

Mutual Exclusion with Busy Waiting

Part 2: Peterson's solution, TSL, and XCHG

Focus of This Note

This note continues busy-waiting mutual exclusion by studying one software solution, **Peterson's solution**, and two hardware-assisted solutions, **TSL** and **XCHG**.

1. From Strict Alternation to Better Ideas

Problem to Solve

Strict alternation prevents two processes from entering their critical regions at the same time, but it can block a process even when the other process is outside its critical region.

Better Strategy

A better software solution combines two ideas:

- a process announces that it is interested in entering the critical region,
- a shared `turn` variable breaks ties when both processes are interested.

2. Peterson's Solution

Peterson's Algorithm

Peterson's solution is a software algorithm for mutual exclusion between two processes. It avoids strict alternation by using both an interest array and a turn variable.

The shared data are:

- `turn`: indicates which process should wait if both want to enter.
- `interested[N]`: records whether each process wants to enter its critical region.

```
#define FALSE 0
#define TRUE 1
#define N 2

int turn;
int interested[N]; /* all values initially FALSE */

void enter_region(int process)
{
    int other;

    other = 1 - process;
    interested[process] = TRUE;
    turn = process;
```

```

while (turn == process && interested[other] == TRUE)
{
    /*loop tasks*/
}

void leave_region(int process)
{
    interested[process] = FALSE;
}

```

How to Use It

Before entering the critical region, a process calls `enter_region(process)`. After leaving the critical region, it calls `leave_region(process)`.

3. How Peterson's Solution Works

Situation	Result
Only process 0 is interested	Process 0 enters immediately because process 1 is not interested.
Only process 1 is interested	Process 1 enters immediately because process 0 is not interested.
Both become interested almost together	Both set their interest flags, but the final write to <code>turn</code> decides who waits.
One process leaves	It sets its interest flag to <code>FALSE</code> , allowing the other waiting process to continue.

Tie-Breaking Example

Suppose both processes call `enter_region` at about the same time. If process 1 stores into `turn` last, then `turn = 1`. Process 1 waits, while process 0 enters the critical region.

Why This Is Better Than Strict Alternation

If the other process is not interested, a process can enter immediately. It does not have to wait merely because it is the other process's turn.

Modern Systems Note

Peterson's solution is a classic conceptual algorithm. Real modern systems usually need atomic variables or memory barriers because compilers and CPUs may reorder memory operations.

4. Peterson's Solution and the Four Conditions

Condition	How Peterson's Solution Handles It
Mutual exclusion	Both processes cannot pass the waiting test at the same time when both are interested.
No speed assumptions	The algorithm does not rely on one process being faster than the other.
No outside blocking	A process that is not interested has <code>interested[process] = FALSE</code> , so it does not block the other.
No endless waiting	The waiting process can enter after the other process leaves and clears its interest flag.

5. The TSL Instruction

TSL

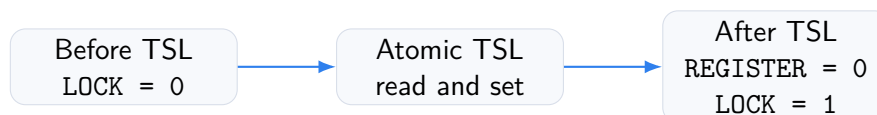
TSL means **Test and Set Lock**. It is a hardware instruction that reads a lock value and sets the lock in one indivisible operation.

The instruction is usually written like this:

```
TSL REGISTER, LOCK
```

It does two things atomically:

1. copy the old value of `LOCK` into `REGISTER`,
2. store a nonzero value into `LOCK`.



Why Atomicity Matters

No other processor can access the lock word between the read and the write. This removes the race that broke the simple lock-variable solution.

6. Using TSL for Mutual Exclusion

```
enter_region:
    TSL REGISTER, LOCK ; copy LOCK to REGISTER and set LOCK to 1
    CMP REGISTER, #0 ; was LOCK zero?
    JNE enter_region ; if not zero, lock was already set
    RET ; critical region entered

leave_region:
    MOVE LOCK, #0 ; release the lock
    RET
```

How the Code Works

If the old value of `LOCK` was 0, the process has acquired the lock and enters the critical region. If the old value was not 0, another process already has the lock, so the process loops and tries again.

Still Busy Waiting

TSL makes lock acquisition correct, but the waiting process still spins in a loop. It is safe mutual exclusion, but it can waste CPU time.

7. TSL vs. Disabling Interrupts

Technique	What It Controls
Disabling interrupts	Stops interrupts only on the CPU that disabled them. It does not stop other CPUs.
TSL / atomic test-and-set	Locks the memory operation itself, so other processors cannot slip between the read and write of the lock.

Key Difference

Disabling interrupts controls scheduling on one CPU. TSL controls access to the shared memory word itself.

8. The XCHG Instruction

XCHG

XCHG atomically exchanges the contents of two locations, such as a register and a memory word. It can be used to build the same kind of spin lock as TSL.

```
enter_region:
    MOVE REGISTER, #1 ; put 1 in the register
    XCHG REGISTER, LOCK ; atomically swap REGISTER and LOCK
    CMP REGISTER, #0 ; was LOCK zero before the swap?
    JNE enter_region ; if not zero, lock was already set
    RET ; critical region entered

leave_region:
    MOVE LOCK, #0 ; release the lock
    RET
```

How XCHG Works

The process places 1 in a register, then atomically swaps it with `LOCK`. If the register receives 0, the lock was free and is now acquired. If the register receives 1, the lock was already taken and the process keeps spinning.

9. TSL and XCHG Compared

Instruction	Atomic Action	Use in Mutual Exclusion
TSL	Reads the old lock value and sets the lock to nonzero.	If the old value was 0, the caller acquired the lock.
XCHG	Swaps a register value with the lock value.	If the old lock value moved into the register was 0, the caller acquired the lock.

Low-Level Synchronization

Hardware atomic instructions such as TSL and XCHG are building blocks for low-level locks used by operating systems and runtime libraries.

10. Cooperation Requirement

Processes Must Follow the Protocol

Critical-region solutions only work if processes call `enter_region` before touching shared data and call `leave_region` after finishing. If a process ignores the protocol, mutual exclusion fails.

Correct Lock Discipline

A process should hold the lock only while it is actually using the protected shared data. Keeping the lock too long increases waiting and reduces parallelism.

11. When Busy Waiting Makes Sense

Good Fit	Poor Fit
The wait is expected to be very short.	The wait may be long or unpredictable.
The cost of sleeping and waking is higher than spinning briefly.	CPU time is valuable and should not be burned in a loop.
Low-level kernel or multiprocessor synchronization.	General user-level waiting for I/O or long critical regions.

12. Quick Review

Key Points

- Peterson's solution uses both `interested[]` and `turn`.
- If only one process is interested, it enters immediately.
- If both are interested, `turn` decides which process waits.
- TSL atomically reads the old lock value and sets the lock.
- XCHG atomically swaps a register and the lock variable.
- Hardware atomic instructions fix the race in simple lock variables.
- TSL and XCHG still use busy waiting, so they can waste CPU time.
- Critical-region methods require all processes to cooperate with the entry and exit protocol.

Sleep and Wakeup

Blocking instead of wasting CPU time

Definition

Sleep is a system call that blocks the calling process. **Wakeup** is a system call that makes a blocked process ready to run again.

1. Why Sleep and Wakeup Are Needed

Main Idea

Peterson's solution, TSL, and XCHG can correctly provide mutual exclusion, but they require busy waiting. A waiting process keeps checking a condition in a tight loop instead of giving up the CPU.

Busy-Waiting Cost

Busy waiting wastes CPU time because the process is running but doing no useful work. It can also create scheduling problems when priorities are involved.

Blocking Alternative

Instead of spinning, a process that cannot continue can go to sleep. Another process wakes it up when the condition it needs becomes true.

2. Priority Inversion Problem

Priority Inversion

Priority inversion can occur when a high-priority process waits for a resource held by a low-priority process, but the low-priority process cannot run long enough to release it.

Consider two processes:

- **H**: high-priority process.
- **L**: low-priority process.

How the Problem Happens

Suppose L is inside its critical region. Then H becomes ready to run and starts busy waiting for the same critical region. Because H has higher priority, the scheduler keeps running H. But H is only spinning, and L never gets CPU time to leave the critical region.

Result

H waits forever even though the only process that can help it is L. The high-priority process prevents the low-priority process from releasing the resource.

Step	Event
1	Low-priority process L enters the critical region.
2	High-priority process H becomes ready to run.
3	Scheduler runs H because it has higher priority.
4	H busy-waits for the critical region.
5	L cannot run, so it cannot leave the critical region.
6	H keeps spinning forever.

3. Sleep and Wakeup Primitives

Sleep

`sleep()` blocks the caller. The process is suspended and does not consume CPU time while waiting.

Wakeup

`wakeup(process)` wakes a specified sleeping process. Some systems instead use `sleep(address)` and `wakeup(address)`, where the address identifies the condition being waited for.



4. Producer-Consumer Problem

Bounded-Buffer Problem

The **producer-consumer problem**, also called the **bounded-buffer problem**, has two processes sharing a fixed-size buffer. The producer inserts items, and the consumer removes items.

Process	Role
Producer	Creates items and puts them into the shared buffer.
Consumer	Removes items from the shared buffer and uses them.
Buffer	Holds at most N items.
count	Shared variable storing the current number of items in the buffer.

Correct Behavior Wanted

If the buffer is full, the producer should sleep until the consumer removes an item. If the buffer is empty, the consumer should sleep until the producer inserts an item.

5. Naive Sleep-Wakeup Code

```
#define N 100 /* number of slots in the buffer */
int count = 0; /* number of items in the buffer */

void producer(void)
{
    int item;

    while (TRUE) {
        item = produce_item();
        if (count == N)
            sleep();
        insert_item(item);
        count = count + 1;
        if (count == 1)
            wakeup(consumer);
    }
}

void consumer(void)
{
    int item;

    while (TRUE) {
        if (count == 0)
            sleep();
        item = remove_item();
        count = count - 1;
        if (count == N - 1)
            wakeup(producer);
        consume_item(item);
    }
}
```

Fatal Weakness

The shared variable `count` is accessed without protection. The test of `count` and the call to `sleep()` are not one atomic operation.

6. The Lost-Wakeup Race

Initial State

Assume the buffer is empty, so `count = 0`. The consumer is about to check whether it should sleep.

Step	Consumer	Producer
1	Reads <code>count</code> and sees 0.	–
2	Is interrupted before actually calling <code>sleep()</code> .	–
3	–	Runs and inserts an item.
4	–	Increments <code>count</code> to 1.
5	–	Calls <code>wakeup(consumer)</code> because the buffer was empty.
6	Wakeup is lost because the consumer is not yet sleeping.	–
7	Runs again and continues using the old decision that <code>count</code> was 0.	–
8	Calls <code>sleep()</code> even though an item is available.	–

Why This Is Serious

Eventually the producer may fill the buffer and go to sleep too. Then the consumer is asleep, the producer is asleep, and no process is left to wake the other.

7. Core Cause of the Bug

Lost Wakeup

A **lost wakeup** occurs when a wakeup is sent to a process that is not yet sleeping. Since the signal is not saved, the process may later go to sleep forever.

The Exact Race

The consumer checks `count == 0`, but before it can sleep, the producer changes `count` and sends a wakeup. The consumer misses the wakeup and then sleeps based on stale information.

Pattern to Notice

The danger is not only shared data. The danger is checking a condition and blocking on that condition without making the two actions atomic.

8. Wakeup Waiting Bit

Quick Fix Idea

One proposed patch is a **wakeup waiting bit**. If a wakeup is sent before the target process is asleep, the bit is set. Later, when the process tries to sleep, it sees the bit, clears it, and stays awake.

Situation	Behavior with Wakeup Waiting Bit
Wakeup arrives while process sleeps	Process is woken normally.
Wakeup arrives before process sleeps	Wakeup bit is saved.
Process later tries to sleep	Saved bit is cleared, and the process remains awake.

Why This Is Not Enough

One wakeup waiting bit may work for this simple two-process example, but it does not solve the general problem. With three or more processes, one bit is not enough to remember all possible missed wakeups.

9. Busy Waiting vs. Blocking

Method	Behavior	Main Problem
Busy waiting	Process repeatedly checks until it can proceed.	Wastes CPU time and can cause priority inversion.
Sleep/wakeup	Process blocks and waits to be awakened.	Can lose wakeups if condition checks and blocking are not protected.

10. Quick Review

Key Points

- Busy waiting wastes CPU time because the process spins in a loop.
- Priority inversion can occur when a high-priority process spins while a low-priority process holds the needed resource.
- `sleep()` blocks a process so it does not consume CPU time.
- `wakeup()` makes a sleeping process ready again.
- The producer-consumer problem uses a fixed-size shared buffer.
- The producer sleeps when the buffer is full.
- The consumer sleeps when the buffer is empty.
- The naive solution fails because a wakeup can be lost.
- A wakeup waiting bit is only a patch, not a general solution.

Semaphores

Counting saved wakeups with atomic operations

Definition

A **semaphore** is an integer synchronization variable used to count saved wakeups and control access to shared resources.

1. Why Semaphores Were Introduced

Main Idea

The sleep-wakeup solution failed because a wakeup could be sent before the target process was actually asleep. That wakeup was lost.

Dijkstra's Idea

E. W. Dijkstra proposed storing wakeups in an integer variable. A semaphore value of 0 means no wakeups are saved. A positive value means one or more wakeups are available for future use.

Problem Being Solved

The goal is to avoid lost wakeups while still blocking processes instead of forcing them to busy-wait.

2. Semaphore Operations

Two Basic Operations

Semaphores use two atomic operations:

- **down**: uses one saved wakeup or blocks if none is available.
- **up**: creates one saved wakeup or wakes one blocked process.

Operation	If Semaphore > 0	If Semaphore $= 0$
down	Decrement the semaphore and continue.	Block the process before completing the operation.
up	Increment the semaphore.	If processes are sleeping on it, wake one of them.

Atomicity Requirement

Checking the semaphore value, changing it, and possibly putting a process to sleep must happen as one indivisible atomic action.

3. The Down Operation

How down Works

down checks whether the semaphore value is greater than 0. If it is, the value is decremented and the process continues. If the value is 0, the process is put to sleep.

```
down(semaphore s)
{
    if (s > 0) {
        s = s - 1;
        continue;
    } else {
        sleep on s;
    }
}
```

Important

The code above is only conceptual. A real **down** operation must be implemented atomically by the operating system.

4. The Up Operation

How up Works

up increments the semaphore. If one or more processes are blocked on that semaphore, the system chooses one of them and allows it to complete its earlier **down**.

```
up(semaphore s)
{
    if (processes are sleeping on s) {
        wake one process blocked on s;
    } else {
        s = s + 1;
    }
}
```

No Blocking on Up

A process never blocks while doing **up**. The operation either saves a wakeup by increasing the semaphore or wakes one process.

5. Why Atomicity Matters

Atomic Action

An **atomic action** is a group of related operations that is performed without interruption. Other processes cannot observe or interfere with the middle of the operation.

Semaphore Safety

Once a semaphore operation starts, no other process may access that semaphore until the operation has completed or the calling process has blocked.

Without Atomicity

If **down** could be interrupted after checking the semaphore but before sleeping, semaphores would have the same lost-wakeup problem as ordinary **sleep** and **wakeup**.

6. Old Names: P and V

Historical Note

Dijkstra originally called the operations **P** and **V**. Modern texts often use **down** and **up** because those names describe the effect on the semaphore value more clearly.

7. Producer-Consumer Using Semaphores

Goal

Use semaphores to solve the producer-consumer problem without lost wakeups and without busy waiting.

The solution uses three semaphores:

Semaphore	Initial Value	Purpose
mutex	1	Controls access to the critical region that modifies the buffer.
empty	N	Counts empty buffer slots.
full	0	Counts full buffer slots.

```
#define N 100 /* number of slots in the buffer */
typedef int semaphore; /* semaphores are a special kind of int */

semaphore mutex = 1; /* controls access to critical region */
semaphore empty = N; /* counts empty buffer slots */
semaphore full = 0; /* counts full buffer slots */

void producer(void)
{
    int item;

    while (TRUE) {
        item = produce_item();
        down(&empty);
        down(&mutex);
        insert_item(item);
        up(&mutex);
        up(&full);
    }
}
```

```

}

void consumer(void)
{
    int item;

    while (TRUE) {
        down(&full);
        down(&mutex);
        item = remove_item();
        up(&mutex);
        up(&empty);
        consume_item(item);
    }
}

```

8. How the Producer Works

Step	Meaning
<code>produce_item()</code>	Create an item outside the critical region.
<code>down(&empty)</code>	Wait until at least one empty buffer slot exists.
<code>down(&mutex)</code>	Enter the critical region safely.
<code>insert_item(item)</code>	Put the item into the shared buffer.
<code>up(&mutex)</code>	Leave the critical region.
<code>up(&full)</code>	Announce that one more full slot exists.

9. How the Consumer Works

Step	Meaning
<code>down(&full)</code>	Wait until at least one full buffer slot exists.
<code>down(&mutex)</code>	Enter the critical region safely.
<code>remove_item()</code>	Remove one item from the shared buffer.
<code>up(&mutex)</code>	Leave the critical region.
<code>up(&empty)</code>	Announce that one more empty slot exists.
<code>consume_item(item)</code>	Use the item outside the critical region.

10. Why This Solves the Lost-Wakeup Problem

Stored Wakeups

The semaphores `empty` and `full` store information about available slots. If an `up` happens before another process does `down`, the semaphore value records that event.

Example

If the producer inserts an item and does `up(&full)` before the consumer waits, `full` becomes positive. Later, the consumer's `down(&full)` consumes that saved wakeup instead of sleeping.

Order Matters

The producer must do `down(&empty)` before entering the buffer, and the consumer must do `down(&full)` before removing from the buffer. Otherwise, the producer could overfill the buffer or the consumer could remove from an empty buffer.

11. Binary and Counting Semaphores

Binary Semaphore

A **binary semaphore** is initialized to 1 and is used to ensure that only one process at a time enters a critical region.

Counting Semaphore

A **counting semaphore** can hold values larger than 1 and is used to count available resources or saved events.

Semaphore	Type	Use in Producer-Consumer
<code>mutex</code>	Binary	Guarantees mutual exclusion for buffer access.
<code>empty</code>	Counting	Counts empty buffer slots.
<code>full</code>	Counting	Counts full buffer slots.

12. Implementation of Semaphores

Single CPU

The operating system can implement `up` and `down` as system calls and briefly disable interrupts while testing and updating the semaphore.

Multiple CPUs

On multiprocessor systems, each semaphore must be protected so that only one CPU examines or updates it at a time. Hardware instructions such as TSL or XCHG can be used for this short internal lock.

Important Distinction

Using TSL or XCHG briefly inside the semaphore implementation is different from making the producer or consumer busy-wait for the buffer. The internal semaphore operation takes only a few instructions; waiting for a buffer slot may take much longer.

13. Semaphores and I/O Interrupts

Device Manager Pattern

An operating system can associate a semaphore, initially 0, with each I/O device. After starting an I/O operation, the device manager does **down** on that semaphore and blocks.

When the device interrupt arrives, the interrupt handler does **up** on the device's semaphore. This makes the device manager ready to run again.



14. Two Uses of Semaphores

Use	Semaphore Example	Purpose
Mutual exclusion	<code>mutex</code>	Ensures only one process at a time reads or writes the shared buffer.
Synchronization	<code>full</code> , <code>empty</code>	Ensures events happen in a valid order: do not consume from an empty buffer or produce into a full one.

Core Distinction

Mutual exclusion protects a critical region. Synchronization controls the order in which processes are allowed to proceed.

15. Quick Review

Key Points

- A semaphore is an integer used to count saved wakeups.
- **down** decrements the semaphore or blocks if the value is 0.
- **up** increments the semaphore or wakes one blocked process.
- Semaphore operations must be atomic.
- Binary semaphores are used for mutual exclusion.
- Counting semaphores are used to count resources or saved events.
- In producer-consumer, `mutex` protects the buffer.
- `empty` counts empty slots, and `full` counts full slots.
- Semaphores can also be used to block device managers until I/O interrupts occur.

Mutexes

Lightweight mutual exclusion for threads and shared resources

Definition

A **mutex** is a simplified semaphore used only for mutual exclusion. It protects a critical region by allowing only one thread or process to hold the lock at a time.

1. Why Mutexes Exist

Main Idea

When the counting ability of a semaphore is not needed, a simpler mechanism can be used. A mutex only records whether a shared resource is unlocked or locked.

Key Use

Mutexes are good for protecting shared data, shared resources, and critical sections. They are not designed to count multiple saved wakeups or multiple available resources.

State	Meaning
Unlocked	The critical region is available. A thread may acquire the mutex.
Locked	Some thread is already inside the critical region. Other threads must wait.

Representation

Only one bit is logically needed, but implementations often use an integer: 0 means unlocked, and any nonzero value means locked.

2. Basic Mutex Operations

Two Main Procedures

Mutexes are normally used with two operations:

- `mutex_lock`: acquire the mutex before entering the critical region.
- `mutex_unlock`: release the mutex after leaving the critical region.

Operation	Behavior
<code>mutex_lock</code>	If the mutex is unlocked, lock it and enter. If it is locked, block or yield until it becomes available.
<code>mutex_unlock</code>	Unlock the mutex and allow one waiting thread to acquire it.
<code>mutex_trylock</code>	Try to acquire the mutex, but return failure instead of blocking if it is already locked.

Cooperation Required

Mutexes are not magic. All threads must follow the protocol: lock before touching shared data and unlock after finishing.

3. User-Space Mutex Implementation

Why User-Space Mutexes Are Useful

Mutexes are simple enough to implement in user-space thread packages, especially when the hardware provides an atomic instruction such as TSL or XCHG.

```
mutex_lock:
    TSL REGISTER, MUTEX ; copy mutex to register and set mutex to 1
    CMP REGISTER, #0 ; was mutex zero?
    JZE ok ; if zero, mutex was unlocked
    CALL thread_yield ; mutex is busy; run another thread
    JMP mutex_lock ; try again

ok:
    RET ; critical region entered

mutex_unlock:
    MOVE MUTEX, #0 ; unlock the mutex
    RET
```

Crucial Difference from Spin Locks

If this mutex cannot acquire the lock, it calls `thread_yield` instead of spinning forever. That gives another thread a chance to run and release the lock.

4. Why Yielding Matters for User-Level Threads

Problem with Busy Waiting

In a user-level thread package, there may be no clock interrupt that preempts one user thread and schedules another. If a thread busy-waits for a lock, it may run forever and prevent the lock holder from running.

Role of `thread_yield`

`thread_yield` calls the user-space thread scheduler. The waiting thread voluntarily gives up the CPU, allowing another thread to run.

Effect

If thread A tries to lock a mutex held by thread B, A yields. B can then run, leave the critical region, and unlock the mutex. Later A can try again and succeed.

Performance Benefit

Because `thread_yield` is a user-space scheduler call, `mutex_lock` and `mutex_unlock` can avoid kernel calls in the uncontended user-thread case.

5. Trylock

Trylock

`mutex_trylock` attempts to acquire a mutex. If the mutex is available, it locks it. If the mutex is already locked, it returns an error code instead of blocking.

Why It Is Useful

Trylock lets a thread choose another action instead of waiting. For example, it may do different work, retry later, or avoid a possible deadlock pattern.

6. Shared Address Space Issue

Threads

Threads in the same process naturally share an address space, so they can all access the same mutex variable.

Processes

Separate processes usually have separate address spaces. For them to share a mutex, semaphore, buffer, or Peterson-style variable, some shared storage must exist.

Approach	Meaning
Kernel object	Semaphores or locks can be stored inside the kernel and accessed through system calls.
Shared memory	Modern systems can map part of one process's address space into another process.
Shared file	If nothing else is available, processes may coordinate through a shared file.

Processes vs. Threads

Even if two processes share much of their address space, they are still not the same as threads. Processes still have separate per-process properties such as open files and timers, while threads in one process share them.

7. Futexes

Futex

A **futex**, or **fast user space mutex**, is a Linux synchronization mechanism that tries to avoid kernel calls when there is no contention, but uses the kernel to block threads when waiting is necessary.

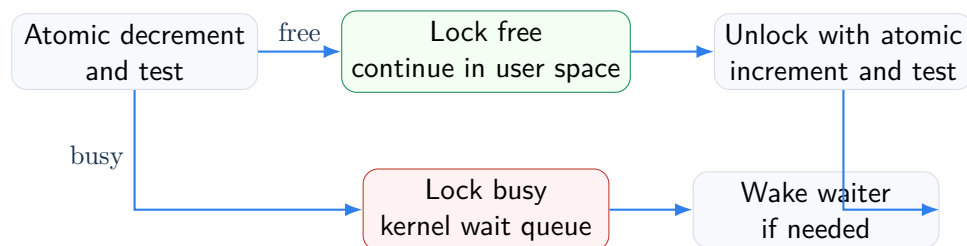
Motivation

Spin locks are fast when waits are very short, but waste CPU cycles under heavy contention. Blocking in the kernel is better under contention, but expensive when locks are usually free.

Best of Both Worlds

A futex stays in user space when the lock is uncontended. It enters the kernel only when a thread must actually wait.

8. How a Futex Works



Typical Futex Lock

Processes share an aligned 32-bit integer as the lock. If an atomic operation shows the lock was free, the thread acquires it in user space. If another thread already holds it, the library makes a system call to put the thread on a kernel wait queue.

Typical Futex Unlock

When the holder releases the lock, it uses an atomic operation and checks whether any threads are blocked in the kernel. If so, it asks the kernel to wake one or more waiters.

9. Pthreads Mutexes

Pthreads Mutex

In Pthreads, a mutex variable is used to guard a critical region. A thread locks the mutex before entering and unlocks it after leaving.

Thread Call	Description
<code>pthread_mutex_init</code>	Create or initialize a mutex.
<code>pthread_mutex_destroy</code>	Destroy an existing mutex.
<code>pthread_mutex_lock</code>	Acquire a lock or block if it is already locked.
<code>pthread_mutex_trylock</code>	Acquire a lock or return failure without blocking.
<code>pthread_mutex_unlock</code>	Release a lock and allow one waiting thread to continue.

Not Mandatory

Pthreads mutexes do not automatically protect data. The programmer must consistently use the correct mutex around every access to the shared data.

10. Condition Variables

Condition Variable

A **condition variable** lets a thread block until some condition becomes true. It is usually used together with a mutex.

Why Mutexes Alone Are Not Enough

A mutex can protect the check of a condition, such as whether a buffer is full. But if the condition is not satisfied, the thread needs a way to block and be awakened later. That is the job of a condition variable.

Thread Call	Description
<code>pthread_cond_init</code>	Create or initialize a condition variable.
<code>pthread_cond_destroy</code>	Destroy a condition variable.
<code>pthread_cond_wait</code>	Block until another thread signals the condition variable.
<code>pthread_cond_signal</code>	Wake one waiting thread.
<code>pthread_cond_broadcast</code>	Wake all waiting threads on the condition variable.

Atomic Wait

`pthread_cond_wait` atomically unlocks the mutex and blocks the calling thread. When the thread wakes up, it reacquires the mutex before returning.

No Memory

Condition variables do not remember signals. If `pthread_cond_signal` is called when no thread is waiting, the signal is lost.

11. Mutexes and Condition Variables Together

Producer-Consumer Pattern

For a buffer, the mutex protects the shared state such as `count`. Condition variables let the producer wait while the buffer is full and let the consumer wait while the buffer is empty.

```
pthread_mutex_t mutex;
pthread_cond_t not_full;
pthread_cond_t not_empty;

int count = 0;
int buffer;

void producer(void)
{
    int item;

    while (TRUE) {
        item = produce_item();

        pthread_mutex_lock(&mutex);
        while (count == 1)
            pthread_cond_wait(&not_full, &mutex);

        buffer = item;
        count = 1;
        pthread_cond_signal(&not_empty);
        pthread_mutex_unlock(&mutex);
    }
}

void consumer(void)
{
    int item;

    while (TRUE) {
        pthread_mutex_lock(&mutex);
        while (count == 0)
            pthread_cond_wait(&not_empty, &mutex);

        item = buffer;
        count = 0;
        pthread_cond_signal(&not_full);
        pthread_mutex_unlock(&mutex);

        consume_item(item);
    }
}
```

Always Recheck the Condition

The waiting thread should use a `while` loop, not a simple `if`. A thread can wake up even though the condition is still false, so it must check again before continuing.

12. Mutexes vs. Semaphores vs. Condition Variables

Primitive	Main Job	Key Detail
Mutex	Mutual exclusion	Locked or unlocked; does not count resources.
Semaphore	Counting and synchronization	Stores wakeups and can count available resources.
Condition variable	Waiting for a condition	Has no memory; must be used with a mutex and condition check.

13. Quick Review

Key Points

- A mutex has two logical states: unlocked and locked.
- Mutexes are simplified semaphores used only for mutual exclusion.
- `mutex_lock` acquires the lock or waits.
- `mutex_unlock` releases the lock and may wake one waiter.
- User-space mutexes can use TSL or XCHG plus `thread_yield`.
- `mutex_trylock` fails instead of blocking when the mutex is busy.
- Processes need shared storage or kernel objects to share synchronization variables.
- Futexes stay in user space unless contention requires kernel blocking.
- Pthreads provides mutex calls and condition-variable calls.
- Condition variables have no memory, so waiting code must recheck the condition.

Monitors

Higher-level synchronization with automatic mutual exclusion

Definition

A **monitor** is a high-level synchronization construct that groups procedures, variables, and data structures into one protected module. Only one process or thread can be active inside a monitor at a time.

1. Why Monitors Were Introduced

Main Idea

Semaphores and mutexes are powerful, but they are easy to use incorrectly. A small mistake in the order of operations can cause deadlock or race conditions.

Semaphore Ordering Problem

In the producer-consumer semaphore solution, the producer should do `down(&empty)` before `down(&mutex)`. If the order is reversed and the buffer is full, the producer can block while holding `mutex`.

Deadlock

If the producer blocks while holding `mutex`, the consumer cannot enter the buffer to remove an item. The consumer also blocks, and both processes wait forever. This is called **deadlock**.

Key Motivation

Monitors reduce programmer error by moving mutual exclusion into the language or runtime system instead of forcing programmers to manually place every lock and unlock correctly.

2. What a Monitor Contains

Monitor Structure

A monitor is like a special module or package containing:

- private variables,
- private data structures,
- procedures that operate on those variables and data structures,
- condition variables for blocking and signaling.

Processes may call monitor procedures, but they cannot directly access the monitor's internal data from outside the monitor.

```

monitor example
  integer i;
  condition c;

  procedure producer();
    ...
  end;

  procedure consumer();
    ...
  end;
end monitor;

```

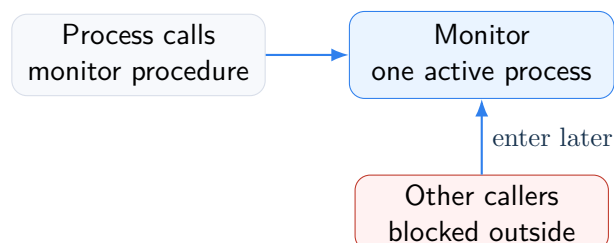
3. Automatic Mutual Exclusion

Monitor Entry Rule

Only one process can be active inside a monitor at any instant.

When a process calls a monitor procedure, the generated code checks whether another process is already active inside the monitor:

- if the monitor is free, the process enters;
- if another process is active, the caller is suspended until the monitor becomes free.



Compiler or Runtime Responsibility

The compiler or runtime arranges the mutual exclusion, often using a mutex or binary semaphore internally. The programmer writes monitor procedures and relies on the monitor rule.

4. Condition Variables in Monitors

Condition Variable

A **condition variable** lets a process inside a monitor block until some condition becomes true.

Mutual exclusion alone is not enough. In producer-consumer, the producer must wait if the buffer is full, and the consumer must wait if the buffer is empty.

Operation	Meaning
<code>wait(condition)</code>	Block the caller and release the monitor so another process can enter.
<code>signal(condition)</code>	Wake one process waiting on that condition, if any exists.

Why Wait Releases the Monitor

If a process waited while still occupying the monitor, no other process could enter to change the condition. Releasing the monitor during `wait` is essential.

5. What Happens After Signal?

The Issue

If one process signals a condition variable and wakes another process, the monitor must still obey the rule that only one process is active inside it at a time.

Proposal	Rule After <code>signal</code>
Hoare	The newly awakened process runs immediately; the signaler is suspended.
Brinch Hansen	A process may signal only as the final action before leaving the monitor.
Common practical variant	The signaler continues, and the awakened process runs after the signaler exits the monitor.

This Note's Simplified Rule

The Brinch Hansen rule is conceptually simple: signal at the end of the monitor procedure, then leave.

6. Condition Variables Have No Memory

Lost Signal

Condition variables are not counters. If `signal` is executed when no process is waiting, the signal is lost.

Difference from Semaphores

Semaphores remember events by incrementing their value. Condition variables do not remember signals. The program must track state with ordinary variables such as `count`.

Rule of Use

A process should test the shared state before waiting. The condition variable only provides the blocking mechanism; the real condition is stored in monitor variables.

7. Producer-Consumer with Monitors

Goal

Use a monitor to protect the buffer automatically and use condition variables to block when the buffer is full or empty.

```
monitor ProducerConsumer
  condition full, empty;
  integer count;

  procedure insert(item: integer);
  begin
    if count = N then wait(full);
    insert_item(item);
    count := count + 1;
    if count = 1 then signal(empty);
  end;

  function remove: integer;
  begin
    if count = 0 then wait(empty);
    remove := remove_item();
    count := count - 1;
    if count = N - 1 then signal(full);
  end;

  count := 0;
end monitor;
```

Condition	Meaning
full	Producer waits here when the buffer has no empty slots.
empty	Consumer waits here when the buffer has no items.
count	Tracks how many items are currently in the buffer.

8. Why Monitors Avoid the Sleep-Wakeup Race

Crucial Difference

Ordinary **sleep** and **wakeup** had a race because one process could try to sleep while another tried to wake it. Inside a monitor, that interleaving cannot happen.

Monitor Protection

If the producer is inside the monitor and discovers that the buffer is full, the consumer cannot enter the monitor until the producer has completed the **wait** operation and is marked as blocked.

Result

The dangerous gap between checking the condition and going to sleep is removed by the monitor's automatic mutual exclusion.

9. Java-Style Monitors

Java Support

Java does not provide classical Hoare-style monitors exactly, but every object can act like a monitor when methods are declared **synchronized**.

Synchronized Methods

If one thread is executing a **synchronized** method of an object, no other thread may execute another **synchronized** method of that same object at the same time.

Java uses `wait()`, `notify()`, and `notifyAll()` for condition-style waiting and signaling inside synchronized code.

10. Java Producer-Consumer Sketch

```
class ProducerConsumerMonitor {
    private final int[] buffer;
    private int count = 0;
    private int lo = 0;
    private int hi = 0;

    ProducerConsumerMonitor(int size) {
        buffer = new int[size];
    }

    public synchronized void insert(int value)
        throws InterruptedException {
        while (count == buffer.length)
            wait();

        buffer[hi] = value;
        hi = (hi + 1) % buffer.length;
        count++;
        notifyAll();
    }

    public synchronized int remove()
        throws InterruptedException {
        while (count == 0)
            wait();

        int value = buffer[lo];
        lo = (lo + 1) % buffer.length;
        count--;
        notifyAll();
        return value;
    }
}
```

Why while Is Used

The waiting condition is checked in a **while** loop because a thread may wake up even though the condition it needs is still false. Rechecking protects correctness.

11. How the Java Monitor Works

Variable / Method	Purpose
buffer	Stores produced items.
count	Counts how many items are currently in the buffer.
lo	Index of the next item to remove.
hi	Index of the next slot to insert into.
insert	Synchronized producer-side monitor procedure.
remove	Synchronized consumer-side monitor procedure.

Safety

When the producer is inside **insert**, the consumer cannot be inside **remove** for the same monitor object. This makes buffer updates safe.

12. Monitors vs. Semaphores

Feature	Semaphores	Monitors
Level	Low-level synchronization primitive.	Higher-level language or runtime construct.
Mutual exclusion	Programmer must use down and up correctly.	Automatic on monitor procedure entry.
Common errors	Wrong order can cause deadlock. Missing operation can cause races.	Fewer locking errors, but condition logic can still be wrong.
Language support	Can be added through system calls or libraries.	Requires language or runtime support.

13. Limitations of Monitors

Language Dependence

Monitors are a programming-language concept. Languages such as C do not have classical monitors built in, so the compiler cannot automatically enforce monitor rules.

Shared-Memory Assumption

Monitors and semaphores assume that processes or threads can share memory. They are not enough for distributed systems where each machine has its own private memory.

Distributed Systems Need More

When computers communicate over a network rather than through shared memory, synchronization primitives alone do not exchange information. Message-passing mechanisms are needed.

14. Quick Review

Key Points

- Monitors were introduced to reduce errors caused by manual semaphore use.
- A monitor groups shared data and procedures into one protected module.
- Only one process can be active inside a monitor at a time.
- Monitor mutual exclusion is handled by the compiler or runtime.
- Condition variables let processes block inside monitors.
- `wait` blocks the caller and releases the monitor.
- `signal` wakes one process waiting on a condition variable.
- Condition variables have no memory, unlike semaphores.
- Java's `synchronized` methods provide monitor-like behavior.
- Monitors require language/runtime support and are mainly for shared-memory systems.

Message Passing

Interprocess communication without shared memory

Definition

Message passing is an interprocess communication method in which processes exchange information by explicitly sending and receiving messages.

Unlike monitors, message passing is not mainly a language construct. The basic operations are usually system calls, and they can be wrapped in library procedures.

1. Basic Primitives

Two Main Operations

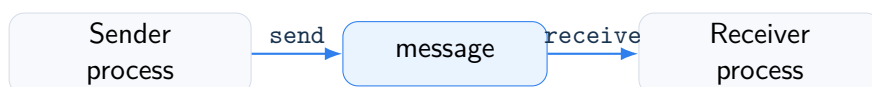
Message passing uses two basic primitives:

- `send(destination, &message)`
- `receive(source, &message)`

Primitive	Meaning
<code>send(destination, &message)</code>	Send a message to a specified destination.
<code>receive(source, &message)</code>	Receive a message from a specified source, or from ANY if the receiver does not care who sent it.

Blocking Receive

If no suitable message is available, **receive** may block until one arrives. Another design is for **receive** to return immediately with an error code.



2. Why Message Passing Is Needed

Main Idea

Semaphores and monitors assume shared memory. Message passing can work even when processes are on different machines and do not share an address space.

Distributed Communication

When two computers communicate over a network, one process cannot simply read or update the other process's memory. Information must be sent as messages.

3. Design Issues in Message-Passing Systems

Extra Problems

Message passing has design problems that do not arise in the same way with semaphores or monitors. These problems become especially important when messages travel across a network.

Issue	Meaning
Lost messages	A message may disappear because of network failure or congestion.
Duplicate messages	A retransmitted message may arrive even though the original already arrived.
Naming	The system must identify the destination of send and the source of receive unambiguously.
Authentication	A process may need proof that it is communicating with the correct service.
Performance	Copying messages is usually slower than using shared-memory synchronization.

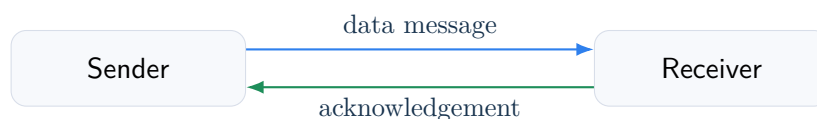
4. Lost Messages and Acknowledgements

Acknowledgement

An **acknowledgement** is a special reply message sent by the receiver to confirm that a message arrived successfully.

Basic Reliability Strategy

The sender transmits a message and waits for an acknowledgement. If the acknowledgement does not arrive within a chosen time interval, the sender retransmits the message.



Timeout

A **timeout** is the maximum time the sender waits for an acknowledgement before assuming that something went wrong.

5. Duplicate Messages and Sequence Numbers

Lost Acknowledgement Problem

Suppose the original message arrives correctly, but the acknowledgement is lost. The sender times out and sends the same message again, so the receiver gets a duplicate.

Sequence Number

A **sequence number** is a number placed in each original message so the receiver can identify old, duplicate, or out-of-order messages.

Duplicate Detection

If the receiver gets a message with the same sequence number as the previous message, it can recognize the message as a retransmission and ignore the duplicate data.

Step	Event	Result
1	Sender sends message #12.	Receiver gets it and processes it.
2	Receiver sends acknowledgement.	The acknowledgement is lost.
3	Sender times out.	Sender retransmits message #12.
4	Receiver sees #12 again.	Receiver treats it as a duplicate.

6. Naming and Authentication

Naming Problem

The system must make sure that the destination named in **send** and the source named in **receive** are clear and unambiguous.

Authentication Problem

A client may think it is talking to a file server, but an imposter process could pretend to be that server. Message systems need ways to prove identities.

7. Performance Cost

Copying Overhead

Copying a message from one process to another is usually slower than doing a semaphore operation or entering a monitor.

Even when the sender and receiver are on the same machine, the operating system may need to copy data between address spaces or manage message buffers.

Implementation Goal

Much work in operating systems and networks focuses on making message passing efficient while preserving isolation between processes.

8. Producer-Consumer Without Shared Memory

Goal

Solve the producer-consumer problem using message passing instead of shared memory, semaphores, or monitors.

Assumptions

- All messages are the same size.
- Messages that have been sent but not yet received are buffered automatically by the operating system.
- A total of N messages is used, similar to N buffer slots.

Key Trick

The consumer starts by sending N empty messages to the producer. The producer must receive an empty message before it can send back a full message.

9. Producer-Consumer Code

```
#define N 100 /* number of slots in the buffer */

void producer(void)
{
    int item;
    message m; /* message buffer */

    while (TRUE) {
        item = produce_item(); /* generate item */
        receive(consumer, &m); /* wait for an empty */
        build_message(&m, item); /* put item in message */
        send(consumer, &m); /* send full message */
    }
}

void consumer(void)
{
    int item, i;
    message m;

    for (i = 0; i < N; i++)
        send(producer, &m); /* send N empties */

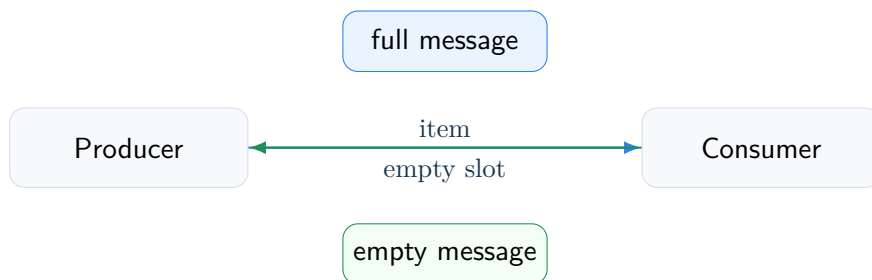
    while (TRUE) {
        receive(producer, &m); /* get full message */
        item = extract_item(&m); /* extract item */
        send(producer, &m); /* return empty */
        consume_item(item); /* use item */
    }
}
```

Constant Number of Messages

The total number of messages in the system stays constant. Messages change between empty and full, but the system never needs more than N message buffers.

10. How Blocking Happens

Situation	Result
Producer is faster	All messages become full and wait for the consumer. The producer blocks waiting for an empty message.
Consumer is faster	All messages become empty and wait for the producer. The consumer blocks waiting for a full message.



11. Direct Process Addressing

Process Names

One way to address messages is to give every process a unique address. Then **send** and **receive** name processes directly.

```
send(consumer, &message)
receive(producer, &message)
```

Naming Requirement

Direct process addressing works only if process names are unique, stable enough to use, and understood by both sides.

12. Mailboxes

Mailbox

A **mailbox** is a system-managed place where messages can be buffered. Processes send messages to a mailbox and receive messages from a mailbox.

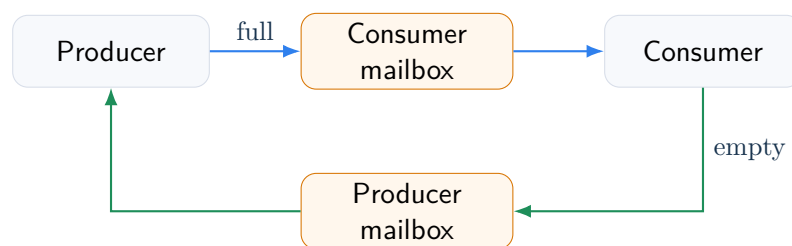
Mailbox Addressing

When mailboxes are used, the address parameters in **send** and **receive** are mailbox names instead of process names.

Mailbox Feature	Meaning
Bounded capacity	A mailbox can hold only a fixed number of messages.
Buffered messages	Messages wait in the mailbox until a receiver accepts them.
Full mailbox	A sender trying to send to a full mailbox is suspended until space becomes available.

Producer-Consumer with Mailboxes

The producer can send full messages to the consumer's mailbox, and the consumer can send empty messages to the producer's mailbox. Each mailbox can be large enough to hold N messages.



13. Rendezvous: No Buffering

Rendezvous

A **rendezvous** is a message-passing design with no buffering. The sender and receiver must meet at the same time for the message transfer to complete.

Order of Operations	Result
send happens first	The sender blocks until a matching receive happens.
receive happens first	The receiver blocks until a matching send happens.
Both are ready	The message can be copied directly from sender to receiver.

Tradeoff

Rendezvous is easier to implement than buffered message passing, but it is less flexible because the sender and receiver are forced to run in lockstep.

14. Buffered vs. Unbuffered Message Passing

Feature	Buffered Messages	Rendezvous
Storage	Messages can wait in system buffers or mailboxes.	No message buffer is kept.
Sender behavior	Sender may continue if buffer space exists.	Sender waits until receiver is ready.
Receiver behavior	Receiver may receive a message already waiting.	Receiver waits until sender is ready.
Flexibility	More flexible.	Less flexible, more synchronized.

15. Message Passing vs. Shared-Memory Synchronization

Feature	Semaphores / Monitors	Message Passing
Memory model	Usually assumes shared memory.	Works without shared memory.
Main purpose	Coordinate access to shared data.	Transfer data and synchronize processes.
Implementation	Semaphore operations or monitor entry rules.	<code>send</code> and <code>receive</code> system calls.
Distributed use	Not enough by themselves across machines.	Natural fit for networked processes.
Cost	Usually lower overhead on one machine.	Usually higher overhead due to copying and buffering.

16. MPI

Message-Passing Interface

MPI stands for Message-Passing Interface. It is a widely used message-passing system in parallel programming, especially scientific computing.

Big Idea

MPI lets programs running on many processors or machines coordinate by explicitly sending and receiving messages.

17. Big Picture

Main Conclusion

Message passing is an interprocess communication method based on `send` and `receive`. It is especially useful when processes do not share memory, but it introduces design issues such as lost messages, duplicate messages, naming, authentication, buffering, and performance.

Exam Shortcut

Remember: message passing uses **send** and **receive**; **receive** may block or return an error; lost messages need acknowledgements and timeouts; duplicates need sequence numbers; mailboxes provide buffering; rendezvous means no buffering.

18. Quick Review

Key Points

- Message passing exchanges information through explicit messages.
- The two basic primitives are **send** and **receive**.
- **receive** can block if no message is available.
- Message passing works even when processes do not share memory.
- Lost messages can be handled with acknowledgements and retransmission.
- Lost acknowledgements can create duplicate messages.
- Sequence numbers help receivers detect duplicates.
- Message systems must solve naming and authentication problems.
- Copying messages can be slower than semaphores or monitors.
- Producer-consumer can be solved by circulating N empty/full messages.
- Mailboxes are buffered destinations for messages.
- Rendezvous message passing has no buffering and forces sender and receiver to meet.
- MPI is a common message-passing system for parallel programming.

Barriers

Group synchronization at phase boundaries

Definition

A **barrier** is a synchronization mechanism that blocks each process at the end of a phase until all processes in the group have reached the same point.

1. Why Barriers Are Used

Main Idea

Some parallel applications are divided into phases. No process may begin the next phase until every process has finished the current phase.

Barriers are different from producer-consumer synchronization. Producer-consumer usually coordinates two sides exchanging data. A barrier coordinates a whole group of processes that must move through a computation together.

Phase Rule

All processes must complete phase n before any process starts phase $n + 1$.

2. Basic Barrier Operation

Barrier Primitive

A process usually reaches a barrier by calling a library procedure such as:

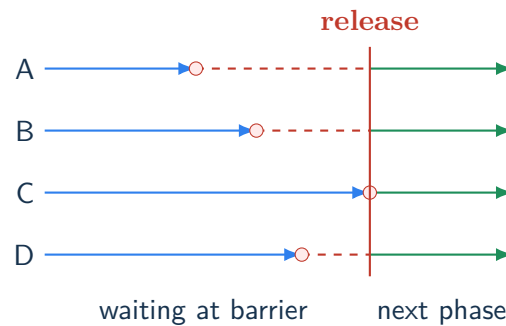
```
barrier()
```

Event	Result
Process reaches barrier early	It is blocked and waits.
Other processes arrive	They also block at the barrier.
Last process arrives	All blocked processes are released together.

Blocking Point

A process that reaches the barrier cannot continue into the next phase by itself. It must wait for the entire group.

3. Barrier Timeline



Reading the Diagram

Processes A, B, and D reach the barrier early and wait. Process C is last. When C arrives, the barrier opens and all processes move into the next phase.

4. Step-by-Step Behavior

Stage	Description
(a) Approaching	Processes are still computing during the current phase.
(b) Waiting	Some processes have reached the barrier and are blocked.
(c) Release	The final process reaches the barrier, so all processes are allowed through.

Group Synchronization

A barrier is useful when the correctness of the next phase depends on every process having completed its current work.

5. Example: Matrix Relaxation

Relaxation Problem

A **relaxation problem** repeatedly updates a large matrix to simulate how a physical system changes over time.

Temperature Simulation

Imagine a matrix whose entries represent temperatures at points on a metal sheet. A flame is placed at one corner, and the program computes how heat spreads through the sheet over time.

Starting from the current matrix, the program applies a transformation to produce the next matrix. For example, it may compute the temperatures after a small time step ΔT . Repeating this produces a sequence of matrices over time.

6. Why the Relaxation Problem Needs Barriers

Parallel Work

If the matrix is extremely large, different processes can compute different parts of the next matrix in parallel.

Main Danger

No process should start iteration $n + 1$ before all processes have finished iteration n . Otherwise, some processes may use old values while others are already using new values.

Barrier Placement

Each process executes `barrier()` after finishing its part of the current iteration. When all processes arrive, the new matrix is complete and everyone can start the next iteration.

7. Matrix Relaxation Sketch

```
for iteration = 0 to max_iterations - 1 do
  compute assigned part of next_matrix from current_matrix

  barrier()

  make next_matrix the current_matrix for the next iteration

  barrier()
end for
```

Why Two Barriers May Appear

The first barrier ensures that all pieces of `next_matrix` are finished. A second barrier may be needed if processes must all finish switching to the new matrix before any process computes again.

8. Barrier Guarantees

Guarantee	Meaning
No early entry	No process enters the next phase before all processes reach the barrier.
Group release	When the last process arrives, all waiting processes can continue.
Phase ordering	The computation proceeds in clear rounds or iterations.

9. What Barriers Do Not Do

Not Mutual Exclusion

A barrier does not protect a critical section. It does not say that only one process may access shared data at a time.

Not Message Passing

A barrier does not itself transfer data between processes. It only coordinates when processes may continue.

Different Purpose

Mutexes, semaphores, and monitors usually protect shared objects or coordinate specific events. Barriers synchronize an entire group at a phase boundary.

10. Simple Implementation Idea

Counter-Based Barrier

A basic barrier can keep a count of how many processes have arrived. If the count is less than the group size, the process waits. If the count reaches the group size, all waiting processes are released.

```
barrier()  
  arrived := arrived + 1  
  
  if arrived < number_of_processes then  
    wait until barrier opens  
  else  
    open barrier and release all waiting processes  
  end if  
end barrier
```

Reusable Barrier

Real barriers are often reused in every iteration, so the implementation must reset safely after all processes have passed through.

11. Common Barrier Problem

Missing Process

If one process crashes, runs forever, or never calls `barrier()`, all other processes may wait at the barrier forever.

Load Imbalance

If one process has much more work than the others, the faster processes spend more time waiting at the barrier. This can reduce parallel performance.

12. Barriers vs. Other Synchronization

Mechanism	Main Use	Typical Scope
Mutex	Protect one critical section.	One shared object or code region.
Semaphore	Count resources or coordinate events.	One resource set or event pattern.
Monitor	Protect shared data with automatic mutual exclusion.	One protected module.
Message passing	Send data and synchronize without shared memory.	Processes or mailboxes.
Barrier	Wait until a whole group reaches the same phase boundary.	A group of cooperating processes.

13. Big Picture

Main Conclusion

Barriers provide group synchronization. When processes are divided into phases, a barrier ensures that no process begins the next phase until all processes have completed the current one.

Exam Shortcut

Remember: a barrier blocks early arrivals, waits for the last process, and then releases the whole group together.

14. Quick Review

Key Points

- Barriers are intended for groups of processes.
- They are useful when a program is divided into phases.
- A process calls `barrier()` at the end of a phase.
- Early processes block at the barrier.
- When the last process arrives, all processes are released.
- Barriers prevent any process from starting phase $n + 1$ too early.
- Matrix relaxation problems often need barriers between iterations.
- A barrier does not provide mutual exclusion by itself.
- If one process never reaches the barrier, the others may wait forever.
- Load imbalance can make fast processes spend time waiting at barriers.

Avoiding Locks: Read-Copy-Update

Concurrent reads with carefully published updates

Definition

Read-Copy-Update, or **RCU**, is a synchronization technique that lets readers access a shared data structure without taking locks, while writers publish new versions carefully and reclaim old versions only after readers are done.

1. Why Avoid Locks?

Main Idea

The fastest lock is no lock at all. If readers can safely run without locking, read-heavy systems can become much faster.

Not Always Possible

In the general case, concurrent reading and writing without locks is unsafe. A reader may observe a temporary, inconsistent state created while a writer is changing the data.

Unsafe Array Example

Suppose process A is sorting an array while process B is computing the average. Because A moves values around, B may count some values twice and miss others entirely. The computed average may be wrong.

2. The RCU Trick

Key Rule

Each reader should see either the old version of the data structure or the new version, but not a broken mixture of both.

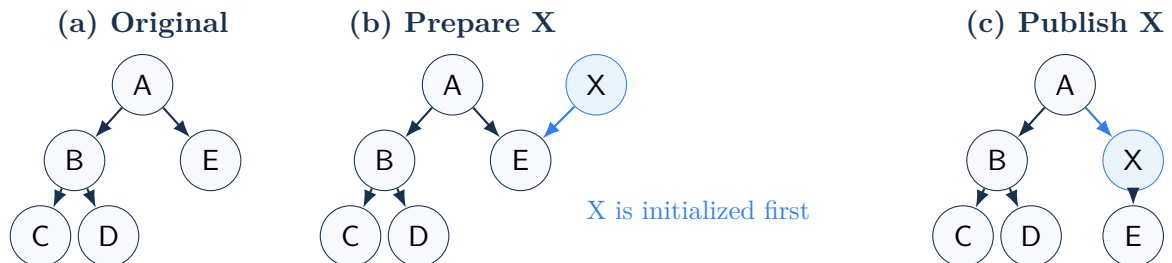
RCU works well for pointer-based data structures such as linked lists and trees. A writer prepares a new or changed piece of the structure privately, then makes it visible with a small atomic pointer update.

Step	Meaning
Read	Readers traverse the current data structure without taking locks.
Copy / prepare	A writer builds or modifies a new version off to the side.
Update	The writer publishes the new version with an atomic pointer change.
Reclaim later	Old memory is freed only after old readers can no longer reference it.

3. Adding a Node Safely

Insertion Idea

Before a new node becomes reachable from the shared tree, the writer completely initializes it. After that, a single atomic pointer update makes the node visible.



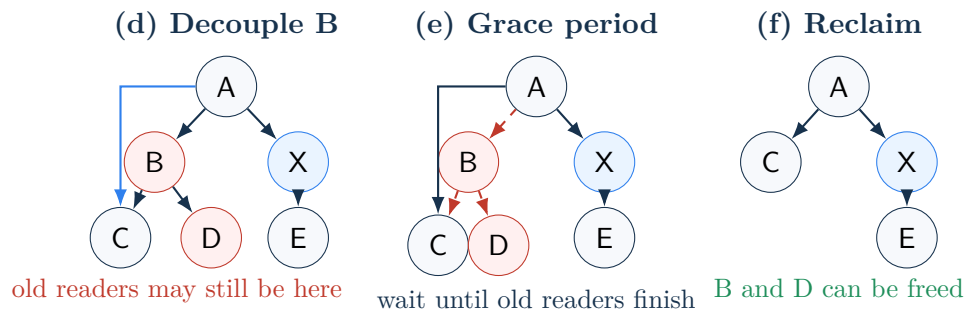
Why Insertion Is Safe

Readers already at node E continue through the old path. Readers that start from A after the atomic update see X fully initialized. No reader sees a half-built node.

4. Removing Nodes Safely

Removal Idea

RCU removal has two separate phases: first remove the old nodes from the shared structure, then reclaim their memory later.



Do Not Free Immediately

After B and D are removed from the public tree, some readers may still hold references to them. Freeing them immediately could leave those readers with dangling pointers.

5. Removal vs. Reclamation

Decoupling

RCU decouples **removal** from **reclamation**. Removal makes old nodes unreachable by new readers. Reclamation frees their memory after old readers are gone.

Phase	Meaning
Removal	Change pointers so new readers no longer reach the old nodes.
Grace period	Wait long enough that every old reader has finished its read-side critical section.
Reclamation	Free the removed nodes safely.

Core Insight

The writer does not need to stop readers from using old nodes. It only needs to avoid freeing those nodes until the readers are finished.

6. Read-Side Critical Sections

Read-Side Critical Section

A **read-side critical section** is the part of a reader that accesses the RCU-protected data structure.

Important Restriction

In the simplified model, code inside an RCU read-side critical section must not block or sleep.

```
rcu_read_lock()

    traverse shared tree
    read nodes and follow pointers

rcu_read_unlock()
```

Why Blocking Is Forbidden

If a reader could sleep while holding a pointer into the old structure, the writer might have to wait an unbounded amount of time before freeing old nodes.

7. Grace Period

Grace Period

A **grace period** is a time interval during which every thread is known to have been outside its RCU read-side critical section at least once.

Safe Reclamation Rule

After a full grace period has passed, readers that could have seen the old version are finished. The removed nodes can then be reclaimed.

Context Switch Criterion

If read-side critical sections cannot block or sleep, one simple way to detect a grace period is to wait until all threads have executed a context switch.

8. Writer Sketch

```
new_node := allocate node
initialize all fields in new_node
initialize all child pointers in new_node

atomically publish pointer to new_node
```

```
old_nodes := remove nodes from public structure

wait for grace period

free old_nodes
```

Atomic Publish

The publishing step must be atomic from the readers' point of view. A reader should never observe a pointer to a partially initialized node.

9. When RCU Works Well

Good Fit	Reason
Read-heavy data	Readers avoid lock overhead.
Pointer-based structures	Writers can publish new links atomically.
Short read-side sections	Grace periods finish quickly.
Old versions can coexist briefly	Readers can safely finish on the version they already saw.

10. Limitations

Not a General Replacement for Locks

RCU does not make every shared update safe. It is most useful when readers can tolerate seeing either an old version or a new version.

Memory Reclamation Is Hard

The writer must know when old readers are gone. Reclaiming too early can cause use-after-free errors.

Reader View

RCU gives readers a consistent traversal, not necessarily the newest possible data. A reader may legally see the old version while another reader sees the new version.

11. RCU vs. Locks

Feature	Locks	RCU
Reader cost	Readers acquire and release a lock.	Readers usually avoid normal lock acquisition.
Writer behavior	Writer often excludes readers.	Writer publishes a new version while old readers continue.
Reader view	Usually sees the protected current state.	Sees either an old consistent version or a new consistent version.
Memory freeing	Freeing can happen after the lock-protected update.	Freeing must wait for a grace period.
Best use	General mutual exclusion.	Read-heavy pointer-based data structures.

12. Big Picture

Main Conclusion

RCU avoids locking readers by allowing old and new versions of a data structure to coexist temporarily. Writers publish updates atomically and reclaim removed data only after a grace period proves that old readers are gone.

Exam Shortcut

Remember: RCU means readers run without normal locks, writers copy or prepare updates, publish with an atomic pointer change, wait for a grace period, and only then reclaim old memory.

13. Quick Review

Key Points

- Avoiding locks can improve performance, especially for read-heavy data.
- Lock-free concurrent access is not safe in the general case.
- RCU makes sure readers see either the old version or the new version.
- A new node is fully initialized before it is made visible.
- Publishing is done with an atomic pointer update.
- Removing nodes does not mean freeing them immediately.
- RCU separates removal from memory reclamation.
- Removed nodes are freed only after a grace period.
- A grace period proves that old readers have left their read-side critical sections.
- Read-side critical sections should not block or sleep in the simplified model.
- RCU is useful for read-heavy pointer-based structures, but it is not a universal replacement for locks.

Scheduling

Choosing which ready process or thread runs next

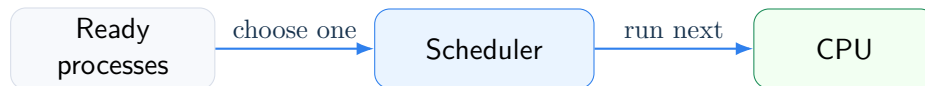
Definition

Scheduling is the operating system activity of choosing which ready process or thread should run on the CPU next.

1. Basic Idea

Why Scheduling Is Needed

In a multiprogrammed system, several processes or threads may be ready to run at the same time. If there is only one CPU, the operating system must choose one of them.



Term	Meaning
Scheduler	The part of the operating system that chooses the next process or thread to run.
Scheduling algorithm	The rule or strategy used by the scheduler to make that choice.
Ready state	The process or thread can run, but is waiting for CPU time.

2. Processes and Threads

Same Basic Problem

Many scheduling ideas apply to both processes and threads: the CPU is limited, and several runnable execution units may compete for it.

Kernel Threads

When the kernel manages threads, scheduling is usually done per thread. The scheduler may care little about which process owns each thread.

3. Historical Motivation

Batch Systems

Early batch systems often ran jobs one after another. With multiprogramming, multiple jobs and users could wait for service, so scheduling became more important.

Batch and Timesharing Together

A mainframe may need to choose between a background batch job and an interactive terminal user. The scheduler affects both system efficiency and user satisfaction.

4. Why Scheduling Matters More in Some Systems

System Type	Scheduling Importance
Simple personal computer	Often only one process is actively demanding the CPU, so scheduling may not be very visible.
CPU-heavy workstation	Rendering, compiling, simulation, and other heavy tasks can make CPU scheduling important.
Network server	Many processes may compete to serve users, so choosing the right one matters.
Mobile or sensor device	CPU, memory, and battery may be limited, so scheduling may also consider energy use.

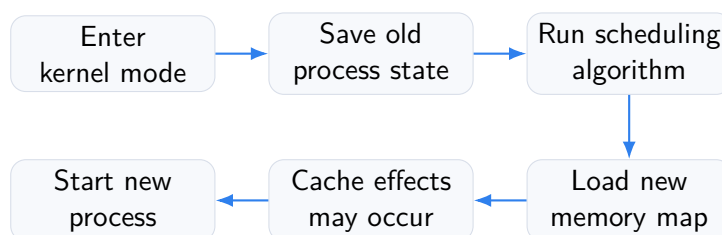
Server Example

If a server must choose between collecting daily statistics and serving a user request, users usually benefit if the request-handling process runs first.

5. Cost of Process Switching

Switching Is Not Free

Changing from one process to another takes CPU time. Too many switches per second can waste a significant amount of processing power.



Typical Work During a Switch

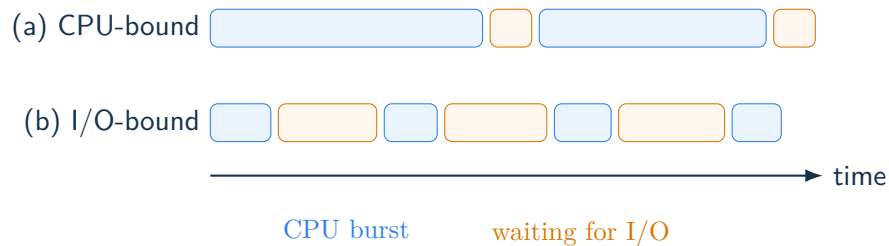
- switch from user mode to kernel mode,
- save registers and other state of the current process,
- choose a new ready process,
- reload memory-management information for the new process,
- restart execution in the selected process,
- possibly reload cache and related hardware state.

6. Process Behavior

CPU Burst

A **CPU burst** is a period during which a process is actively using the CPU before it blocks, usually for I/O.

Most processes alternate between CPU bursts and waiting for I/O such as disk or network operations.



7. CPU-Bound and I/O-Bound Processes

Type	CPU Burst Pattern	Meaning
CPU-bound	Long CPU bursts, infrequent I/O waits.	The process spends most of its time computing.
I/O-bound	Short CPU bursts, frequent I/O waits.	The process often blocks waiting for external devices.

Key Factor

A process is I/O-bound because it does little computation between I/O requests. The important scheduling clue is the length of the CPU burst, not the length of the I/O operation.

Why I/O-Bound Processes Often Deserve Quick Service

If an I/O-bound process gets the CPU quickly, it can issue its next disk or network request quickly. This helps keep I/O devices busy and improves overall system balance.

8. When to Schedule

Scheduling Decision Point

A **scheduling decision point** is a moment when the operating system may need to choose a process or thread to run.

Situation	Why Scheduling May Be Needed
New process is created	Both parent and child may be ready, so the scheduler chooses which runs next.
Process exits	The exiting process cannot continue, so another ready process must be chosen.
Process blocks	A process waiting for I/O, a semaphore, or another event cannot use the CPU.
I/O interrupt occurs	A blocked process may become ready after the device completes its work.
Clock interrupt occurs	A running process may have used up its allowed CPU time.

Idle Process

If no ordinary process is ready, the system normally runs an idle process until useful work becomes available.

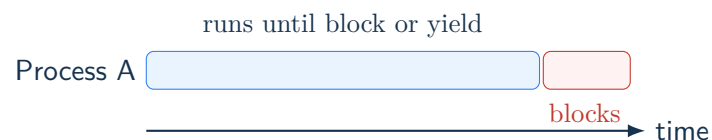
9. Nonpreemptive Scheduling

Nonpreemptive Scheduling

A **nonpreemptive** scheduling algorithm lets the chosen process run until it blocks or voluntarily gives up the CPU.

Main Property

Clock interrupts do not normally force the current process off the CPU. After interrupt handling, the same process usually resumes.



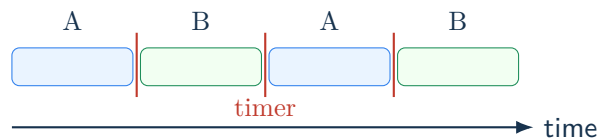
10. Preemptive Scheduling

Preemptive Scheduling

A **preemptive** scheduling algorithm lets a process run for at most a fixed time interval. If it is still running when the interval ends, it can be suspended.

Clock Requirement

Preemptive scheduling requires a clock interrupt or timer mechanism so control returns to the scheduler.



11. Categories of Scheduling Algorithms

Different Environments, Different Goals

Scheduling algorithms are designed differently because batch systems, interactive systems, and real-time systems optimize for different outcomes.

Category	Description
Batch	Jobs run without an impatient user waiting at a terminal. Long time slices or nonpreemptive algorithms may be acceptable.
Interactive	Users expect quick responses. Preemption is important so one process cannot monopolize the CPU.
Real time	Programs have deadlines or timing constraints. The scheduler must make timing behavior reliable.

12. Batch Scheduling

Batch Environment

Batch systems run jobs such as payroll, inventory, billing, banking calculations, or insurance processing. Users usually wait for final output, not immediate interaction.

Why Long Runs Can Be Acceptable

Because there is no interactive user waiting for each keystroke response, batch systems can often use nonpreemptive scheduling or long time periods. This reduces process-switching overhead.

13. Interactive Scheduling

Interactive Environment

Interactive systems include personal computers, terminals, and many servers. They must keep service responsive for users.

Need for Preemption

Without preemption, one CPU-hungry or buggy process could keep running and deny service to other users or programs.

Foreground Before Background

If a user opens a file while a background task is checking email, the user-visible request should normally get quick CPU service.

14. Real-Time Scheduling

Real-Time Environment

Real-time systems run tasks with timing requirements. A process may have to run before a deadline to avoid losing data or degrading output quality.

Difference from General Interactive Systems

Real-time systems usually run programs designed for the application. Interactive systems may run arbitrary programs, including badly behaved ones.

15. Goals for All Systems

Goal	Meaning
Fairness	Comparable processes should receive comparable service.
Policy enforcement	The scheduler should enforce stated system priorities and rules.
Balance	The CPU, memory, and I/O devices should be kept busy when possible.

Balance Example

A good mix of CPU-bound and I/O-bound processes can keep both the CPU and disk busy. Running all CPU-bound jobs first and all I/O-bound jobs later can leave one resource idle while another is overloaded.

16. Batch System Goals

Goal	Meaning
Throughput	Maximize the number of jobs completed per hour.
Turnaround time	Minimize the time from job submission to job completion.
CPU utilization	Keep the CPU busy as much as possible.

Throughput vs. Turnaround

A scheduler that always runs short jobs may complete many jobs per hour, but long jobs may wait too long or even starve if short jobs keep arriving.

CPU Utilization Caution

High CPU utilization is useful as a capacity signal, but it is not the whole goal. Users usually care more about completed work and waiting time.

17. Interactive System Goals

Goal	Meaning
Response time	Minimize the time between a user request and the system's response.
Proportionality	Make response times match user expectations about how long tasks should take.

Proportionality

Users expect uploading a large video to take time, but they expect disconnecting from the server afterward to be quick. A short-looking operation that takes too long feels worse than a long-looking operation that takes time.

18. Real-Time System Goals

Goal	Meaning
Meeting deadlines	Run required tasks before their timing deadlines.
Predictability	Avoid irregular timing that causes quality degradation, especially in multimedia.

Multimedia Example

Occasional delay may not crash the system, but irregular audio scheduling can quickly reduce perceived quality. Predictable timing matters.

19. Scheduling Goals Summary

Environment	Main Goals
All systems	Fairness, policy enforcement, balance.
Batch	Throughput, turnaround time, CPU utilization.
Interactive	Response time, proportionality.
Real time	Meeting deadlines, predictability.

20. Simple Scheduler Sketch

```
while system is running do
  if current process blocks, exits, or is preempted then
    choose a ready process according to the scheduling policy
    switch to the chosen process
  end if
end while
```

Policy Matters

The scheduler's policy determines what best means: shortest waiting time, highest priority, earliest deadline, best response time, or some other goal.

21. Big Picture

Main Conclusion

Scheduling decides which ready process or thread gets the CPU next. Good scheduling depends on the environment: batch systems care about completed jobs, interactive systems care about quick response, and real-time systems care about timing guarantees.

Exam Shortcut

Remember: scheduler chooses the next ready process; CPU-bound means long CPU bursts; I/O-bound means short CPU bursts; nonpreemptive runs until block or yield; preemptive uses timer interrupts; different systems have different scheduling goals.

22. Quick Review

Key Points

- Scheduling is needed when multiple ready processes or threads compete for one CPU.
- The scheduler is the OS component that chooses what runs next.
- The scheduling algorithm is the rule used to make that choice.
- Process switching has overhead, so excessive switching wastes CPU time.
- Processes alternate CPU bursts with I/O waits.
- CPU-bound processes have long CPU bursts.
- I/O-bound processes have short CPU bursts and frequent I/O waits.
- Scheduling decisions happen on creation, exit, blocking, I/O interrupts, and clock interrupts.
- Nonpreemptive scheduling lets a process run until it blocks or yields.
- Preemptive scheduling can suspend a process after a time interval.
- Batch, interactive, and real-time systems have different scheduling goals.
- Fairness, policy enforcement, and balance matter in all systems.
- Batch systems emphasize throughput, turnaround time, and CPU utilization.
- Interactive systems emphasize response time and proportionality.
- Real-time systems emphasize deadlines and predictability.

Scheduling in Batch Systems

First-come first-served, shortest job first, and shortest remaining time next

Definition

Batch scheduling chooses the order in which noninteractive jobs run. Since users are usually not waiting for immediate responses, batch schedulers often focus on throughput, turnaround time, and CPU utilization.

1. Where Batch Scheduling Fits

Main Idea

Batch systems run jobs submitted for later completion, such as payroll, inventory, banking calculations, or insurance claim processing.

Different From Interactive Scheduling

In batch systems, there is usually no user sitting at a terminal expecting a quick response to each command. This makes nonpreemptive algorithms or long time slices more acceptable.

Some scheduling algorithms are useful in both batch and interactive systems, but the classic batch algorithms are easiest to understand first.

2. Batch Scheduling Goals

Goal	Meaning
Throughput	Complete as many jobs per hour as possible.
Turnaround time	Minimize the time from job submission to job completion.
CPU utilization	Keep the CPU busy while avoiding excessive switching overhead.

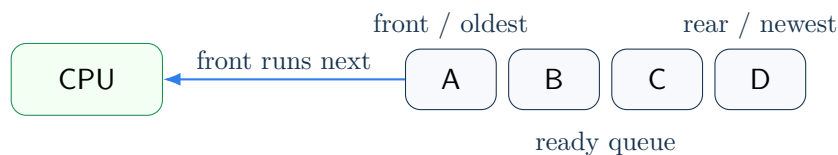
3. First-Come, First-Served

FCFS

First-come, first-served, or **FCFS**, is a nonpreemptive scheduling algorithm that gives the CPU to processes in the order they request it.

Ready Queue

FCFS uses a single queue of ready processes. New jobs and newly unblocked processes are added to the end of the queue. The scheduler removes the next process from the front.



Nonpreemptive Rule

Once a process starts running under FCFS, it is allowed to run until it blocks or finishes. It is not interrupted just because it has run for a long time.

4. FCFS Advantages

Advantage	Explanation
Simple	The scheduler only needs a FIFO queue.
Easy to implement	Add to the back, remove from the front.
Fair in arrival order	Jobs are served in the order they requested the CPU.
Low overhead	It avoids frequent preemptions and context switches.

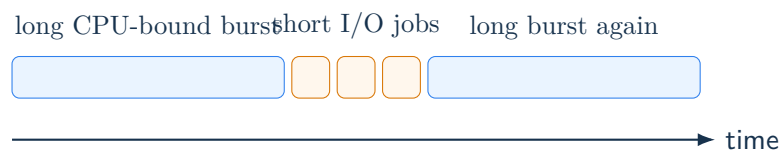
5. FCFS Disadvantage

Convoy Effect

FCFS can make many short I/O-bound processes wait behind one long CPU-bound process. This is often called the **convoy effect**.

Why I/O-Bound Jobs Suffer

Suppose one CPU-bound process runs for 1 second at a time, while many I/O-bound processes need only a little CPU time before starting disk reads. Under FCFS, the I/O-bound processes may wait behind the long CPU burst again and again.



Better With Preemption

If the long CPU-bound process were preempted every 10 msec, the I/O-bound processes could issue disk requests much sooner and finish far faster.

6. Shortest Job First

SJF

Shortest job first, or **SJF**, is a nonpreemptive batch scheduling algorithm that runs the waiting job with the shortest known running time.

Main Assumption

SJF assumes the scheduler knows the run time of each job in advance. This can be realistic in repeated batch workloads where similar jobs are run regularly.

Insurance Company Example

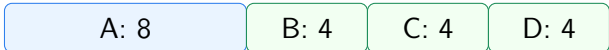
If an insurance company runs the same type of claim-processing batch every day, it may be able to estimate how long that job will take.

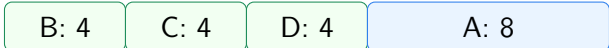
7. SJF Turnaround Example

Original Job Lengths

Suppose four jobs have running times:

$$A = 8, \quad B = 4, \quad C = 4, \quad D = 4$$

(a) Original 

(b) SJF 

—————→ time

Order	Completion Times	Average Turnaround
A, B, C, D	8, 12, 16, 20	$(8 + 12 + 16 + 20)/4 = 14$ minutes.
B, C, D, A	4, 8, 12, 20	$(4 + 8 + 12 + 20)/4 = 11$ minutes.

Result

Running the shorter jobs first reduces the average turnaround time from 14 minutes to 11 minutes in this example.

8. Why SJF Is Optimal

Turnaround Formula

For four jobs with running times a , b , c , and d , the average turnaround time is:

$$\frac{4a + 3b + 2c + d}{4}$$

Reasoning

The first job contributes to every later completion time, so it has the largest effect on the average. Therefore the shortest job should go first, the next shortest second, and the longest last.

Important Condition

Shortest job first is optimal only when all jobs are available at the same time and their running times are known.

9. SJF Limitation: Arrival Times

Counterexample

Consider five jobs with run times $A = 2$, $B = 4$, $C = 1$, $D = 1$, and $E = 1$. Jobs A and B arrive at time 0. Jobs C, D, and E arrive at time 3.

Order	Why It Happens	Average Turnaround
A, B, C, D, E	SJF initially chooses A because C, D, and E have not arrived yet.	4.6
B, C, D, E, A	Not the immediate SJF choice, but gives a lower average turnaround in this case.	4.4

Lesson

Knowing future arrivals can change the best schedule. Simple SJF only chooses among jobs that are currently available.

10. Shortest Remaining Time Next

SRTN

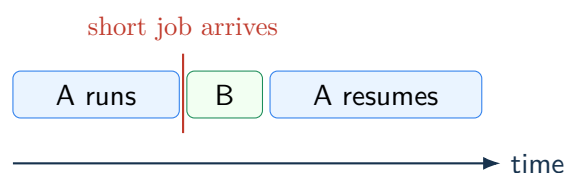
Shortest remaining time next, or **SRTN**, is the preemptive version of shortest job first. The scheduler always runs the process with the smallest remaining time.

How It Works

When a new job arrives, the scheduler compares the new job's total required time with the currently running process's remaining time.

Preemption Rule

If the new job can finish sooner than the current process's remaining time, the current process is suspended and the new job starts.



11. SJF vs. SRTN

Feature	Shortest Job First	Shortest Remaining Time Next
Preemption	Nonpreemptive.	Preemptive.
Choice rule	Run the shortest available job.	Run the job with the shortest remaining time.
New short job arrives	Current job keeps running.	Current job may be suspended.
Main benefit	Minimizes average turnaround when all jobs are available together.	Gives very good service to newly arriving short jobs.
Main assumption	Job lengths are known.	Job lengths and remaining times are known.

12. Big Picture

Main Conclusion

Batch scheduling algorithms try to improve overall job completion behavior. FCFS is simple and fair by arrival order, but can make short I/O-bound jobs wait. SJF reduces average turnaround when job lengths are known. SRTN improves on SJF by preempting long jobs when shorter jobs arrive.

Exam Shortcut

Remember: FCFS is nonpreemptive FIFO; SJF is nonpreemptive shortest known job first; SRTN is preemptive shortest remaining time first.

13. Quick Review

Key Points

- Batch systems run noninteractive jobs.
- Batch scheduling often focuses on throughput, turnaround time, and CPU utilization.
- FCFS runs jobs in arrival order.
- FCFS is simple, fair by queue order, and easy to implement.
- FCFS can make short I/O-bound jobs wait behind long CPU-bound jobs.
- SJF runs the shortest available job first.
- SJF assumes job running times are known in advance.
- SJF minimizes average turnaround time when all jobs are available simultaneously.
- SJF is not always optimal when jobs arrive at different times.
- SRTN is the preemptive version of SJF.
- SRTN runs the job with the least remaining time.
- SRTN can suspend the current process when a shorter job arrives.

Round-Robin Scheduling

Fair preemptive scheduling with a fixed time quantum

Definition

Round-robin scheduling is a preemptive scheduling algorithm in which each runnable process receives the CPU for a fixed time interval called a **quantum**.

1. Main Idea

Fair Rotation

Round robin cycles through the ready processes. Each process gets a turn on the CPU, then the scheduler moves on to the next runnable process.

Quantum

A **quantum** is the maximum amount of time a process may run before the scheduler can preempt it and give the CPU to another process.

Situation	Scheduler Action
Process uses entire quantum	Preempt it and put it at the end of the ready list.
Process blocks before quantum ends	Switch immediately because the process cannot continue.
Process finishes before quantum ends	Switch immediately because the process is done.

2. Ready List Rotation

Implementation

Round robin is easy to implement with a list or queue of runnable processes. The scheduler takes a process from the front, runs it, and if it is still runnable after its quantum, places it at the back.

(a) Before B runs



(b) After B uses its quantum



Key Movement

When B uses its whole quantum, B goes to the end of the list and F becomes the next current process.

3. Round-Robin Scheduler Sketch

```
while ready list is not empty do
  p := remove process from front of ready list

  run p for at most one quantum

  if p finished then
    discard p
  else if p blocked then
    wait for p to become ready again
  else
    add p to the end of the ready list
  end if
end while
```

4. Why Quantum Length Matters

Main Tradeoff

The only really interesting design issue in round robin is choosing the quantum length.

Quantum Choice	Consequence
Too short	Too many context switches, lowering CPU efficiency.
Too long	Poor response time for short requests near the end of the queue.
Reasonable middle	Balances responsiveness with context-switch overhead.

5. Context-Switch Overhead

Context Switch

A **context switch** is the work needed to stop one process and start another, including saving and loading registers, memory maps, tables, and possibly cache state.

Overhead Fraction

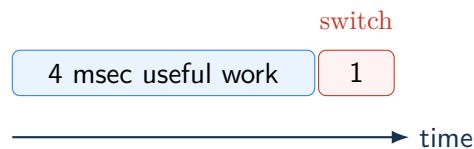
If a process uses its full quantum, the approximate wasted fraction is:

$$\frac{\text{context switch time}}{\text{quantum} + \text{context switch time}}$$

Short Quantum Example

Suppose a context switch takes 1 msec and the quantum is 4 msec. After 4 msec of useful work, the CPU spends 1 msec switching. The overhead is:

$$\frac{1}{4 + 1} = 20\%$$



Too Much Waste

Spending 20% of CPU time on administrative switching overhead is usually too much.

6. Longer Quantum Improves Efficiency

Longer Quantum Example

If the context switch is still 1 msec but the quantum is 100 msec, the overhead is approximately:

$$\frac{1}{100 + 1} \approx 1\%$$

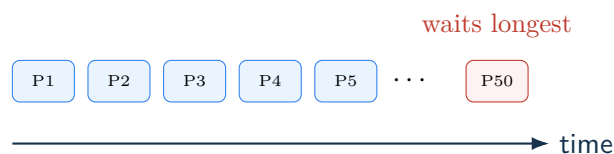
Efficiency Benefit

A longer quantum reduces the percentage of CPU time lost to switching overhead.

7. Long Quantum Hurts Response Time

Server Example

Suppose 50 requests arrive in a short time, and each process may use a full 100 msec quantum. The last process may wait about 5 seconds before it gets its first chance to run.



User Perception

Even if the CPU is efficient, a short interactive request that waits several seconds before starting will feel sluggish.

8. Quantum and CPU Bursts

Mean CPU Burst

If the quantum is longer than the average CPU burst, preemption will not happen often. Most processes will block before their quantum expires.

Why This Helps

When processes block before the quantum ends, context switches happen because they are logically necessary. This avoids unnecessary timer-based preemptions.

Case	Effect
Quantum shorter than many CPU bursts	Frequent preemption and high switching overhead.
Quantum much longer than CPU bursts	Fewer preemptions, but possible poor response under load.
Quantum near a good compromise	Acceptable overhead and acceptable response time.

9. Choosing a Good Quantum

Too Short

A very short quantum gives good responsiveness in theory, but can waste CPU time on constant context switches.

Too Long

A very long quantum behaves more like first-come, first-served scheduling and can make short requests wait behind long ones.

Common Compromise

A quantum around 20–50 msec is often a reasonable compromise between CPU efficiency and response time.

10. Round Robin vs. FCFS

Feature	FCFS	Round Robin
Preemption	Nonpreemptive.	Preemptive after a quantum.
Ready structure	FIFO queue.	Circular ready queue or list.
Long CPU-bound job	Can hold CPU until it blocks.	Loses CPU when its quantum expires.
Short interactive request	May wait behind a long job.	Gets a turn after at most one pass through the queue.
Overhead	Low switching overhead.	Depends strongly on quantum length.

11. Quick Review

Key Points

- Round robin is a preemptive scheduling algorithm.
- Each process gets a fixed time quantum.
- If a process uses its full quantum, it is moved to the end of the ready list.
- If a process blocks or finishes early, the CPU switches immediately.
- Round robin is simple and fair because processes take turns.
- A short quantum improves responsiveness but increases context-switch overhead.
- A long quantum improves CPU efficiency but can hurt response time.
- A 4 msec quantum with a 1 msec switch wastes about 20% of CPU time.
- A 100 msec quantum with a 1 msec switch wastes about 1% of CPU time.
- If the quantum is longer than the average CPU burst, most processes block before preemption.
- A quantum around 20–50 msec is often a reasonable compromise.

Priority Scheduling

Choosing the runnable process with the highest priority

Definition

Priority scheduling is a scheduling method in which every process has a priority, and the runnable process with the highest priority is selected to run.

1. Why Priority Scheduling Is Needed

Main Idea

Round-robin scheduling treats runnable processes as equally important. Priority scheduling allows the system to represent that some processes should run before others.

Multiuser System

In a university system, the operating policy might give higher priority to administrative or faculty work than to ordinary student jobs.

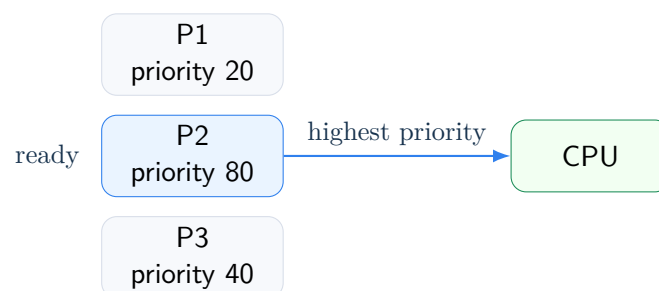
Single-User PC

Even on one user's computer, a real-time video display process may deserve higher priority than a background mail-sending daemon.

2. Basic Rule

Priority Rule

Among all runnable processes, run the process with the highest priority.



3. Avoiding Infinite Running

Problem

If a high-priority process is always runnable, lower-priority processes may never get the CPU.

Two Common Controls

- decrease the priority of the currently running process at each clock tick;
- assign each process a maximum time quantum, then run the next-highest-priority process when the quantum is used up.

Process Switch

If priority reduction makes the running process drop below another runnable process, the scheduler may switch to the higher-priority process.

4. Static Priorities

Static Priority

A **static priority** is assigned before or when the process starts and does not automatically change because of recent behavior.

Example	Possible Priority Policy
Military system	Processes started by generals may get priority 100, colonels 90, majors 80, and so on.
Commercial center	High-priority jobs may cost more per hour than medium- or low-priority jobs.
UNIX-like system	The nice command lets a user voluntarily lower a process's priority.

Policy-Based

Static priorities are useful when priority should reflect external policy, payment level, user category, or administrative importance.

5. Dynamic Priorities

Dynamic Priority

A **dynamic priority** is adjusted by the system while the process runs, often to improve responsiveness or system balance.

Why Dynamic Priorities Help

The scheduler can reward behavior that helps system performance, such as blocking quickly for I/O instead of using the whole CPU quantum.

6. Giving I/O-Bound Processes Good Service

I/O-Bound Motivation

An I/O-bound process often needs only a little CPU time before issuing its next I/O request. Running it quickly helps keep the I/O device busy while other processes compute.

Bad Delay

If an I/O-bound process waits too long for the CPU, it may occupy memory while doing nothing, and the I/O device may sit idle longer than necessary.

7. Dynamic Priority Formula

Priority Based on Quantum Use

One simple dynamic rule is:

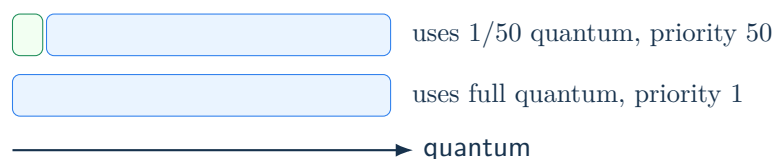
$$\text{priority} = \frac{1}{f}$$

where f is the fraction of the last quantum that the process used.

Last Quantum Use	Fraction f	Priority $1/f$
1 msec of 50 msec	$1/50$	50
25 msec of 50 msec	$1/2$	2
50 msec of 50 msec	1	1

Interpretation

A process that blocks quickly gets a high priority next time. A process that uses the whole quantum gets a low priority.



8. Priority Classes

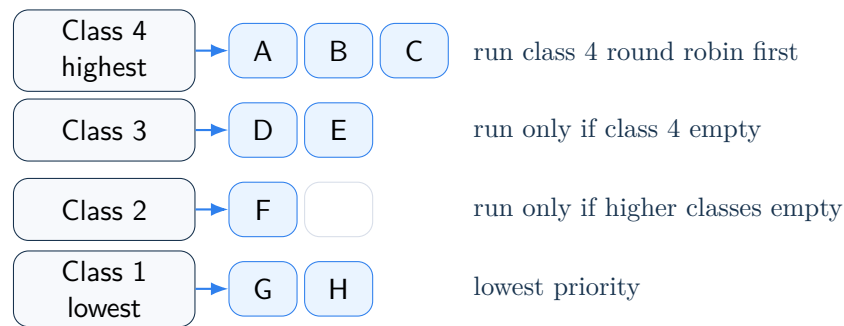
Priority Class

A **priority class** is a group of processes with the same broad priority level. The scheduler chooses among classes first, then chooses within a class.

Common Combination

Priority scheduling can be used among priority classes, while round-robin scheduling is used within each class.

9. Four Priority Classes



Scheduling Rule

As long as any class 4 process is runnable, run class 4 processes round robin. If class 4 is empty, run class 3 round robin. Continue downward only when all higher classes are empty.

10. Starvation

Starvation

Starvation occurs when a process waits indefinitely because higher-priority processes keep getting the CPU.

Priority Class Problem

If class 4 always has runnable processes, classes 3, 2, and 1 may never run.

Aging

One common fix is **aging**: gradually increase the priority of processes that have waited too long, so they eventually get a chance to run.

11. Priority Scheduling Sketch

```
while ready processes exist do
  choose the highest-priority nonempty class
  choose a process from that class

  run the process for one quantum

  if the process is still runnable then
    put it back in the appropriate ready queue
  end if

  adjust priorities if the policy requires it
end while
```

12. Priority Scheduling vs. Round Robin

Feature	Round Robin	Priority Scheduling
Basic assumption	All processes are roughly equal.	Some processes are more important.
Selection rule	Run the next process in queue order.	Run the highest-priority runnable process.
Fairness style	Fair by turn-taking.	Fair only within the chosen priority policy.
Risk	Poor service for important work.	Starvation of low-priority work.
Common mix	One ready queue.	Priority classes with round robin inside each class.

13. Big Picture

Main Conclusion

Priority scheduling lets the operating system prefer important processes over less important ones. Priorities may be static or dynamic, and priority classes are often combined with round-robin scheduling. The main danger is starvation of low-priority processes.

Exam Shortcut

Remember: priority scheduling runs the highest-priority runnable process; dynamic priorities can favor I/O-bound processes; priority classes often use round robin inside each class; starvation must be prevented by priority adjustment or aging.

14. Quick Review

Key Points

- Priority scheduling assigns a priority to each process.
- The highest-priority runnable process is selected to run.
- Priority scheduling is useful when not all processes are equally important.
- A high-priority process can be limited by lowering its priority over time or by using a maximum quantum.
- Static priorities are assigned from outside policy or user choice.
- Dynamic priorities are adjusted by the system during execution.
- I/O-bound processes can be rewarded because they often use only a small fraction of their quantum.
- A simple dynamic rule is $\text{priority} = 1/f$, where f is the fraction of the last quantum used.
- Priority classes can use priority scheduling among classes and round robin within a class.
- Lower-priority classes may starve if priorities are never adjusted.
- Aging raises the priority of long-waiting processes to reduce starvation.

Multiple Queues

Priority classes with different time quanta

Definition

Multiple-queue scheduling uses several ready queues, usually ordered by priority. Each queue may have its own time quantum and scheduling rule.

1. Why Multiple Queues Were Introduced

Main Problem

Round robin gives every process the same quantum. This is simple, but it does not handle all workloads well.

Different Needs

Interactive processes need quick response, so they benefit from short quanta and high priority. CPU-bound processes can run efficiently with larger quanta, but they should not delay interactive work.

2. CTSS Motivation

Historical Context

One early priority scheduler was used in CTSS, the M.I.T. Compatible Time-Sharing System, on the IBM 7094.

Slow Process Switching

The IBM 7094 could hold only one process in memory. Switching processes meant writing the current process to disk and reading another one from disk. This made process switching very expensive.

Scheduling Goal

CTSS needed good interactive response while avoiding too many expensive swaps for CPU-bound processes.

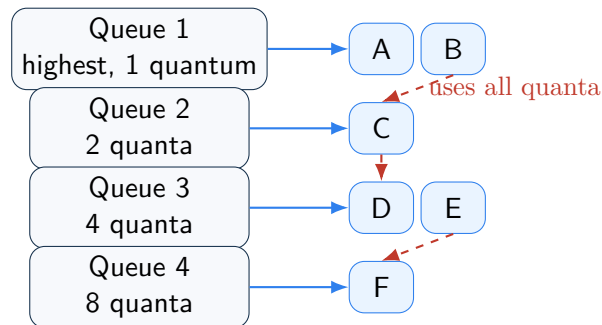
3. Small vs. Large Quantum

Quantum Choice	Benefit	Cost
Small quantum	Good response for interactive processes.	Many process switches and swaps.
Large quantum	Fewer switches for CPU-bound processes.	Poor response if everyone gets a large quantum.
Multiple queues	Combines both ideas.	More complex policy.

4. CTSS Solution

Priority Classes

CTSS used priority classes. High-priority classes ran sooner with small quanta. Lower-priority classes ran less often but received larger quanta.



Demotion Rule

Whenever a process used up all the quanta allocated to it, it was moved down one priority class.

5. Scheduling Rule

How the Scheduler Chooses

The scheduler always starts with the highest-priority nonempty queue. Processes inside the chosen queue are run round-robin style.

```
choose the highest-priority nonempty queue
run one process from that queue

if the process uses all quanta assigned to that queue then
    move it down one queue
else if the process blocks or becomes interactive then
    keep it high enough for good response
end if
```

6. Why CPU-Bound Processes Move Down

CPU-Bound Behavior

A CPU-bound process tends to use its entire allocation. Under the CTSS policy, that behavior makes it sink to lower-priority queues.

Effect

As a CPU-bound process moves lower, it runs less frequently, but when it does run, it gets a larger block of CPU time. This reduces swapping overhead.

7. 100-Quantum Example

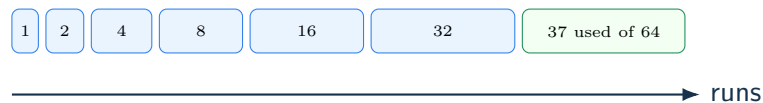
Continuous Computation

Suppose a process needs 100 quanta of CPU time without much interaction.

Run	Allocation	Total Used After Run	Result
1	1 quantum	1	Move down.
2	2 quanta	3	Move down.
3	4 quanta	7	Move down.
4	8 quanta	15	Move down.
5	16 quanta	31	Move down.
6	32 quanta	63	Move down.
7	64 quanta	100	Finishes after using only 37 of the 64.

Result

Only 7 loads or swaps are needed, including the initial load, instead of about 100 with pure round robin using one quantum at a time.



8. Why Interactive Processes Benefit

Interactive Behavior

Interactive processes often use a little CPU time and then block waiting for user input or I/O. They usually do not consume a long CPU allocation.

Good Response

Because interactive processes do not repeatedly use full quanta, they tend to remain near high-priority queues and get quick service.

9. Boosting Interactive Processes

CTSS Terminal Rule

To avoid punishing a process forever, CTSS moved a terminal's process to the highest-priority class whenever the user typed a carriage return, or Enter key.

Reason

The system assumed that pressing Enter meant the process was about to become interactive and should receive quick service.

10. Policy Can Be Gamed

Practical Problem

Users discovered that repeatedly pressing Enter could keep a CPU-bound process in a high-priority class, improving its response time unfairly.

Lesson

The policy worked in principle, but users could exploit it. Designing a scheduler that behaves well in practice is harder than designing one that sounds good in theory.

11. Multiple Queues vs. Earlier Algorithms

Algorithm	Main Rule	Main Limitation
Round robin	Every process gets the same quantum in turn.	Same quantum may be poor for mixed workloads.
Priority scheduling	Highest-priority runnable process runs first.	Low-priority processes can starve.
Multiple queues	Use priority queues with different quanta.	Policy is harder to tune and can be gamed.

12. Modern Name

Multilevel Feedback Queue

This idea is commonly known as a **multilevel feedback queue**: processes can move between priority queues based on their behavior.

Feedback

The scheduler uses feedback from past behavior. A process that uses all its CPU time looks CPU-bound and moves down. A process that behaves interactively can stay high or be boosted.

13. Big Picture

Main Conclusion

Multiple queues try to combine good interactive response with efficient handling of CPU-bound jobs. High-priority queues give short, quick turns. Lower-priority queues give larger quanta less often, reducing expensive switching.

Exam Shortcut

Remember: CTSS used multiple priority queues; lower queues got larger quanta; using all allocated quanta caused demotion; interactive activity could boost a process; bad boost rules can be exploited.

14. Quick Review

Key Points

- Multiple queues divide runnable processes into priority classes.
- CTSS used this idea because process switching required expensive swapping.
- High-priority queues used small quanta for quick response.
- Lower-priority queues used larger quanta to reduce switching overhead.
- A process that used all its allocated quanta was moved down one class.
- A CPU-bound process sinks to lower queues and runs less often but for longer.
- A 100-quantum CPU-bound job could finish with 7 swaps instead of about 100.
- Interactive processes tend to stay near higher-priority queues.
- CTSS boosted a terminal process when the user pressed Enter.
- Users could exploit that boost by pressing Enter repeatedly.
- The broader idea is now called a multilevel feedback queue.

Shortest Process Next

Predicting short interactive CPU bursts

Definition

Shortest Process Next (SPN) is a scheduling idea that runs the runnable process whose next CPU burst is estimated to be the shortest.

1. Connection to Shortest Job First

Basic Idea

Shortest Job First gives the minimum average response time for batch jobs when the job lengths are known. SPN tries to apply the same idea to interactive systems.

Key Difference

For batch systems, the scheduler may know or approximate the whole job length. For interactive systems, a process repeatedly performs short bursts of work between waits, so the scheduler predicts the length of the next burst.

2. Interactive Process Pattern

Typical Behavior

An interactive process usually follows this cycle:

wait for command → execute command → wait for command → execute command

Scheduling View

If each command execution is treated as a separate small job, then running the shortest command execution first can reduce average response time.



3. Main Problem

Unknown Future

The scheduler does not know which runnable process will have the shortest next CPU burst. It must estimate the next burst from previous behavior.

Practical Rule

Keep an estimated next running time for each process. When choosing among runnable processes, run the process with the smallest estimate.

4. Estimating the Next Burst

Weighted Average

Suppose a process has an old estimate E_{old} , and its newest measured CPU burst is T_{new} . A new estimate can be computed as:

$$E_{\text{new}} = aE_{\text{old}} + (1 - a)T_{\text{new}}$$

Symbol	Meaning
E_{old}	Previous estimate for the process's next CPU burst.
T_{new}	Actual CPU time used during the most recent burst.
a	Weight given to the old estimate, where $0 \leq a \leq 1$.
$1 - a$	Weight given to the newest measured burst.

5. Meaning of a

Memory of the Past

The value of a controls how quickly the estimate reacts to recent behavior.

Choice of a	Effect	Result
a close to 0	New measurements matter more.	Estimate changes quickly.
$a = \frac{1}{2}$	Old estimate and new measurement are balanced.	Simple and practical.
a close to 1	Old estimate matters more.	Estimate changes slowly.

6. Aging

Aging

Aging is the technique of predicting the next value in a sequence by combining the newest measured value with the previous estimate.

Why It Helps

Recent CPU bursts influence the estimate strongly, while older bursts gradually lose importance. This lets the scheduler adapt without being too unstable.

7. Successive Estimates When $a = \frac{1}{2}$

Repeated Updates

Let T_0 be the original estimate. Let T_1, T_2 , and T_3 be the next measured bursts. With $a = \frac{1}{2}$, the estimates become:

$$\begin{aligned}E_0 &= T_0 \\E_1 &= \frac{T_0}{2} + \frac{T_1}{2} \\E_2 &= \frac{T_0}{4} + \frac{T_1}{4} + \frac{T_2}{2} \\E_3 &= \frac{T_0}{8} + \frac{T_1}{8} + \frac{T_2}{4} + \frac{T_3}{2}\end{aligned}$$

Observation

After three new measured bursts, the original estimate T_0 has only weight $\frac{1}{8}$. Older information fades away over time.

$$\begin{aligned}E_1 & \quad \boxed{T_0 : 1/2} \quad \boxed{T_1 : 1/2} \\E_2 & \quad \boxed{T_0} \quad \boxed{T_1} \quad \boxed{T_2 : 1/2} \\E_3 & \quad \boxed{T_0} \quad \boxed{T_1} \quad \boxed{T_2} \quad \boxed{T_3 : 1/2}\end{aligned}$$

8. Why $a = \frac{1}{2}$ Is Easy

Simple Implementation

When $a = \frac{1}{2}$, the new estimate is just the average of the old estimate and the newest measurement:

$$E_{\text{new}} = \frac{E_{\text{old}} + T_{\text{new}}}{2}$$

Efficient Computation

On a binary computer, dividing by 2 can be done with a right shift by 1 bit. This makes the update cheap to implement.

```
for each process:
    store estimated_next_burst

when a process finishes a CPU burst:
    measured_burst = actual CPU time used
    estimated_next_burst = (estimated_next_burst + measured_burst) / 2

when the CPU becomes free:
    choose the runnable process with the smallest estimated_next_burst
```

9. Small Example

Updating an Estimate

Suppose the current estimate is 8 ms and the latest measured burst is 4 ms. With $a = \frac{1}{2}$:

$$E_{\text{new}} = \frac{8 + 4}{2} = 6 \text{ ms}$$

The process is now predicted to need 6 ms next time.

Process	Old Estimate	Latest Burst	New Estimate
A	8 ms	4 ms	6 ms
B	5 ms	9 ms	7 ms
C	12 ms	6 ms	9 ms

Scheduling Decision

After these updates, process A has the smallest estimated burst, so SPN would run A first among these runnable processes.

10. Strengths and Weaknesses

Aspect	Explanation
Main benefit	Can improve average response time by favoring short interactive bursts.
Main challenge	The actual next burst is unknown, so the scheduler depends on prediction.
Good behavior	Processes with consistently short bursts receive quick service.
Risk	Long-burst or CPU-bound processes may wait if short-burst processes keep arriving.

11. Big Picture

Main Conclusion

Shortest Process Next brings the Shortest Job First idea into interactive scheduling by estimating each process's next CPU burst. The estimate is updated from past behavior, commonly using aging.

Exam Shortcut

Remember: SPN chooses the process with the smallest predicted next CPU burst. The prediction is often updated using

$$E_{\text{new}} = aE_{\text{old}} + (1 - a)T_{\text{new}}.$$

With $a = \frac{1}{2}$, old information fades by half each update.

12. Quick Review

Key Points

- Shortest Job First is optimal for minimizing average response time in batch systems when job lengths are known.
- Shortest Process Next adapts this idea to interactive processes.
- Each command execution can be treated like a small job.
- The scheduler cannot know the next CPU burst exactly, so it estimates it.
- The estimate is based on previous behavior of the process.
- A common update rule is $E_{\text{new}} = aE_{\text{old}} + (1 - a)T_{\text{new}}$.
- A small a forgets old behavior quickly.
- A large a remembers old behavior longer.
- With $a = \frac{1}{2}$, updating is simple: add the old estimate and new measurement, then divide by 2.
- Aging makes older measurements gradually lose influence.

Guaranteed Scheduling

Fair CPU sharing through measurable promises

Definition

Guaranteed scheduling is a scheduling approach where the operating system promises each user or process a fair share of CPU time, then chooses processes based on how far behind or ahead they are relative to that promise.

1. Different Scheduling Philosophy

Main Idea

Instead of only using priorities, quanta, or predicted burst lengths, guaranteed scheduling starts with a performance promise and tries to keep it.

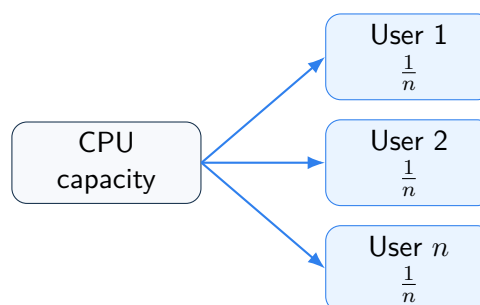
Fairness Promise

If n equal users are logged in, each user should receive about $\frac{1}{n}$ of the CPU power. Similarly, if one user has n equal runnable processes, each process should receive about $\frac{1}{n}$ of the CPU cycles.

2. What Is Being Guaranteed

Equal Share

The guarantee is not that every process finishes at the same time. The guarantee is that each equal competitor receives a fair fraction of CPU service over time.



Assumption

The simple rule assumes the users or processes are equal competitors. If the system supports different weights or priorities, the promised shares must be adjusted.

3. Information the System Tracks

Accounting Data

To keep the promise, the scheduler must know how much CPU time each process has already received since it was created.

Tracked Value	Purpose
Creation time	Tells how long the process has existed.
Elapsed time since creation	Used to compute the process's fair entitlement.
Actual CPU time consumed	Tells how much CPU service the process has already received.
Number of competitors n	Determines the fair share each process should receive.

4. CPU Entitlement

Entitled CPU Time

If a process has existed for T seconds while competing with n equal processes, then its fair CPU entitlement is:

$$\text{entitled CPU time} = \frac{T}{n}$$

Meaning

The entitlement says how much CPU time the process should have received by now if the CPU had been divided evenly among all equal competitors.

5. The Ratio

Service Ratio

The scheduler compares actual service with entitled service:

$$\text{ratio} = \frac{\text{actual CPU time consumed}}{\text{CPU time entitled}}$$

Ratio	Meaning	Scheduling Interpretation
0.5	The process has received half of what it should have received.	It is behind and should be favored.
1.0	The process has received exactly its fair share.	It is on target.
2.0	The process has received twice what it should have received.	It is ahead and can wait.

6. Scheduling Rule

Lowest Ratio Runs

Guaranteed scheduling runs the process with the lowest ratio, because that process is farthest behind its promised share.

When to Switch

The selected process runs until its ratio rises above the ratio of its closest competitor. Then the scheduler chooses the process that is now most behind.

```
for each runnable process:
    elapsed_time = current_time - creation_time
    entitled_cpu = elapsed_time / number_of_competitors
    ratio = actual_cpu_used / entitled_cpu
```

```
choose the runnable process with the lowest ratio
run it until its ratio is no longer the lowest
```

7. Snapshot Example

Four Equal Processes

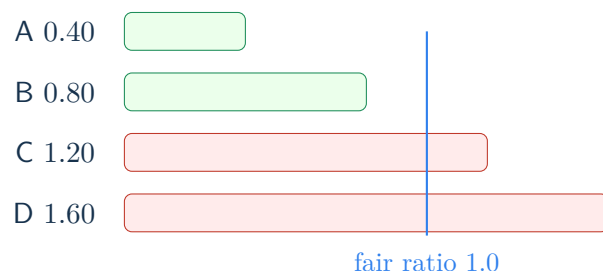
Suppose four equal processes have all existed for 100 seconds. Each process is entitled to:

$$\frac{100}{4} = 25 \text{ seconds of CPU time}$$

Process	Actual CPU Used	Entitled CPU	Ratio
A	10 s	25 s	$10/25 = 0.40$
B	20 s	25 s	$20/25 = 0.80$
C	30 s	25 s	$30/25 = 1.20$
D	40 s	25 s	$40/25 = 1.60$

Decision

Process A has the lowest ratio, so it has received the least CPU time relative to its fair share. The scheduler should run A next.



8. Running Until the Next Process Catches Up

Closest Competitor

In the snapshot, A has ratio 0.40 and B has the next lowest ratio 0.80. A should run until it catches up to B's service level.

Simplified Calculation

At the snapshot, A has used 10 seconds and B has used 20 seconds. If A runs for about 10 more seconds while B waits, A reaches 20 seconds of actual CPU use, so A is no longer clearly behind B.

Real Systems Update Continuously

In an actual implementation, elapsed time and entitlement keep changing while the process runs. The key idea is still the same: keep running the most under-served process until another process becomes more under-served.

9. User-Level vs. Process-Level Fairness

Multiple Users

On a multiuser system, the promise may be made per user. If n users are logged in, each user should receive about $\frac{1}{n}$ of the CPU power.

Single User

On a single-user system, the same idea can be applied to processes. If n equal processes are running, each process should receive about $\frac{1}{n}$ of the CPU cycles.

Reason

The same accounting method works in both cases. The scheduler compares actual CPU usage against promised CPU usage and favors the one that is behind.

10. Why This Is Fair

Fairness by Measurement

The algorithm does not merely hope that CPU time is shared fairly. It measures each process's actual CPU time and compares it with what the process should have received.

Self-Correcting Behavior

If a process has received too little CPU time, its ratio becomes low, so it is selected. If it has received too much CPU time, its ratio becomes high, so other processes are selected first.

11. Strengths and Weaknesses

Aspect	Explanation
Main benefit	Provides a clear fairness promise and a measurable way to enforce it.
Good for	Systems where users or processes should receive predictable shares of CPU time.
Required support	Accurate accounting of CPU time used by each process or user.
Main limitation	It focuses on fair CPU share, not necessarily shortest response time.
Practical issue	The number of competitors can change as users log in, processes exit, or new processes start.

12. Comparison with Other Scheduling Ideas

Algorithm	Main Rule	Main Goal
Round robin	Give each runnable process a turn.	Simple time sharing.
Priority scheduling	Run the highest-priority runnable process.	Respect importance or urgency.
Shortest Process Next	Run the process with the shortest estimated next burst.	Reduce average response time.
Guaranteed scheduling	Run the process with the lowest actual-to-entitled CPU ratio.	Keep a fair CPU-share promise.

13. Big Picture

Main Conclusion

Guaranteed scheduling is based on a simple promise: each equal competitor should receive its fair fraction of CPU time. The scheduler enforces this by tracking actual CPU usage, computing entitlement, and running the process with the lowest ratio.

Exam Shortcut

Remember the ratio:

$$\text{ratio} = \frac{\text{actual CPU time consumed}}{\text{CPU time entitled}}$$

Lowest ratio runs first. A ratio below 1 means the process is behind its fair share. A ratio above 1 means it has already received more than its fair share.

14. Quick Review

Key Points

- Guaranteed scheduling makes a measurable performance promise.
- With n equal users, each should receive about $\frac{1}{n}$ of the CPU power.
- With n equal processes, each should receive about $\frac{1}{n}$ of the CPU cycles.
- The system tracks how much CPU time each process has used since creation.
- A process's entitlement is its elapsed lifetime divided by n .
- The important ratio is actual CPU consumed divided by CPU time entitled.
- A ratio of 0.5 means the process has received only half of its fair share.
- A ratio of 2.0 means the process has received twice its fair share.
- The scheduler runs the process with the lowest ratio.
- The running process continues until its ratio rises above a close competitor's ratio.

Lottery Scheduling

Probabilistic CPU sharing with tickets

Definition

Lottery scheduling is a scheduling algorithm that gives processes lottery tickets for a resource. Whenever the resource must be allocated, the scheduler randomly chooses one ticket, and the process holding that ticket gets the resource.

1. Motivation

Problem with Strict Guarantees

Guaranteed scheduling gives predictable fairness, but it requires careful accounting and can be difficult to implement exactly.

Lottery Idea

Lottery scheduling gives similar long-run fairness with a simpler mechanism: assign tickets, draw a random winning ticket, and run the process that owns it.

Historical Note

Lottery scheduling was introduced by Waldspurger and Weihl in 1994 as a randomized way to manage resource allocation.

2. Basic CPU Scheduling Rule

Scheduling Lottery

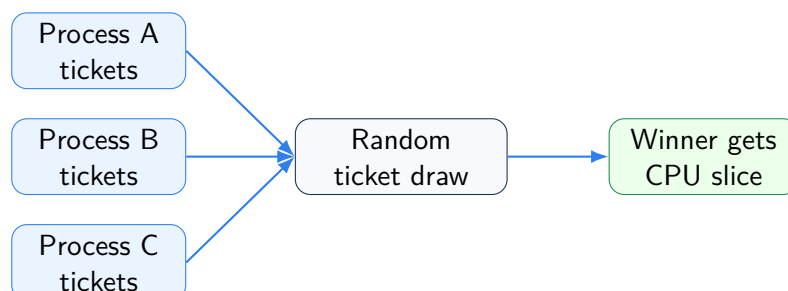
When the CPU becomes available, the operating system holds a lottery among all runnable processes. The process holding the winning ticket receives the next CPU time slice.

Time-Slice Example

The system might hold a lottery 50 times per second. Each winner then receives:

$$\frac{1}{50} \text{ second} = 20 \text{ ms}$$

of CPU time.



3. Tickets Represent Shares

Ticket Fraction

If a process owns a fraction f of all outstanding tickets, then it has probability f of winning each lottery. In the long run, it should receive about fraction f of the resource.

$$f = \frac{\text{tickets held by process}}{\text{total tickets in system}}$$

Clear Meaning

Unlike a priority number, a ticket count has a direct interpretation. If a process owns 20% of the tickets, it should receive about 20% of the CPU over time.

4. Simple Ticket Example

100 Total Tickets

Suppose there are 100 tickets total, and process A owns 20 of them. Then:

$$\frac{20}{100} = 0.20 = 20\%$$

Process A has a 20% chance of winning each lottery and should receive about 20% of the CPU in the long run.

Process	Tickets	Chance per Lottery	Expected CPU Share
A	20	$20/100 = 20\%$	About 20%
B	30	$30/100 = 30\%$	About 30%
C	50	$50/100 = 50\%$	About 50%



100 total tickets

5. Unequal Importance

More Tickets, More Chance

All processes can participate in the lottery, but important processes can be given more tickets. More tickets mean a higher probability of winning CPU time.

Main Rule

If one process should receive twice as much CPU as another, give it twice as many tickets.

Short Run vs. Long Run

Lottery scheduling is probabilistic. A process with 20% of the tickets does not win exactly every fifth lottery. It receives about 20% of the CPU over a longer period.

6. Algorithm

```
total_tickets = sum tickets of all runnable processes
winning_ticket = random number from 1 to total_tickets

current_sum = 0
for each runnable process:
    current_sum = current_sum + process.tickets
    if current_sum >= winning_ticket:
        run this process for one time slice
        break
```

Implementation Point

The scheduler only needs ticket counts and a random draw. It does not need to compute exact future CPU usage for every process.

7. Responsiveness

New Process

If a new process arrives and receives tickets, it can win immediately at the next lottery.

Why It Is Responsive

The new process does not need to wait for a long accounting adjustment. Its chance of running appears as soon as its tickets are included in the lottery.

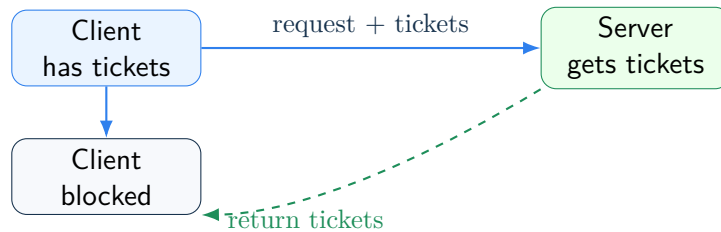
8. Ticket Transfer

Ticket Transfer

Cooperating processes can temporarily transfer tickets to each other. This lets CPU share move to the process that is currently doing useful work.

Client and Server

When a client sends a request to a server and then blocks, the client can give its tickets to the server. The server is then more likely to run and finish the request quickly. When the server finishes, it returns the tickets to the client.



Server Without Clients

If no clients need service, a server may need no tickets at all. Tickets can be supplied by clients only while the server is doing work for them.

9. Video Server Example

Different Frame Rates

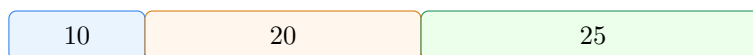
Suppose three video-stream processes need frames at 10, 20, and 25 frames per second. Give them 10, 20, and 25 tickets.

10 : 20 : 25

Stream Process	Required Frame Rate	Tickets	CPU Share
P1	10 frames/sec	10	$10/55 \approx 18.2\%$
P2	20 frames/sec	20	$20/55 \approx 36.4\%$
P3	25 frames/sec	25	$25/55 \approx 45.5\%$

Result

The CPU is divided automatically in approximately the required ratio 10 : 20 : 25. Processes needing higher frame rates receive more tickets and therefore win more often.



ticket ratio matches frame-rate ratio

10. Comparison with Priority Scheduling

Approach	Meaning of Control Value	Result
Priority scheduling	A priority number says which process is more important, but the exact CPU share may be unclear.	Good for ordering, less direct for proportional sharing.
Lottery scheduling	Ticket fraction directly represents expected resource fraction.	Clear proportional sharing in the long run.

Clear Rule

A process holding fraction f of the tickets gets about fraction f of the resource.

11. Strengths and Weaknesses

Aspect	Explanation
Main benefit	Gives predictable long-run proportional sharing with a simple randomized algorithm.
Responsive behavior	New processes can win at the next lottery after receiving tickets.
Flexible control	More important processes can receive more tickets.
Cooperation support	Processes can transfer tickets to speed up related work.
Main limitation	Short-term results are random, so exact fairness is only approximate over time.

12. Big Picture

Main Conclusion

Lottery scheduling allocates CPU time by randomly choosing among tickets. The more tickets a process holds, the more often it should win. This gives simple, responsive, proportional sharing without the detailed accounting required by strict guaranteed scheduling.

Exam Shortcut

Remember: tickets are probabilities. If a process owns 20 out of 100 tickets, it has a 20% chance of winning each lottery and should receive about 20% of the CPU in the long run.

13. Quick Review

Key Points

- Lottery scheduling gives processes tickets for resources such as CPU time.
- Each scheduling decision is made by drawing a random winning ticket.
- The process holding the winning ticket receives the next CPU time slice.
- If lotteries happen 50 times per second, each winner receives about 20 ms.
- A process with fraction f of the tickets receives about fraction f of the resource in the long run.
- More important processes can be given more tickets.
- Lottery scheduling is responsive because new ticket holders can win immediately.
- Cooperating processes can transfer tickets.
- Clients can give tickets to servers while waiting for replies.
- A video server can allocate tickets in proportion to frame-rate needs, such as 10 : 20 : 25.

Fair-Share Scheduling

Scheduling by user share, not only process count

Definition

Fair-share scheduling is a scheduling approach that gives each user or group a promised fraction of the CPU, then schedules that user's processes so the promised shares are respected.

1. Problem with Process-Only Scheduling

Usual Assumption

Many scheduling algorithms treat every runnable process as an independent competitor, without considering which user owns it.

Unfair Result

If user 1 starts nine processes and user 2 starts one process, ordinary round robin gives user 1 nine turns for every one turn given to user 2.

User 1 gets 90%, User 2 gets 10%.

User	Number of Processes	CPU Share with Process-Level Round Robin
User 1	9	$9/10 = 90\%$
User 2	1	$1/10 = 10\%$

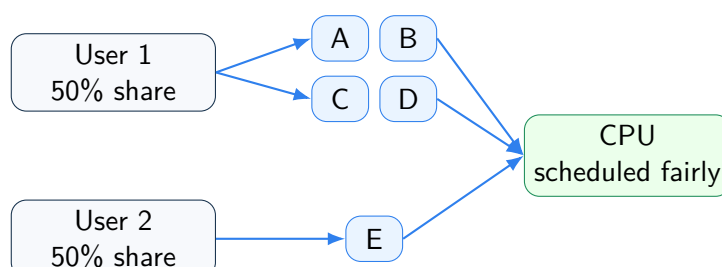
2. Fair-Share Idea

User-Level Fairness

Fair-share scheduling first decides how much CPU each user should receive. The scheduler then chooses among processes in a way that keeps each user close to that promised share.

Main Difference

The unit of fairness is no longer just the process. The scheduler also considers the owner of each process.



3. Promised CPU Fractions

CPU Share

Each user is allocated a fraction of the CPU. If two users are each promised 50%, each should receive half the CPU time, regardless of how many processes they own.

Key Rule

The number of processes owned by a user should not automatically increase that user's total CPU share.

4. Example Setup

Two Users

Assume two users are both promised 50% of the CPU.

- User 1 owns four processes: A, B, C, and D.
- User 2 owns one process: E.

User	Processes	Promised CPU Share
User 1	A, B, C, D	50% total across all four processes
User 2	E	50% total for process E

5. Equal User Shares

Valid Schedule

A possible sequence that gives each user 50% of the CPU is:

A E B E C E D E A E B E C E D E ...

Why It Works

Every other CPU slot goes to E, so user 2 receives half of the CPU. The remaining half is shared among A, B, C, and D, so user 1 also receives half of the CPU.

A E B E C E D E A E B E

User 1 and User 2 alternate CPU ownership

User	Slots in One Cycle	CPU Share
User 1	A, B, C, D	$4/8 = 50\%$
User 2	E, E, E, E	$4/8 = 50\%$

6. Unequal User Shares

User 1 Gets Twice as Much

If user 1 is entitled to twice as much CPU time as user 2, a possible sequence is:

A B E C D E A B E C D E ...

Ratio

In each six-slot cycle, four slots go to user 1 and two slots go to user 2:

User 1 : User 2 = 4 : 2 = 2 : 1.

A B E C D E A B E C D E

User 1 receives two CPU slots for each one of User 2

User	Slots in One Cycle	CPU Share
User 1	A, B, C, D	$4/6 \approx 66.7\%$
User 2	E, E	$2/6 \approx 33.3\%$

7. How the Scheduler Thinks

Two-Level Decision

The scheduler can first decide which user should receive the next CPU slot, then choose one of that user's runnable processes.

choose the user or group most behind its promised CPU share

within that user or group:

choose one runnable process

run it for one time slice

update CPU usage for both the process and its owner

Process Selection Inside a User

Within a user's own share, the system may use round robin, priority scheduling, or another local rule. Fair-share scheduling mainly controls how much total CPU each owner receives.

8. Why This Prevents Abuse

Without Fair Share

A user can get more CPU time simply by starting more processes.

With Fair Share

Starting more processes divides that user's own allocation among more processes. It does not increase the user's total CPU allocation.

9. Different Notions of Fairness

Flexible Policy

Many valid schedules can satisfy the same user-level CPU shares. The exact sequence can vary depending on how the system defines fairness.

Fairness Choice	Meaning
Equal users	Each user receives the same CPU fraction.
Weighted users	Some users receive larger CPU fractions than others.
Group fairness	CPU share is assigned to groups, projects, or departments.
Per-user local policy	Each user's own processes are scheduled within that user's allocation.

10. Comparison with Process-Level Round Robin

Approach	What Is Treated Fairly	Weakness
Process-level round robin	Individual processes.	Users with more processes get more total CPU.
Fair-share scheduling	Users, groups, or owners.	Requires accounting by owner and a share policy.

11. Big Picture

Main Conclusion

Fair-share scheduling prevents CPU share from depending only on how many processes a user starts. It enforces fairness at the owner level by giving each user or group a promised CPU fraction.

Exam Shortcut

Remember: round robin is fair to processes; fair-share scheduling is fair to users or groups. If user 1 owns four processes and user 2 owns one process, fair-share scheduling can still give both users 50% of the CPU.

12. Quick Review

Key Points

- Process-only scheduling ignores who owns each process.
- With ordinary round robin, a user with more processes can receive more total CPU time.
- Fair-share scheduling considers the owner of each process.
- Each user or group is assigned a promised CPU fraction.
- The scheduler chooses processes so the owner-level shares are enforced.
- If two users are promised 50% each, both receive half the CPU even if one owns more processes.
- A valid equal-share sequence is A E B E C E D E
- If user 1 deserves twice as much CPU as user 2, a valid sequence is A B E C D E
- The exact sequence can vary; the important constraint is the CPU-share promise.
- Fair-share scheduling prevents users from gaining CPU share merely by creating more processes.

Scheduling in Real-Time Systems

Meeting deadlines when time is part of correctness

Definition

A **real-time system** is a system in which time is essential. The computer must respond to external events correctly and within a fixed amount of time.

1. Why Time Matters

Main Idea

In ordinary systems, a correct answer may still be useful even if it arrives a little late. In real-time systems, a late answer may be just as bad as a wrong answer.

Deadline Failure

The scheduler must make sure important computations finish before their deadlines. If the computation takes too long, the system may fail even if the final result is logically correct.

2. Examples of Real-Time Systems

System	Timing Requirement
Compact disc player	Bits must be converted into music fast enough to avoid distorted sound.
Hospital patient monitor	Sensor readings must be processed quickly enough to detect dangerous conditions.
Aircraft autopilot	Control decisions must be made in time to keep the aircraft stable.
Factory robot control	Motor and sensor actions must happen at precise times.

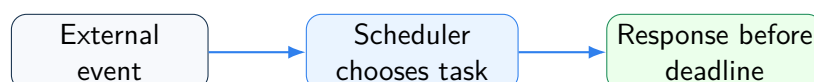
Key Lesson

For real-time systems, correctness has two parts: producing the right answer and producing it before the deadline.

3. External Events and Responses

Stimuli

Real-time systems usually react to stimuli generated by physical devices outside the computer.



4. Hard vs. Soft Real Time

Type	Meaning	Consequence of Missing a Deadline
Hard real time	Deadlines are absolute and must be met.	Failure may be unacceptable or dangerous.
Soft real time	Deadlines are important but occasional misses can be tolerated.	Performance degrades, but the system may continue.

Hard Real-Time Rule

In a hard real-time system, missing a deadline is considered a system failure.

Soft Real-Time Rule

In a soft real-time system, missing an occasional deadline is undesirable, but it may be acceptable.

5. Predictable Processes

Known Behavior

Real-time behavior is usually achieved by dividing the program into processes or tasks whose behavior is predictable and known in advance.

Short Tasks

These tasks are often short-lived and can run to completion in much less than a second. This makes it easier for the scheduler to reason about deadlines.

6. Periodic and Aperiodic Events

Event Types

Real-time events can be classified by when they occur.

Event Type	Meaning	Example
Periodic	Occurs at regular intervals.	A sensor reading every 100 ms.
Aperiodic	Occurs unpredictably.	An emergency button press or sudden fault signal.

Scheduler's Job

When an event is detected, the scheduler must run the needed process in time for the deadline to be met.

7. Schedulability

Schedulable System

A real-time system is **schedulable** if the required tasks can actually be scheduled so all required deadlines are met.

CPU Load Test

For m periodic events, if event i occurs every P_i seconds and needs C_i seconds of CPU time each time, then the total utilization is:

$$\sum_{i=1}^m \frac{C_i}{P_i}$$

Schedulability Condition

The system can be handled only if:

$$\sum_{i=1}^m \frac{C_i}{P_i} \leq 1$$

8. Meaning of the Formula

Symbol	Meaning
m	Number of periodic event streams.
P_i	Period of event i , or how often it occurs.
C_i	CPU time needed to handle one occurrence of event i .
$\frac{C_i}{P_i}$	Fraction of CPU capacity consumed by event i .
$\sum \frac{C_i}{P_i}$	Total CPU utilization required by all periodic events.

Impossible Load

If the sum is greater than 1, the tasks collectively demand more CPU time than the CPU can provide. Such a system cannot be scheduled under this simple load test.

9. Worked Example

Three Periodic Events

A soft real-time system has three periodic events:

- Event 1: period 100 ms, CPU time 50 ms.
- Event 2: period 200 ms, CPU time 30 ms.
- Event 3: period 500 ms, CPU time 100 ms.

$$\frac{50}{100} + \frac{30}{200} + \frac{100}{500} = 0.50 + 0.15 + 0.20 = 0.85$$

Result

Since $0.85 < 1$, the system is schedulable by this utilization test. The events require 85% of the CPU, leaving about 15% spare capacity.



CPU utilization: 0.85 out of 1.00

10. Adding a Fourth Event

Remaining Capacity

The first three events use 0.85 of the CPU, so the remaining capacity is:

$$1.00 - 0.85 = 0.15$$

Fourth Event Limit

If a fourth event has period 1 second, its CPU time C_4 must satisfy:

$$\frac{C_4}{1} \leq 0.15$$

So C_4 can be at most 0.15 seconds, or 150 ms.

Result

The fourth event can be added only if it needs no more than 150 ms of CPU time per event.

11. Important Assumption

Ignored Overhead

The utilization calculation assumes context-switching overhead and other scheduler overheads are small enough to ignore.

Practical Meaning

Real systems may need extra safety margin because interrupts, context switches, cache effects, and OS overhead also consume time.

12. Static vs. Dynamic Scheduling

Scheduling Type	When Decisions Are Made	Requirement
Static scheduling	Before the system starts running.	Requires complete advance knowledge of tasks and deadlines.
Dynamic scheduling	At run time, after execution has started.	Can react to changing conditions and less predictable workloads.

Static Scheduling

Static scheduling works only when the system has perfect information in advance about the work to be done and the deadlines to be met.

Dynamic Scheduling

Dynamic scheduling makes decisions while the system is running, so it is more flexible when events or workloads are not fully known in advance.

13. Strengths and Challenges

Aspect	Explanation
Main goal	Meet timing deadlines, not just maximize throughput or fairness.
Key requirement	Tasks must have predictable timing behavior.
Schedulability test	Checks whether periodic CPU demand is at most total CPU capacity.
Hard real-time risk	Missing one deadline may mean system failure.
Soft real-time risk	Occasional misses may be tolerable but reduce quality.

14. Big Picture

Main Conclusion

Real-time scheduling is about completing work before deadlines. A system is schedulable only if the CPU can provide enough time for all required event handling. For periodic events, the basic utilization test is $\sum C_i/P_i \leq 1$.

Exam Shortcut

Remember: hard real time means deadlines must not be missed. Soft real time means occasional misses are tolerable. For periodic events, compute:

$$\sum_{i=1}^m \frac{C_i}{P_i}$$

If the result is at most 1, the workload passes the basic schedulability test.

15. Quick Review

Key Points

- In a real-time system, time is part of correctness.
- A correct result delivered too late may be useless.
- Examples include CD players, patient monitors, aircraft autopilots, and robot controllers.
- Hard real-time systems have deadlines that must be met.
- Soft real-time systems can tolerate occasional deadline misses.
- Real-time programs are often divided into predictable short tasks.
- Periodic events occur at regular intervals.
- Aperiodic events occur unpredictably.
- For periodic events, utilization is $\sum C_i/P_i$.
- The system is schedulable by the basic test if $\sum C_i/P_i \leq 1$.
- Static scheduling decides before execution begins.
- Dynamic scheduling decides at run time.

Policy Versus Mechanism

Separating who decides from who enforces

Definition

Policy versus mechanism is the design principle that the operating system should provide general scheduling mechanisms, while policy choices can be supplied by the entity that has better knowledge of what should happen.

1. Old Assumption

Independent Competitors

Many scheduling discussions assume that all processes belong to different users and are competing independently for the CPU.

Often True

This assumption works for many time-sharing systems, where unrelated users and unrelated processes compete for processor time.

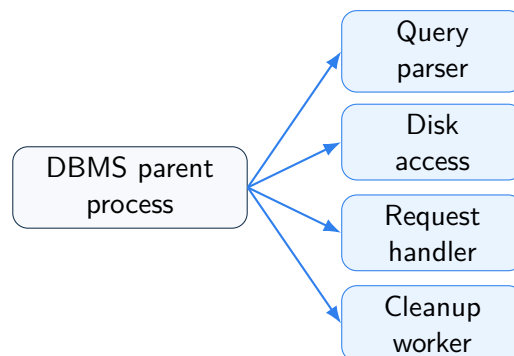
2. When the Assumption Fails

Related Processes

Sometimes one process creates and controls many child processes. These children are not independent competitors in the usual sense; they are parts of a larger application.

Database System

A database-management-system process may have many child processes. One child may parse queries, another may access the disk, another may handle client requests, and another may perform cleanup work.



3. The Knowledge Problem

Parent Knows More

The parent process may know which child is most important, which one is time critical, and which one can safely wait.

Kernel Knows Less

A general-purpose kernel scheduler often sees only runnable processes, priorities, CPU usage, and blocking behavior. It may not understand the application's internal priorities.

Result

If the scheduler accepts no useful input from user processes, it may choose a process that is technically runnable but not the best choice for the application.

4. Policy and Mechanism

Mechanism

A **mechanism** is the low-level facility that performs an action. In scheduling, the mechanism is the code that chooses and dispatches a process.

Policy

A **policy** is the rule that decides what should be done. In scheduling, the policy decides which process should be favored and why.

Concept	Question Answered	Example
Mechanism	How can scheduling be carried out?	Kernel priority scheduler.
Policy	Which process should receive better treatment?	Parent gives urgent child a higher priority.

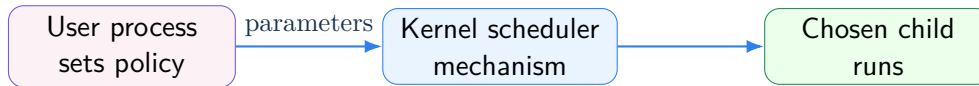
5. Separation Principle

Main Principle

Separate the scheduling mechanism from the scheduling policy. The kernel provides a parameterized scheduling mechanism, and user processes can supply or adjust the parameters.

Why It Helps

The kernel keeps control over the actual scheduling operation, while the application can express information that the kernel would not otherwise know.



6. Parameterized Scheduling

Parameterized Algorithm

A scheduling algorithm is **parameterized** when its behavior depends on adjustable values, such as priorities, weights, CPU shares, or deadlines.

Policy Input

The user process does not run the scheduler directly. It only supplies parameters that influence how the scheduler behaves.

7. Database Example

Kernel Mechanism, User Policy

Suppose the kernel uses priority scheduling and provides a system call that lets a parent process set or change the priorities of its children.

Part	Role
Kernel scheduler	Mechanism: dispatches runnable processes according to priority.
System call	Interface: lets a parent set child priorities.
DBMS parent process	Policy: decides which child is most important right now.
Child processes	Work units: query parsing, disk access, request handling, cleanup.

```

parent process observes its children

if a child is time critical:
    set that child's priority high
else if a child can wait:
    set that child's priority low

kernel scheduler:
    run runnable processes according to priority
  
```

8. What the Parent Controls

Control Without Direct Scheduling

The parent controls how its children are likely to be scheduled by changing their scheduling parameters. It does not directly take over the CPU scheduler.

Kernel Still Enforces

The kernel still performs the actual scheduling decision, protects the system, and enforces rules about what a user process is allowed to change.

9. Why Not Put All Policy in the Kernel

Too Much Application Knowledge

The kernel cannot reasonably know every application's internal meaning of importance, urgency, and dependency.

Better Location

Application-specific policy often belongs near the application, because the application understands its own children and their roles.

10. Security and Limits

Need for Constraints

Letting user processes influence scheduling must be controlled. Otherwise a process could unfairly raise all of its children above other users' processes.

Safe Design

The kernel can restrict policy changes to a process's own children, limit priority ranges, enforce permissions, and still apply global fairness rules.

11. Advantages

Advantage	Explanation
Flexibility	Different applications can express different scheduling needs.
Better decisions	The application can provide information the kernel does not know.
Cleaner design	The kernel provides general mechanisms instead of hard-coding every policy.
Controlled power	The kernel can allow useful influence without giving up protection.

12. Comparison

Design	How Scheduling Is Decided	Main Weakness
Fixed kernel policy	Kernel makes all policy decisions internally.	May ignore application-specific knowledge.
Policy-mechanism separation	Kernel provides mechanism; user process supplies parameters.	Requires careful interfaces and protection.

13. Big Picture

Main Conclusion

Policy-mechanism separation lets the kernel keep control of scheduling while allowing user processes to express application-specific scheduling policy. The mechanism stays in the kernel; the policy can be influenced by the process that understands the work.

Exam Shortcut

Remember: mechanism means how scheduling is performed; policy means what scheduling choice is desired. In the database example, the kernel priority scheduler is the mechanism, and the parent process setting child priorities is the policy.

14. Quick Review

Key Points

- Many schedulers assume processes are independent CPU competitors.
- Sometimes one parent process controls many related child processes.
- A DBMS may have children for query parsing, disk access, request handling, and other work.
- The parent may know which child is most urgent or important.
- The kernel scheduler may not know the application's internal priorities.
- Policy-mechanism separation solves this by keeping the scheduler mechanism general.
- The kernel provides the scheduling mechanism.
- The user process can supply policy by setting scheduling parameters.
- Example: a parent uses a system call to change child priorities.
- The parent influences scheduling, but the kernel still performs and controls scheduling.
- The kernel must enforce permissions and limits to prevent abuse.

Thread Scheduling

Scheduling when processes contain multiple threads

Definition

Thread scheduling is the problem of deciding which thread should run when processes may contain multiple threads. The answer depends strongly on whether threads are managed in user space, in the kernel, or both.

1. Two Levels of Parallelism

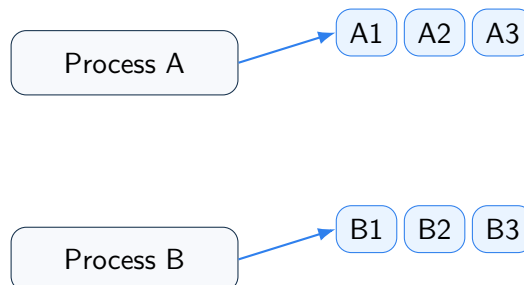
Processes and Threads

When several processes each have multiple threads, the system has two levels of execution units:

- Processes compete for CPU time.
- Threads inside those processes also compete to run.

Main Question

Who chooses the next thread: the kernel scheduler or a user-level run-time system inside the process?



2. User-Level Threads

User-Level Threads

With **user-level threads**, the kernel does not know that threads exist. It schedules processes as usual, and a run-time system inside the selected process chooses which user-level thread runs.

Two Scheduling Decisions

For user-level threads:

1. The kernel chooses a process.
2. The process's run-time system chooses a thread inside that process.

kernel scheduler:

- choose a process, such as process A
- give A one process quantum

run-time system inside A:

- choose one user-level thread, such as A1
- run A1 until it yields, blocks in user space, or A's quantum ends

3. No Kernel Clock for User Threads

No Thread-Level Preemption

Because the kernel is not aware of user-level threads, it cannot use clock interrupts to preempt one user-level thread and switch to another thread in the same process.

Consequence

If thread A1 keeps running, it can consume all of process A's CPU quantum. When A later receives another quantum, A1 may continue again.

Limited Damage

This behavior does not steal CPU time from other processes. The kernel still gives other processes whatever share the process scheduler assigns to them.

4. User-Level Thread Example

50-ms Process Quantum

Suppose process A has a 50-ms quantum and its threads each do about 5 ms of CPU work before yielding. The run-time system may schedule:

A1 A2 A3 A1 A2 A3 A1 A2 A3 A1

before the kernel switches to process B.

A1 A2 A3 A1 A2 A3 A1 A2 A3 A1 B

Kernel sees A's process quantum, not A's individual threads

Not Possible Here

With user-level threads and these parameters, a sequence like A1 B1 A2 B2 A3 B3 is not possible inside A's quantum, because the kernel schedules process A first, not individual A and B threads.

5. User-Level Thread Scheduler

Scheduling Policy

The run-time system inside a process can use familiar scheduling algorithms for its own threads, such as round robin or priority scheduling.

Cooperative Assumption

The main constraint is that a user-level thread must cooperate by yielding or blocking. There is no kernel timer interrupt forcing it to give way to another user-level thread in the same process.

6. Kernel-Level Threads

Kernel-Level Threads

With **kernel-level threads**, the kernel knows about each thread and schedules individual threads directly.

Direct Thread Scheduling

The kernel can choose A1, then B1, then A2, even though these threads belong to different processes.

kernel scheduler:

```
choose one runnable kernel-level thread
run it for a thread quantum
preempt it if it exceeds the quantum
```

7. Kernel-Level Thread Example

Interleaving Threads

With a 50-ms quantum and threads that block after 5 ms, the kernel might run:

A1 B1 A2 B2 A3 B3

over a short interval.

A1 B1 A2 B2 A3 B3

Kernel schedules threads directly

Kernel Choice

The kernel does not have to schedule by process first, although it may consider process ownership when that helps performance or fairness.

8. Side-by-Side Comparison

Feature	User-Level Threads	Kernel-Level Threads
Who knows about threads?	User-level run-time system.	Kernel.
Kernel schedules	Processes.	Threads.
Thread preemption	No kernel preemption among user threads.	Kernel can forcibly preempt threads.
Possible interleaving	Mostly threads from the selected process during its quantum.	Threads from different processes can interleave directly.
Thread switch cost	Very low.	Higher, because the kernel is involved.
Blocking I/O effect	One blocking thread may block the whole process.	One blocking thread does not have to block the whole process.

9. Performance Difference

User-Level Speed

Switching among user-level threads can take only a small number of machine instructions because it happens inside the process without entering the kernel.

Kernel-Level Cost

Switching kernel-level threads is more expensive because it involves the kernel. Switching to a thread in another process can require changing the memory map and may disturb the cache.

Practical Tradeoff

User-level threads are fast to switch, but the kernel cannot schedule them individually. Kernel-level threads cost more to switch, but they give the kernel more control and avoid blocking the whole process on one thread's I/O.

10. Same-Process Preference

Cache and Memory Map

The kernel may know that switching from a thread in process A to another thread in process A is cheaper than switching to a thread in process B.

Scheduling Hint

If two runnable threads are otherwise equally important, the kernel may prefer a thread from the same process as the thread that just blocked. This can avoid unnecessary memory-map and cache disruption.

11. Application-Specific Scheduling

User-Level Advantage

User-level threads can use an application-specific thread scheduler. The run-time system may understand what each thread does better than the kernel can.

Web Server

Suppose a Web server has a dispatcher thread and worker threads. If a worker blocks on disk I/O and the dispatcher is ready, the user-level run-time system may choose the dispatcher next so it can start another worker.

Why This Helps

The application-specific scheduler can maximize useful parallelism because it understands the roles of the threads.

12. Kernel Limitation

Less Application Knowledge

With kernel-level threads, the kernel usually does not know which thread is a dispatcher, which is a worker, or which action would maximize application-level progress.

Possible Workaround

Threads can be assigned different priorities, but priority numbers may still not capture the full application-specific policy.

13. Choosing Between Them

Need	Better Fit
Very fast thread switching	User-level threads.
Application-specific scheduling policy	User-level threads.
Kernel preemption of individual threads	Kernel-level threads.
One thread blocking without stopping the whole process	Kernel-level threads.
Interleaving threads from different processes directly	Kernel-level threads.

14. Big Picture

Main Conclusion

Thread scheduling depends on where threads are implemented. With user-level threads, the kernel schedules processes and the run-time system schedules threads inside each process. With kernel-level threads, the kernel schedules individual threads directly.

Exam Shortcut

Remember: user-level thread switching is fast and application-aware, but the kernel cannot preempt or schedule those threads individually. Kernel-level threads are slower to switch, but the kernel can preempt them, interleave threads from different processes, and avoid blocking an entire process when one thread waits for I/O.

15. Quick Review

Key Points

- Multithreaded systems have both process-level and thread-level parallelism.
- With user-level threads, the kernel schedules processes, not threads.
- A user-level run-time system chooses which thread runs inside the selected process.
- User-level threads are not preempted individually by kernel clock interrupts.
- A badly behaved user-level thread can consume its process's whole quantum.
- Other processes are still protected by the kernel's process scheduler.
- With kernel-level threads, the kernel schedules individual threads.
- Kernel-level scheduling can produce sequences such as A1 B1 A2 B2 A3 B3.
- User-level thread switching is much faster.
- Kernel-level threads avoid blocking the whole process when one thread blocks on I/O.
- The kernel may prefer switching to another thread in the same process to reduce overhead.
- User-level thread schedulers can use application-specific knowledge.

The Dining Philosophers Problem

A classic synchronization problem about deadlock and starvation

Definition

The **dining philosophers problem** is a classic synchronization problem introduced by Dijkstra in 1965. It asks how several competing processes can share limited resources without deadlock or starvation.

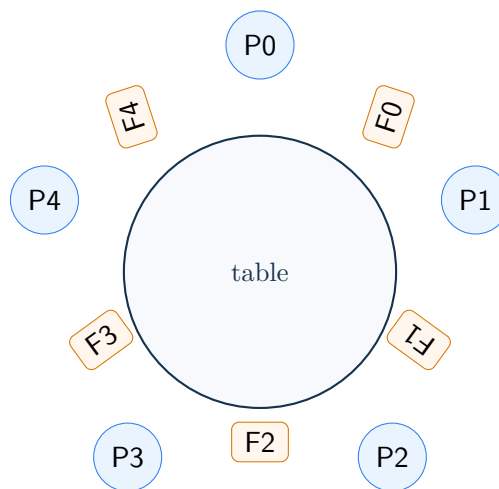
1. Problem Setup

Dining Table

Five philosophers sit around a circular table. Each philosopher has a plate of spaghetti, and there is one fork between each pair of neighboring plates.

Two-Fork Requirement

A philosopher needs two forks to eat: the fork on the left and the fork on the right. A philosopher can eat only after acquiring both forks.



2. Philosopher Behavior

Life Cycle

Each philosopher repeatedly alternates between thinking and eating.

```
while true:
    think
    acquire left fork
    acquire right fork
    eat
    release left fork
    release right fork
```

Main Question

Can we write a program for each philosopher so every philosopher can eat when possible and the system never gets stuck?

3. Why the Problem Matters

Synchronization Model

The philosophers represent processes. The forks represent shared resources. Eating represents entering a critical activity that requires multiple resources.

Core Difficulty

The difficult part is not taking one fork. The difficult part is coordinating two shared resources without creating deadlock or starvation.

4. The Obvious Nonsolution

Take Left, Then Take Right

The most direct solution is for each philosopher to take the left fork, then the right fork, then eat.

```
#define N 5

void philosopher(int i)
{
    while (TRUE) {
        think();
        take_fork(i);
        take_fork((i + 1) % N);
        eat();
        put_fork(i);
        put_fork((i + 1) % N);
    }
}
```

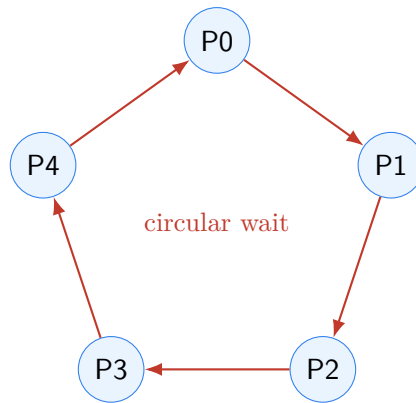
Why It Fails

If all five philosophers take their left forks at the same time, every philosopher holds one fork and waits forever for the right fork. This is deadlock.

5. Deadlock

Deadlock

Deadlock occurs when every process in a set is waiting for an event that only another process in the same set can cause.



Dining Philosophers Deadlock

Each philosopher holds the left fork and waits for the right fork. Since nobody releases a fork, nobody can continue.

6. Try-and-Retry Attempt

Modified Idea

After taking the left fork, a philosopher checks whether the right fork is available. If not, the philosopher puts down the left fork, waits, and tries again.

Still Not Enough

If all philosophers act in lockstep, they may all pick up their left forks, all fail to get their right forks, all put down their left forks, and all repeat forever.

7. Starvation

Starvation

Starvation occurs when processes keep running or retrying, but some process never makes progress.

Difference from Deadlock

In deadlock, the processes are stuck waiting. In starvation, activity may continue, but useful progress for some process never occurs.

Dining Philosophers Starvation

The philosophers may repeatedly release and retry without anyone actually eating.

8. Random Waiting

Random Backoff

One practical improvement is to wait for a random amount of time before trying again. This reduces the chance that all philosophers remain synchronized forever.

Practical Use

Random retry works well in many systems. For example, network protocols can use random waiting after collisions.

Not a Proof

Random waiting makes failure unlikely, but it does not provide a deterministic guarantee. Some safety-critical systems require a solution that always works.

9. Single Mutex Solution

Simple Safe Idea

Protect the fork-acquisition and fork-release code with one binary semaphore, or mutex. Only one philosopher can try to acquire forks at a time.

```
while true:
    think
    down(mutex)
    take left fork
    take right fork
    eat
    put left fork
    put right fork
    up(mutex)
```

Correctness

This avoids deadlock because only one philosopher can hold forks while entering the eating section.

Performance Bug

Only one philosopher can eat at a time. With five forks, two non-neighboring philosophers should be able to eat simultaneously.

10. Better Semaphore Solution

State-Based Solution

A better solution keeps a state for each philosopher: thinking, hungry, or eating. A philosopher may eat only if neither neighbor is eating.

State	Meaning
THINKING	The philosopher is not trying to eat.
HUNGRY	The philosopher wants to eat and is trying to acquire forks.
EATING	The philosopher has both forks and is eating.

Neighbor Rule

Philosopher i can enter EATING only if philosopher i 's left neighbor and right neighbor are not eating.

11. Neighbor Macros

Circular Table

With N philosophers numbered 0 to $N - 1$, the neighbors of philosopher i are:

$$\text{LEFT} = (i + N - 1) \bmod N$$

$$\text{RIGHT} = (i + 1) \bmod N$$

Example

If $N = 5$ and $i = 2$, then LEFT is 1 and RIGHT is 3.

12. Data Structures

Object	Purpose
<i>state</i> [N]	Records whether each philosopher is THINKING, HUNGRY, or EATING.
<i>mutex</i>	Protects updates to the shared state array.
<i>s</i> [N]	One semaphore per philosopher, used to block hungry philosophers until forks are available.

Key Design

The mutex protects the shared bookkeeping. The per-philosopher semaphores allow individual hungry philosophers to sleep until they are allowed to eat.

13. Core Algorithm

```
#define N 5
#define LEFT (i + N - 1) % N
#define RIGHT (i + 1) % N

#define THINKING 0
#define HUNGRY 1
```

```

#define EATING 2

int state[N];
semaphore mutex = 1;
semaphore s[N];

void philosopher(int i)
{
    while (TRUE) {
        think();
        take_forks(i);
        eat();
        put_forks(i);
    }
}

```

14. Taking Forks

```

void take_forks(int i)
{
    down(&mutex);
    state[i] = HUNGRY;
    test(i);
    up(&mutex);
    down(&s[i]);
}

```

Meaning

The philosopher marks itself HUNGRY, tests whether both forks are available, then blocks on its own semaphore if it cannot eat yet.

15. Putting Forks

```

void put_forks(int i)
{
    down(&mutex);
    state[i] = THINKING;
    test(LEFT);
    test(RIGHT);
    up(&mutex);
}

```

Meaning

After eating, the philosopher becomes THINKING and checks whether either neighbor can now start eating.

16. The Test Procedure

```

void test(int i)
{
    if (state[i] == HUNGRY &&
        state[LEFT] != EATING &&
        state[RIGHT] != EATING) {
        state[i] = EATING;
        up(&s[i]);
    }
}

```

Meaning

The test procedure grants permission to eat only when philosopher i is hungry and neither neighbor is eating.

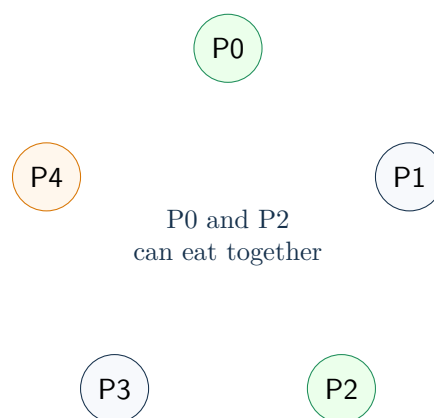
17. Why the Better Solution Works

Property	Reason
No neighboring eaters	A philosopher can eat only if both neighbors are not eating.
No deadlock	Hungry philosophers do not hold one fork while waiting for another. They wait on their own semaphore.
More parallelism	Non-neighboring philosophers can eat at the same time.
Controlled state changes	Shared state is updated while holding mutex.

Maximum Parallelism for Five

With five philosophers, at most two philosophers can eat at the same time, because any third philosopher would be adjacent to one of them.

18. Parallel Eating Example



Observation

P0 and P2 are not neighbors, so they can eat at the same time. P4 cannot eat while P0 is eating because P4 is P0's neighbor.

19. Comparison of Attempts

Attempt	Main Idea	Problem
Take left, then right	Simple fork acquisition.	Can deadlock.
Release and retry	Put down left fork if right fork is unavailable.	Can starve in lockstep.
Random retry	Wait random time before trying again.	Practical, but not a deterministic guarantee.
Single mutex	Only one philosopher enters fork code at a time.	Safe but allows only one eater.
State + semaphores	Eat only when neither neighbor is eating.	More complex, but gives safe parallelism.

20. Big Picture

Main Conclusion

The dining philosophers problem shows that sharing multiple resources is harder than protecting one critical section. A correct solution must avoid deadlock, avoid starvation, and still allow as much safe parallelism as possible.

Exam Shortcut

Remember: taking the left fork then the right fork can deadlock. Releasing and retrying can starve. A single mutex is safe but too restrictive. The state-array semaphore solution lets a philosopher eat only when neither neighbor is eating.

21. Quick Review

Key Points

- Dijkstra introduced the dining philosophers problem in 1965.
- Five philosophers share five forks around a circular table.
- Each philosopher needs both left and right forks to eat.
- Taking the left fork and then the right fork can deadlock.
- Deadlock occurs when everyone waits forever for someone else to release a resource.
- Releasing the left fork and retrying can still cause starvation.
- Random waiting is useful in practice but does not guarantee correctness.
- A single mutex prevents deadlock but allows only one philosopher to eat at a time.
- A better solution tracks THINKING, HUNGRY, and EATING states.
- A philosopher may eat only if neither neighbor is eating.
- One semaphore per philosopher lets hungry philosophers block individually.
- The mutex protects the shared state array.

The Readers and Writers Problem

Sharing a database between many readers and exclusive writers

Definition

The **readers and writers problem** is a synchronization problem in which many processes may safely read shared data at the same time, but a writer must have exclusive access.

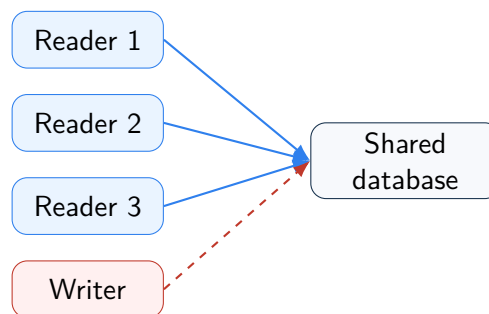
1. Problem Model

Database Access

The problem models systems such as airline reservation databases, where many processes may want to read records and some processes may need to update them.

Access Rule

Multiple readers may access the database simultaneously. If a writer is updating the database, no other process may access it, not even a reader.



2. Core Constraints

Situation	Allowed?
Reader with reader	Yes. Multiple readers can read together.
Writer with reader	No. A writer needs exclusive access.
Writer with writer	No. Only one writer may update at a time.

Main Difficulty

The scheduler must allow reader concurrency without letting readers and writers overlap incorrectly.

3. Semaphores and Counter

Shared Variables

The classic semaphore solution uses two semaphores and a reader count.

Variable	Purpose
<i>mutex</i>	Protects access to the reader count <i>rc</i> .
<i>db</i>	Controls access to the database itself.
<i>rc</i>	Counts how many readers are currently reading or entering.

Why Two Semaphores

mutex protects the bookkeeping variable *rc*. *db* protects the database from writers or from the first reader entering while a writer is active.

4. Reader Algorithm

```
typedef int semaphore;
semaphore mutex = 1; /* controls access to rc */
semaphore db = 1; /* controls access to the database */
int rc = 0; /* number of readers */

void reader(void)
{
    while (TRUE) {
        down(&mutex);
        rc = rc + 1;
        if (rc == 1) down(&db);
        up(&mutex);

        read_database();

        down(&mutex);
        rc = rc - 1;
        if (rc == 0) up(&db);
        up(&mutex);

        use_data_read();
    }
}
```

First Reader Locks Database

The first reader does a down on *db*. This prevents writers from entering while readers are active.

Last Reader Unlocks Database

The last reader does an up on *db*. This allows a waiting writer to enter after all readers are gone.

5. Writer Algorithm

```
void writer(void)
{
    while (TRUE) {
```

```
    think_up_data();  
    down(&db);  
    write_database();  
    up(&db);  
}  
}
```

Writer Rule

A writer simply waits for exclusive access to *db*, writes the database, and then releases *db*.

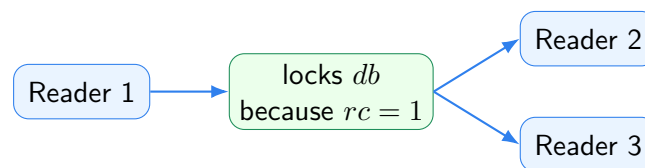
Exclusive Access

While a writer holds *db*, no reader or other writer can access the database.

6. How Readers Enter

Reader Grouping

Once the first reader has locked *db*, later readers do not lock *db* again. They only increment *rc*, read the database, and later decrement *rc*.



Result

Many readers can be inside the database at the same time, but writers are blocked until *rc* becomes 0.

7. Why *mutex* Is Needed

Race on Reader Count

The variable *rc* is shared by all readers. Without *mutex*, two readers could update it at the same time and corrupt the count.

Protected Bookkeeping

Readers hold *mutex* only while changing *rc* and checking whether they are the first or last reader. They do not hold *mutex* while reading the database.

8. Correctness Properties

Property	Reason
Multiple readers allowed	After the first reader locks <i>db</i> , later readers can enter by updating <i>rc</i> .
Writer is exclusive	A writer must hold <i>db</i> , and readers keep <i>db</i> locked while any reader exists.
No writer-writer overlap	The <i>db</i> semaphore allows only one writer to enter.
Reader count is safe	Updates to <i>rc</i> are protected by <i>mutex</i> .

9. Subtle Policy Decision

Reader Preference

The presented solution gives preference to readers. If a reader is active, later readers are allowed to enter even if a writer is already waiting.

Why This Increases Concurrency

Letting new readers join existing readers keeps the database busy and allows many read operations to proceed together.

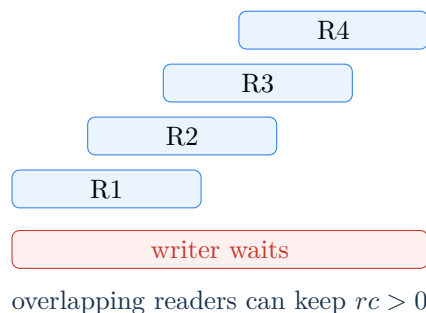
10. Writer Starvation

Starvation Risk

If readers keep arriving while at least one reader is active, a waiting writer may never get access to the database.

Continuous Readers

Suppose a new reader arrives every 2 seconds, and each reader takes 5 seconds to finish. There may always be at least one reader active, so a writer waiting on *db* may never enter.



11. Writer-Priority Alternative

Modified Rule

If a writer is waiting, newly arriving readers can be suspended behind the writer instead of being admitted immediately.

Effect

The writer only waits for readers that were already active when the writer arrived. It does not wait for readers that arrive later.

Tradeoff

Writer priority reduces writer starvation, but it can reduce concurrency because readers may be blocked even though other readers are already reading.

12. Fairness Choices

Policy	Benefit	Cost
Reader preference	High reader concurrency.	Writers may starve.
Writer preference	Writers do not wait behind newly arriving readers.	Reader concurrency can be reduced.
Fair ordering	Reduces starvation for both groups.	More complex to implement.

13. Comparison with Dining Philosophers

Problem	Shared Resource Pattern	Main Lesson
Dining philosophers	Several processes need two exclusive resources.	Avoid deadlock and starvation when acquiring multiple resources.
Readers and writers	Many readers can share, but writers need exclusivity.	Balance concurrency against fairness.

14. Big Picture

Main Conclusion

The readers and writers problem shows how to allow high concurrency for readers while preserving exclusive access for writers. The classic semaphore solution is simple and efficient for readers, but it can starve writers if readers keep arriving.

Exam Shortcut

Remember: *mutex* protects the reader count *rc*, while *db* protects the database. The first reader locks *db*, the last reader unlocks *db*, and writers lock *db* exclusively.

15. Quick Review

Key Points

- The readers and writers problem models shared database access.
- Multiple readers may read at the same time.
- A writer needs exclusive access.
- The semaphore *mutex* protects the reader count *rc*.
- The semaphore *db* protects the database.
- The first reader locks *db*.
- Later readers only increment *rc* and read.
- The last reader unlocks *db*.
- Writers do down on *db*, write, then up on *db*.
- The basic solution gives preference to readers.
- Continuous readers can starve a waiting writer.
- Writer-priority solutions avoid this by blocking new readers behind waiting writers.